OxSteed
Enterprise Operations Audit

3,500 worldwide employees
6,000,000 registered users

Full admin and super-admin review
65 tab reviews + five enterprise management workspaces

Source snapshot: e7ac9415
Audit date: October 2, 2026
Purpose: flaws, expansion risks, operator design, missing controls, and measurable readiness gates.

Decision: healthy observed deployment, but enterprise-scale operating readiness is not yet established. No claim of complete live action testing or legal certification.

Contents

OxSteed enterprise operations audit 6

Decision and purpose 6

Current reality and evidence boundary 6

Concrete defects that should be fixed before expansion 7

Capacity assumptions: users are not concurrent traffic 8

Shared enterprise findings and controls 9

Per-tab audit inventory 13

Regular-admin tab reviews 14

1. Dashboard - /admin/dashboard 14

2. Operations Overview - /admin/operations 16

3. Users - /admin/users 18

4. Jobs - /admin/jobs 20

5. Reports - /admin/reports 22

6. Moderation - /admin/moderation 24

7. Enforcement - /admin/enforcement 26

8. Content Removals - /admin/content-removals 28

9. Skills - /admin/skills 30

10. Support Tickets - /admin/support 32

11. Live Support - /admin/support/chat 34

12. Quote Requests - /admin/quote-requests 36

13. Direct Ops - /direct/admin 38

14. Direct Service Areas - /admin/direct-service-areas 40

15. Direct Catalog - /admin/direct-catalog 42

16. Messages - /admin/messages 44

17. Search - /admin/search 46

Completion and evidence boundary 47

18. Businesses - /admin/businesses 48

19. Tool Rentals - /admin/tool-rentals 50

20. Planned Needs - /admin/planned-needs 52

21. Payments - /admin/payments 54

22. Orders - /admin/orders 56

23. Templates - /admin/templates 58

24. Consulting - /admin/consulting 60

25. Consulting Pay - /admin/consulting/payments 62

26. Scheduling - /admin/consulting/scheduler 64

27. Business Launch - /admin/consulting/business-launch 66

28. Consulting Sessions - /admin/consulting/sessions 68

29. Org Chart - /admin/corporate/org-chart 70

30. Corporate Titles - /admin/corporate/assignments 72

31. Logistics Ops - /admin/logistics 74

32. Learn Submissions - /admin/learn/submissions 76

33. Learn Product Links - /admin/learn/product-links 78

34. Learn AI Suggestions - /admin/learn/ai-suggestions 80

Super-admin tab reviews 82

35. Super Dashboard - /admin/super/dashboard 82

36. Money - /admin/super/money 84

37. Restaurant Credit - /admin/super/restaurant-credit 86

38. OxSteed Credit - /admin/super/oxsteed-credit and all 13 subpages 88

39. Revenue - /admin/super/revenue 91

40. Fee Config - /admin/super/fees 93

41. Pricing Config - /admin/super/pricing 95

42. Pricing Overrides - /admin/super/pricing-overrides 97

43. Financials - /admin/super/financials 99

44. Payouts - /admin/super/payouts 101

45. Conversations - /admin/super/conversations 103

46. Admin Accounts - /admin/super/admin-accounts 105

47. Permissions - /admin/super/permissions 107

48. Regions - /admin/super/regions 109

49. Internal Access - /admin/super/logistics-internal-access 111

50. Gate Lockouts - /admin/super/access-gate-lockouts 113

51. Create User - /admin/super/users/create 115

52. Search Logs - /admin/super/search-logs 117

53. Settings - /admin/super/settings 119

54. Audit Log - /admin/super/audit-log 121

55. User Action Log - /admin/super/audit-log/user-actions 123

56. Legal (all subpages) - /admin/legal/{acceptances,audit-log,compliance-metrics,ccpa-requests} 125

57. Visitors - /admin/super/visitors 127

58. Profile Visibility - /admin/super/profile-visibility 129

59. One Truth - /admin/super/one-truth 131

60. Franchise HQ - /admin/super/franchises 133

61. Careers - /admin/super/careers 135

62. Country Packs - /admin/super/country-packs 137

63. Corporate Setup - /admin/super/corporate/departments 139

64. Mode Policies - /admin/super/mode-policies 141

65. Mode Approvals - /admin/super/mode-approvals 143

Management capabilities required for the target company 145

Management page 1. Workforce Identity, Delegation, and Access Lifecycle 146

Management page 2. Treasury, Entity Accounting, Tax, and Financial Close 148

Management page 3. Global Service Operations, Capacity, and Safety 150

Management page 4. Reliability, Releases, Integration Recovery, and Cost 152

Management page 5. Market Governance, Privacy, Contracts, and Policy Assurance 154

Additional functionality backlog and build-versus-integrate decisions 156

Verification contract: how the report becomes an operable company 158

Data, concurrency, and recovery testing 158

Release and operating gates 159

Ordered delivery program and closure evidence 160

Final certification decision 160

OxSteed enterprise operations audit

Decision and purpose

Target: 3,500 employees worldwide and 6,000,000 registered users. Decision: the current codebase is a substantial product foundation, but enterprise-scale operational readiness is NOT established. This report replaces the earlier short audit rather than extending its stale conclusions. It reviews all 65 current sidebar tabs, their supporting workflows, shared infrastructure, and management capabilities needed to operate the company. Each tab starts on a separate PDF page; longer reviews continue onto subsequent pages. Five additional management-workspace reviews address capabilities missing or fragmented across existing pages.

The requirement is not simply that buttons render. OxSteed must prevent unauthorized decisions, duplicate money movement, unowned customer problems, incorrect regional launches, and outages caused by growth. Employees need a controlled way to do their work without becoming platform-wide super administrators. Executives need reliable business evidence, finance needs reconcilable books, and operators need recovery paths when providers, infrastructure, or people fail.

No audit can honestly guarantee that software will never need another fix. The durable objective is **no known critical gaps before expansion, explicit handling of foreseeable failure scenarios, and continuous detection of new risks**. Applying only design recommendations, without the acceptance tests and operational gates in this report, does not meet that objective. This document is an engineering and operating-model assessment, not a legal opinion or certification of financial licensing in any country.

Current reality and evidence boundary

Source snapshot: oxsteed/old-file, branch old-files-claude, revision e7ac9415936c5450dab071cf0ebf9f1c318ea2d8, reviewed on October 2, 2026. All line references are to that snapshot and must be rechecked after changes. Read-only live verification at 08:55:06 UTC / 04:55:06 EDT found /ready healthy, expected/applied migrations **640/640**, with zero pending or unexpected filenames. Database, Redis, realtime initialization, worker initialization, traffic acceptance, and the enabled credit rail were reported healthy. Release was content-1dd496a985552c72; Git SHA, image digest, and deployment ID in the readiness payload were **null**. This snapshot is not a guarantee of later uptime or provider readiness.

The former database architecture/startup incident is not the current state: production inspection found the ARM64-compatible oxsteed/postgis:16-arm64 database and healthy application, Redis, and ClamAV containers. Do not treat the older exec format error as an unresolved outage. Conversely, fixing it does not remove the physical-host failure domain.

GitHub CI run 36984369426 succeeded for revision e7ac9415. Its server/client verification and fresh PostGIS migration rehearsal passed. The rehearsal applies all migrations twice to a new database. It does NOT demonstrate a populated, production-sized upgrade, mixed-version compatibility, disaster recovery, or six-million-user performance. The authenticated browser step explicitly printed **0 passed, 2 skipped**, because its base URL and account configuration were absent. Green CI is therefore not authenticated operational certification.

The earlier verification round reported 310 server suites passed, 3 skipped; 3,339 tests passed, 46 skipped. Client localization, type checking, and production build passed in that round. Translation scanning still warned about 322 references without catalog keys (46 raw-key risks and 276 fallback references) and 703 long strings identical to English. Warnings are not the same as broken interpolation gates, but they cannot be described as complete human-verified localization. These counts are snapshot evidence, not a new rerun of every suite in this report-writing session.

The source inventory has **34 regular-admin and 31 super-admin sidebar entries: 65, not 64**. There are additional detail and sub-workspace routes. Existing Customer 360, Business 360, delivery dispatch, delivery tracking, incidents, support operations, credit reconciliation, and reconciliation-exception routes must be extended and made discoverable, not replaced by duplicate systems. See client/src/admin/adminRouteManifest.js:18-151, client/src/admin/AdminApp.jsx:230,

client/src/admin/AdminApp.jsx:252, client/src/pages/DeliveryDispatchPage.jsx, and client/src/pages/DeliveryTrackingOpsPage.jsx.

Reading findings correctly

- **Confirmed defect:** a specific code behavior contradicts the intended contract, with evidence and a reproducible acceptance test. A static defect does not imply observed production exploitation.
- **Code observation / scale risk:** the implementation is visible, but its harmful outcome under expansion is not yet measured. Validate before choosing a remedy.
- **Verification gap:** necessary evidence is missing; do not convert absence of proof into a passing status or an invented defect.
- **Design requirement:** a capability needed for the target operating model, possibly covered partially elsewhere. Check existing implementations before adding another service.

P0 means an immediate security, integrity, or service-continuity blocker. P1 means a release/expansion blocker for the affected business operation. P2 means an important control or usability deficiency that needs a committed owner and milestone. P3 means lower-risk polish. In tab reviews using qualitative ratings, High corresponds to a material P1 expansion/control risk and Medium to P2 unless a specific priority is stated. A P1 scale gate does not mean the small current deployment is necessarily failing today. Findings use E identifiers for shared enterprise controls, TNN-FNN for tab-specific issues, and M identifiers for management workspaces. Fix shared foundations once and apply them consistently.

Concrete defects that should be fixed before expansion

The tab reviews are not only a list of hypothetical enterprise features. Source inspection identifies present contract, lifecycle, and authorization defects. The following are priority entry points; the complete numbered reviews contain further findings, evidence, owners, and acceptance criteria. They were not exercised against production money or customer records in this audit.

- **T22-F01 - Orders shipping/cancellation:** server/controllers/adminOrdersController.js:229,339 calls withTransaction(callback), but server/utils/transaction.js:39-40 requires withTransaction(pool, operation) and calls pool.connect(). These mutation paths cannot use that helper correctly. Order operations/payment engineering must fix the call contract and test success, rollback, retries, and safe financial handoff with an actual isolated database.
- **T12-F02 - Quote currency corruption on unrelated edits:** server/controllers/quoteRequestsController.js:631,816 defaults omitted currency to USD and writes it while a notes/status-only client edit omits currency. Preserve omitted fields and validate explicit monetary changes. Test a non-USD quote through notes/status changes, including approval/history, without altering its currency or price.
- **T20-F01/F02 - Planned work can publish after administrative cancellation:** cancellation does not establish the durable pause expected by scheduling; the scheduler selects candidates before its per-row transaction and rereads only ID/published linkage, not current pause/status (server/jobs/plannedNeedsScheduler.js:32-79). Domain scheduling must revalidate eligibility under lock and test cancellation between selection and publication.
- **T10-F01 - Support ticket ownership race:** claim logic reads/checks and then updates by ID, allowing competing operators to overwrite ownership. Support engineering must use an atomic owner/version transition and return a conflict to the losing claimant, preserving the existing stronger live-conversation ownership patterns.
- **T11-F01 / T12-F01 / T23-F01 / T25-F01 - Inconsistent privileged API gates:** live support, quote detail, template grants, and consulting financial actions have role-only or different-grant paths compared with neighboring admin endpoints. Authentication exists; these are least-privilege inconsistencies, not claims of public anonymous access. Security/IAM and each domain must enforce the same scope/action authority through direct APIs, not just menus.
- **T17-F01 - Federated search crosses domain permission boundaries:** the search engine returns requested/default message and payment data under search-log permission without the equivalent

message/finance/participant restrictions (server/controllers/adminSearchController.js:27-30,105-118,153-179). Scope permitted entity types and fields before querying; a search permission must not become alternate access to private messages or financial records.

- **T18-F01 - Business deletion UI/API mismatch:** client/src/admin/pages/BusinessModeration.jsx:274-281 sends {reason}, but server/controllers/businessModerationController.js:430-436 requires confirmed. Align explicit confirmation and test the real client-to-controller contract without removing authorization or lifecycle safeguards.
- **T35-F01 - Super Dashboard completion rate:** server/controllers/superAdminController.js:235-239,356-358 combines period completions with lifetime cancellations. Data/operations owners must define and calculate one consistent cohort; old cancellations must not alter a seven-day result merely because the platform grew.
- **T36-F01 / T37-F01 - Currency-ambiguous management totals:** Money and Restaurant Credit overview boundaries aggregate native values without an adequate per-currency reporting contract. This is a reporting problem, not proof that isolated wallets can spend across restaurants. Finance must prevent unqualified mixed-currency sums and approve any reporting FX snapshot.
- **T05-F01 / T10-F02 / T38-F02 - Incomplete operational queues:** paginated server results are not fully reachable from some report/support/compliance screens. An empty filtered page or the first 25/50 results does not establish a cleared workload. Domain owners must expose complete scoped navigation, counts, age, and assignment, verified with records beyond the first page.
- **T56-F01/F04 - Legal assurance is not durable publication:** legal count-query parameter slicing is inconsistent with the bound filters (server/controllers/legalDocumentsController.js:261-266), while document drafts/approval/live labels are held in a process-local Map (server/routes/legalDocuments.js:13,148,196,208,222). Persist reviewed document versions and connect publication to the real public content and acceptance version; a restart or second node must not erase or disagree on an approved document.
- **T58-F01 - Profile-visibility review lock is not held across the decision:** server/services/profileVisibilityApprovals.js:257-284 issues a pooled SELECT ... FOR UPDATE and later an unconditional update without one explicit transaction. Independent reviewers can race. Use one transactional/versioned decision and execute authorized effects idempotently; retain the existing maker/checker restriction.
- **T59-F01 - One Truth pagination does not bound work:** server/controllers/oneTruthController.js:693-711 loads/enriches all scoped users and builds suspect pairs before slicing the page. At six million users this synchronous request requires redesign as a bounded indexed/queued investigation with safe progress, not a larger page limit. Confirm algorithm cost and memory against realistic distributions.

The highest-value next deliverable is an implementation backlog keyed to these findings, with failing isolated reproductions before fixes, rather than another cosmetic dashboard refresh. High-severity findings must not be closed solely by adding a frontend confirmation dialog, increasing a query limit, or assigning more super-admin accounts.

Capacity assumptions: users are not concurrent traffic

Six million registered users is a data-size requirement, not a request-rate specification. Use the following **planning assumptions**, not claimed measurements: 20% daily active users means 1.2 million daily users; 60 dynamic requests each means 72 million requests/day, roughly 833 requests/second averaged over 24 hours. A ten-times peak is approximately 8,333 requests/second. A first design test target is 10,000 dynamic requests/second at the representative mix, plus separate media/CDN traffic. Analytics, retries, webhooks, background jobs, and internal API calls must be measured and added rather than hidden in that figure.

Assume 10% of daily users can hold concurrent sockets: 120,000 connections; exercise 240,000 during reconnect/failover stress. This is a test hypothesis, not a fact about usage. Assume 1,500 simultaneously active employees out of 3,500 and test at least 500 admin requests/second during operational incidents. Admin analytical workloads must not steal customer booking or payment capacity. For transactional

scenarios, create realistic distributions: active/inactive users, large businesses, hot restaurants, shared drivers, popular helpers, skewed geographic demand, long support histories, and high audit volumes. Six million trivial rows with no relationships is not a meaningful scale fixture.

Proposed initial service objectives, to be approved with budget and product promises: 99.95% monthly availability for critical transaction APIs; database-backed ordinary list/read p95 at or below 500 ms; admin interactive p95 at or below one second for bounded queries; internal committed-event propagation p95 under five seconds; privileged account revocation across HTTP and sockets within 60 seconds; critical financial loss recovery point under five minutes and service recovery under 60 minutes. Financial correctness requires zero duplicate captures/payouts and no unaccounted ledger imbalance; latency averages cannot waive that invariant. External provider response times must be reported separately. Different tiers, countries, and safety-critical operations can need stricter objectives.

Reassess these numbers from telemetry before buying infrastructure. The report does not recommend moving everything to a microservice architecture or immediately running six million users through every endpoint. First isolate workloads, measure bottlenecks, and preserve authoritative domain transactions.

Shared enterprise findings and controls

E01 - P1 expansion blocker: one physical production failure domain

Code observation and live infrastructure evidence. README.md:109-122 documents the production MacBook/Colima model; docker-compose.coolify.yml:2-16,114-118 defines the app with local persistent paths and no high-availability deployment topology. Live inspection places application dependencies on that host. A power loss, disk failure, sleep policy, host upgrade, network loss, or Colima failure can affect the whole business. R2 object storage does not make the compute/database tier highly available.

SRE must establish a supported production topology with at least independent compute failure domains, supervised database failover, shared durable storage, documented network dependencies, and an independent recovery environment. A transition can keep the Mac as a development/recovery node; it should not remain the sole worldwide production dependency at the target size. Use tested infrastructure-as-code and architecture-compatible, immutable images. Acceptance: remove a compute host under representative load without losing committed transactions; restore routing and queues within the approved recovery objective; demonstrate that the operator console and incident channel remain reachable independently of the failed application.

E02 - P1 expansion blocker: realtime state is process-local

Code observation. server/bootstrap/realtime.js:29-40 initializes Socket.IO without a shared adapter in that bootstrap. server/services/websocketService.js:46-49 keeps customer connections, agent connections, ticket sockets, and viewers in process-local Maps. Simply adding HTTP replicas can split chat presence, support routing, and fanout across workers.

Adopt a supported cross-node broadcast adapter and an explicit shared presence/assignment contract, including connection ownership, expiration, authorization refresh, and reconnect state reload. An adapter alone does not distribute arbitrary application Maps. If HTTP long-polling stays enabled, configure and test affinity; Socket.IO's multi-node guidance explains the adapter and affinity requirements. Acceptance: users attached to different nodes exchange updates; node termination and mass reconnect do not lose durable messages, duplicate case assignments, or expose a prior participant's room. Broadcast only authorized projections, never globally cached sensitive payloads.

E03 - P1 expansion blocker: worker ownership and lease-loss behavior

Code observation / verification gap. server/bootstrap/backgroundWorkers.js:71-197 starts many scheduled and delivery workers with each app instance. Existing job locks are valuable, not absent: server/jobs/jobLockingService.js:17-25,55-100 acquires/refreshes leases. However, its wrapper logs renewal errors while the handler continues (85-86). A lease can expire and another worker can start unless the domain action has its own idempotency/fencing safeguards.

Separate web and worker capacity, inventory each consumer, declare concurrency and lease rules, and require durable event IDs and transaction-level idempotency for irreversible effects. Long jobs must detect

lease loss and fence unsafe writes, or use a queue protocol that safely owns work until completion. Do not stop all work blindly if an already-committed operation needs reconciliation. Acceptance: pause a worker beyond its lease, restart it after a competitor finishes, replay messages in duplicate/reverse order, and prove one financial/stock/status effect with a retained reconciliation trail. Monitor oldest actionable queue age and consumer heartbeat, not process uptime alone.

E04 - P1 expansion blocker: database budget and admin query isolation

Code observation / verification gap. server/db.js:17-23 defaults the connection pool to ten, caps configured size at 100, and has connection/idle timeouts; this pool declaration does not set a universal statement timeout. docker-compose.coolify.yml:47 defaults DB_POOL_MAX to ten. Multiplying replicas and workers multiplies connections. Raising every pool to 100 is not a capacity plan.

Define database connection budgets per workload, query/lock deadlines, cancellation, bounded pagination, indexes validated by EXPLAIN on representative skew, retention/partition strategy for high-volume events, and asynchronous exports. Use replicas/analytical pipelines for reports with visible lag, while approvals and finance read authoritative state. Consider a pooler only after testing transaction semantics, session features, and migration behavior. Acceptance: six-million-user fixtures, large audit/history tables, and simultaneous admin exports cannot starve quote acceptance, bookings, support, or payment verification; saturation returns explicit retryable states rather than misleading zeros.

E05 - P1 expansion blocker: global permission model is not a worldwide workforce model

Code observation / design requirement. server/middleware/adminAuth.js:126-177 grants super-admin bypass and intentionally rejects region-specific grants for existing platform-wide delegated endpoints. That fail-closed behavior should stay until actual scoped handlers are implemented. The comment explicitly says current delegated endpoints are platform-wide. It is not enough to let a regional grant pass and then execute an unrestricted query.

Implement employee role bundles, legal-entity/region/team/object scopes, action limits, restricted reads, and separation of duties. Scope every list, detail, export, websocket subscription, approval, and bulk job on the server. Retain narrowly controlled break-glass access with reason, expiry, and independent review. client/src/admin/lib/GrantGuard.jsx:39,85-88,109-126 uses a five-minute cache and role bypass; navigation caching is a convenience, never the authority. Acceptance: a regional operator cannot access another region by altering IDs, filters, export parameters, or socket rooms; revocation reaches existing sessions; a maker cannot approve their own high-risk change, including via a second browser.

E06 - P1 verification blocker: green browser step means skipped coverage

Confirmed verification gap. .github/workflows/ci.yml:52-60 supplies optional environment configuration to the browser script. client/scripts/adminAuthenticated.browser-check.cjs:15-30,90,112,141-142 skips when unconfigured and excludes dynamic record paths from the static route sweep. Actual run 36984369426 confirms the skip. Its smoke assertions are not an exhaustive role/action suite.

Create an isolated, production-like staging environment with deterministic seeded staff/customer/business/driver accounts, provider test fixtures, mandatory role matrix, and resettable domain data. Required release coverage must fail closed when credentials, seeds, or browser infrastructure are missing. Test success, denial, conflict, stale-state, and outage paths. Do not install permanent production test identities that can move real money. Acceptance: report exact tests executed and skipped for all 65 tabs plus supporting detail routes, Chromium/Firefox/WebKit, both themes, mobile, RTL, five languages, and critical mutating journeys.

E07 - P1 release-control gap: build/deploy identity and promotion evidence

Code observation and live evidence. Dockerfile:34-36 runs the ordinary client build, while CI separately runs build:ci. Native Coolify webhook deployment can be independent of CI promotion. Healthy readiness currently reports null git/image/deployment metadata. This does not prove every failed commit is deployed, but it means the audited safety gate and deployed artifact identity are not demonstrated end to end.

Promote an immutable artifact only after mandatory checks pass; bind revision, image digest, schema contract, feature configuration, and deployment ID into a signed/reviewable release record. Add canary traffic, rollback health criteria, migration compatibility, and change approval for financial/configuration changes. Readiness should expose safe identifiers without secrets. Acceptance: a deliberately failing commit cannot become production; operators can map a customer error to one exact artifact/configuration version; rollback restores the prior supported app without reversing irreversible business transactions.

E08 - P1 integrity/recovery gate: backups must restore a complete business

Code observation / verification gap. scripts/backup-coolify-production.sh:15-23,85-104 creates host-bound database/configuration artifacts and optional mirroring. Its manifest does not itself establish complete upload coverage, independently controlled immutable storage, or recoverability after loss of the host. scripts/verify-production-restore.sh:40-55 rehearses a temporary database inside the existing production database container: useful, but not an independent disaster restore and potentially competing for resources.

Inventory database, object media, agreements, proofs, provider mappings, jobs/outbox, configurations, and required recovery credentials. Prove encrypted off-host copies, access separation, retention, deletion protection where supported, and independently recoverable access. Treat secrets in configuration backups as highly privileged material without exposing them in reports. Logical dumps remain useful; low-loss recovery needs a tested design such as physical base backups plus archived WAL, which is distinct from pg_dump according to PostgreSQL 16 PITR documentation. Acceptance: recover on a fresh host with production network unavailable, verify balances/relationships/media, reconcile provider outcomes before resuming workers, and record actual recovery point/time.

E09 - P1 observability gap: readiness is not continuous business health

Code observation. server/services/readinessService.js:45-58 compares migration filenames; this predicate alone does not verify execution status/checksums. Startup migration checks may add protection, so do not equate this narrow observation to an observed corrupt schema. 93-127 uses runtime initialization flags for workers/realtime. A ready process can still have an old queue, stale subscription, incorrect audit chain, or delayed payout.

Keep liveness, critical traffic readiness, deployment schema verification, and business-health panels distinct. Add consumer heartbeat/lag, provider freshness, reconciliation variance, support SLA, stale data indicators, upload scanner saturation, incident ownership, and correlation IDs. Optional-provider outages should degrade that workflow rather than unnecessarily kill the entire site. Acceptance: inject failed/stalled consumers, partially applied schema markers, and provider revocation; the right alert fires with actionable references while unrelated safe workflows stay usable. Unknown values must never become zero or green.

E10 - P1 worldwide operation gap: entity, currency, time, and policy boundaries

Design requirement requiring cross-module verification. Country Packs, Regions, pricing, legal acceptances, and financial services already exist. Their existence is not approval to operate every financial/delivery product everywhere. Record legal entity, country/service area, currency, timezone, policy version, tax treatment, and permitted payment rail on authoritative transactions where applicable. Keep restricted OxSteed Credit and restaurant-specific credit distinct.

Finance, legal, operations, and product owners need a per-country launch checklist and evidence approvals. Currency changes must not rewrite historical transactions; time windows need local timezone plus unambiguous stored instants; daylight-saving, all-day, and cross-border scenarios require explicit handling. Acceptance: an unapproved country/product combination is blocked server-side; customer receipts, operational schedules, and finance reports preserve original currency/entity/policy even after profile changes. This report does not certify licenses, insurance, employment classification, or tax registration: qualified owners must determine those requirements.

E11 - P1 integrity requirement: unified work visibility without a shadow ledger

Design requirement. Customer/Business 360, support, work contexts, delivery, and financial controls are already present. Integrate their explicit record references instead of inferring related work from names or copying balances into a new admin ledger. Every status label must identify whether it is proposed, approved, committed, disputed, or externally pending.

Require immutable correlation references, durable domain events, transactionally saved changes/events, idempotent consumers, per-consumer delivery evidence, and safe replay. Payments/stock/support/authorization services remain authoritative. Provider webhooks must tolerate duplicate and unordered delivery; Stripe's webhook guidance supports verifying signatures and handling such delivery semantics. Acceptance: replay an approved extra, reversed payment event, stock adjustment, and delivery completion without duplicate money/inventory effects; show the true next action and one timeline across permitted views.

E12 - P2 design debt with P1 high-risk consequences: operator consistency

Design requirement with tab-specific evidence below. Establish one admin design contract: server-bounded data grids, URL-persisted filters, accessible saved views, explicit selection scopes, timezone/currency labels, authoritative refresh, stale-data badges, and per-row error recovery. Financial/destructive bulk actions need preview, affected counts, excluded records, approval limits, progress, resumability, cancellation semantics, and an audit receipt. Never label a retry button as safe until its endpoint is idempotent.

Use task-based workspaces rather than giving 3,500 employees an undifferentiated 65-tab menu. Preserve deep links and role-specific queues. Apply keyboard focus, non-color status cues, clear destructive separation, screen-reader announcements, and Arabic RTL without reversing identifiers. WCAG 2.2 is the accessibility reference; this is a target, not a passed conformance audit. Acceptance: representative operators complete their five common tasks without guessing routes or consulting a super-admin, and disabled/unavailable conditions say why and what to do next.

E13 - P1 privacy/security requirement: controlled investigation and evidence access

Design requirement. Users, Conversations, Search Logs, Visitors, audit pages, profiles, and support can collectively reveal highly sensitive histories. Admin login and MFA are already checked (server/middleware/adminAuth.js:43-71). Extend that foundation with least-privilege field projections, time-limited sensitive access, reason/case linkage, export approval, secure evidence storage, retention/legal-hold rules, and access-audit review. An audit hash chain is not a substitute for independent storage and separation from database administrators.

Publication-time supply-chain evidence: GitHub's push response on October 2 reported **31 dependency vulnerabilities on the repository's default branch: 18 high and 13 moderate**. The repository dependency-alert console is the source for triage. This is not a fresh dependency scan of e7ac9415, nor proof that every alert is exploitable or present in the deployed artifact. Security/engineering must map alerts to the audited lockfiles and production dependency tree, determine exposure, apply/test safe upgrades or documented mitigations, and record accepted residual risk. A green CI run must not substitute for that triage; applicable unmitigated high-risk findings are expansion blockers.

Use OWASP ASVS as a structured application verification reference. Test object authorization, session theft/revocation, CSRF where relevant, unsafe uploads, injection, SSRF, export leakage, and denial-of-service against this implementation. Acceptance: a support agent sees only necessary cases; a financial analyst cannot retrieve private message bodies through related-record APIs; sensitive access is attributable and reviewable; privacy removal preserves legally required financial evidence under the approved retention policy.

E14 - P1 continuity requirement: provider, configuration, and AI failure handling

Design requirement. Stripe, maps/routing, email, SMS, storage, scanner, external calendars, AI, and trackers have separate outages, quotas, permissions, and version lifecycles. Build an inventory with owners, connectivity health, least-privilege setup, rotation/expiry procedures, outbound deadlines, retries, circuit breaking, budget alarms, and explicit unavailable states. External AI output must remain advisory for moderation, pricing, finance, and access unless a narrowly approved deterministic action is separately authorized.

Acceptance: every dependency can be unavailable without losing committed requests or fabricating outcomes; critical secrets/configurations can rotate through a tested procedure; rate limits do not cause unbounded retry storms; spend limits and degraded modes are visible. No report should expose API keys, tokens, passwords, or full configuration backup contents. All providers remain subject to their actual approved country/product capabilities.

E15 - P1 operating-model requirement: accountable closure, not a collection of tabs

Design requirement. Give each domain a business owner, technical owner, trained operator role, on-call escalation, runbook, SLO, and failure backlog. Define handoffs between support, fraud, finance, delivery, security, and engineering. Add immutable approval decisions, queue assignment/claiming, escalation deadlines, business-calendar rules, and follow-the-sun handover. A globally visible queue with no accountable team is not managed work.

Acceptance: for every lifecycle in the final matrix, a failed, abandoned, conflicted, or overdue record lands in an owned actionable queue; its authorized operator can resolve it or escalate without direct SQL or undocumented super-admin intervention. Every tab finding is closed with evidence, explicitly deferred with an exposure limit, or recorded as a release blocker. An unresolved P0/P1 is not renamed a future enhancement to get a green launch.

Per-tab audit inventory

The following reviews cover all 65 sidebar tabs. The numbered tab headings are the coverage manifest for this document. Supporting detail/sub-workspace routes are included in the relevant tab review and must also be represented in acceptance tests. No tab is certified simply because its route exists. Tab owners must resolve references to shared findings rather than implementing duplicate remedies independently.

Regular-admin tab reviews

1. Dashboard - /admin/dashboard

Mission / what exists

The dashboard gives leadership and shift operators a platform summary and entry points into existing workflows. client/src/admin/pages/super/Dashboard.jsx:138 serves both regular and super administrators. Regular admins call /admin/dashboard; super admins receive period-based statistics plus activity, geographic summaries, health, and security incidents. Abortable fetches, partial auxiliary loading, incident acknowledgment, and resolution with an entered note already exist. Financial and geographic panels are not evidence of a missing dashboard. The enterprise question is whether those summaries remain correctly scoped, interpretable, and inexpensive when many regional supervisors use them simultaneously.

Evidence

UI: client/src/admin/pages/super/Dashboard.jsx:149, :159, :198, :211. API: server/routes/admin.js:65, :161; statistics: server/controllers/adminController.js:19, server/controllers/superAdminController.js:172. Authorization: server/middleware/adminAuth.js:126, :169. Existing regression contracts: client/scripts/adminRouteContracts.test.mjs:161 and server/__tests__/adminPermissionScope.test.js:23.

Findings

- **T01-F01 - High [design requirement]: regional supervisors cannot obtain a scoped dashboard through the current global grant contract.** The regular endpoint requires manage_operations; requirePermission deliberately requires an unscoped grant. This is a protective boundary, not a demonstrated regional data leak. Super dashboard endpoints require super-admin status. Delegating a global grant simply to let a country manager see local totals creates excessive authority. Add a distinct scoped summary contract that resolves allowed regions and organizational responsibilities on the server, with an explicit global-view privilege.
- **T01-F02 - High [code observation]: bounded visual output hides potentially expensive aggregate work.** Super statistics execute multiple counts over users, helper profiles, and jobs (superAdminController.js:190, :216, :233); regular statistics use several scalar counts (adminController.js:24). A card count is small but its SQL work can grow with the entire population. No load measurement was obtained. Use shared, timestamped aggregates and incremental counters where accuracy permits; verify query plans rather than asserting that every count lacks an index.
- **T01-F03 - Medium [code observation]: failed auxiliary sources can resemble empty business activity.** Settled activity and geo failures become empty arrays (Dashboard.jsx:181); failed core statistics can become an empty object (superAdminController.js:199). The incident pane has an explicit error, which is a useful existing pattern. Apply that pattern to every source so an unavailable geographic feed cannot be interpreted as zero regional activity.
- **T01-F04 - High [verification gap]: release health lacks complete self-reported provenance.** The supplied healthy /ready snapshot has null SHA, digest, and deployment ID. server/utils/release.js:53 already resolves these fields. Populate and validate this existing mechanism; do not mislabel the deployed image as unknown merely because these fields are null.

Expansion scenario

At a follow-the-sun handoff, supervisors in Montréal, Paris, and Cairo open the dashboard while a global incident commander compares regions. A delayed aggregate and a failed geo request must not produce an apparent drop in demand. The country manager should see local backlog without receiving customer-management, financial-recovery, or worldwide settings rights. Leadership also needs to distinguish the source timestamp from the time their browser last refreshed, particularly during deployment and recovery.

Exact recommended controls and UX

Show a persistent scope chip, source timestamp, measurement timezone, and per-card available/stale/unavailable state. Resolve scope before aggregation, not after returning global records. Cache equivalent authorized summaries by scope and period with request coalescing. Keep acknowledgment and resolution in the existing incident system; link every exception to its authoritative record. Display observed image provenance alongside readiness, including an explicit incomplete-metadata warning that does not imply liveness failure. Define metric semantics for "active," "suspended," and period boundaries with named owners.

Dependencies / owner

Platform Operations owns metric definitions and incident workflows; IAM owns scoped grants; Data Engineering owns aggregates; SRE owns provenance and capacity evidence. Regional dashboards depend on the existing regions/permission system, not a new role per city.

Measurable acceptance criteria

With six million synthetic users, realistic job histories, and 200 concurrent authorized dashboard viewers, p95 response time is below two seconds and DB pool wait remains below 100 ms. Repeated refreshes share aggregate work. A region-only operator cannot retrieve global totals by URL or API. Injected geo/query failures display unavailable, never a fabricated zero. Every deployed instance reports the expected SHA, deployment ID, and independently identified image digest; authenticated keyboard and Arabic-layout checks execute without skips.

2. Operations Overview - /admin/operations

Mission / what exists

Operations Overview is an implemented cross-domain command center, not an absent product page. It summarizes delivery, jobs, food orders, support, payments, disputes, queues, integrations, Credit, SLA, risk, privacy/export work, change control, observability, and CRM. Cards point toward existing authoritative modules and carry correlation IDs. Financial, settings, audit, and Credit sections apply additional checks. The client refreshes every 30 seconds only while visible and aborts stale fetches. This is a useful foundation for worldwide incident management; its remaining challenge is delivering actionable, scoped signals without repeatedly aggregating all operational tables.

Evidence

UI polling and cards: client/src/admin/pages/OperationsOverview.jsx:177, :193, :219, :262. Route permission: server/routes/admin.js:68. Source summaries and gates: server/controllers/adminOperationsController.js:15, :26, :59, :114, :160, :192. Tests: server/__tests__/adminOperationsController.test.js:26, :61, :85. Shared DB pool default/cap: server/db.js:17.

Findings

- **T02-F01 - High [code observation]: the overview multiplies metadata probes and aggregate queries per viewer.** statusSummary and countTable first query information_schema, then count records; fifteen sections run concurrently, with nested parallel queries. LIMIT 50 bounds returned statuses, not rows examined by GROUP BY. At 200 visible viewers, 30-second polling alone averages approximately 6.7 overview requests per second, before manual refreshes. This arithmetic is a scenario, not an observed outage. Cache schema capability information and coalesce shared scope summaries with explicit freshness budgets.
- **T02-F02 - High [design requirement]: command-center access is global and its nonfinancial sections lack regional variants.** The route requires global manage_operations. Additional gates correctly deny regional financial grants; the tests verify that safeguard. Delivery, support, risk, and CRM still need server-side scope-aware aggregates for local managers. Separate region-scoped observation from platform-wide intervention, with auditable emergency access rather than routine super-admin promotion.
- **T02-F03 - Medium [code observation]: status totals do not convey incident ownership or queue urgency.** The support card groups statuses; the SLA card counts breaches; observability counts failed events/webhooks (adminOperationsController.js:125, :160, :192). These are real signals, but they lack oldest-item age, current owner, impact region, and remediation deadline in this response. Add those projections from existing incident, SLA, delivery-recovery, and routing records, without copying their mutable state into a second command-center database.
- **T02-F04 - Medium [verification gap]: provider connection counts do not prove delivery recovery.** Connected integrations and pending webhook counts (:144) describe stored state, not a provider round trip or reconciliation result. The healthy readiness snapshot proves dependencies were ready at 08:55 UTC, not that every optional integration or country-specific provider was usable.

Expansion scenario

A European webhook backlog develops while North American support volumes spike and Credit reconciliation is healthy. A shift lead must identify the oldest failed delivery, its tenant, and the accountable team without opening fifteen modules or seeing unrelated customer data. An incident commander should compare global impact and delegate local work. If 200 operators refresh simultaneously during this incident, observability reads must leave database and worker headroom for the actual recovery operations.

Exact recommended controls and UX

Keep existing section isolation, correlation IDs, and hidden-tab polling behavior. Add scope selection constrained by server grants, source-generated timestamps, maximum-age indicators, and clickable oldest exceptions. Poll with jitter and shared caching; favor event-triggered invalidation for acute incidents. Expose provider capability states separately from connected-record counts and distinguish unsupported, unconfigured, degraded, and healthy. Place owner, acknowledgment, due time, and last recovery result beside exceptional counts. Extend the existing incident workflow for assignment and escalation; retain domain pages as the write authority.

Dependencies / owner

SRE owns query budgets and polling; domain teams own exception definitions; Support Operations owns SLA projections; IAM owns region/tenant scope; Integration Operations owns provider evidence. Read models must preserve Credit visibility restrictions already implemented.

Measurable acceptance criteria

With 200 overview viewers, one summary refresh performs bounded shared work per scope, p95 remains below two seconds, and unrelated transactional p95 degrades by less than 10%. Injected source failure affects only its card and retains a traceable correlation ID. Country operators cannot query other regions or restricted finance sections. Every open exception exposes owner and deadline or explicitly identifies itself as unassigned. Demonstrate one webhook/provider outage and successful replay in an isolated environment; publish the resulting evidence independently of a green /ready response.

3. Users - /admin/users

Mission / what exists

Users is a substantial administration interface with search, role/plan/status/location filters, sorting, page sizes, CSV export, profile/detail links, messaging entry points, and privileged account actions. The quick-create modal uses the full account-creation endpoint. Password recovery, deletion scheduling/restoration, impersonation, verification, notes, and richer detail views already exist elsewhere in the same control plane. The standard list excludes admin/system identities. Enterprise readiness requires making this existing lifecycle scalable and delegable while reducing routine exposure of six million customers' identifying and financial information.

Evidence

UI: client/src/admin/pages/UsersList.jsx:19, :46, :99, :185; URL/filter contracts: client/scripts/adminRouteContracts.test.mjs:22. List SQL: server/controllers/superAdminController.js:536, :585, :616, :632. Routes: server/routes/admin.js:137, :181, :196; regional support: server/middleware/regionAuth.js:119, :163. Export: superAdminController.js:2427. Existing safe deletion documentation in code begins at :2475.

Findings

- **T03-F01 - High [design requirement]: a global customer-management grant is too broad for routine country support.** List/detail/name/ban endpoints use global manage_users. Region matching already supports country boundaries (regionAuth.js:66), but those standard endpoints do not use record-scoped authorization. Do not claim regional grants bypass the middleware: they are denied. Add narrowly scoped read, contact, status, verification, and recovery rights with explicit tenant/region predicates and field masks. Retain super-only role changes and impersonation as privileged workflows.
- **T03-F02 - High [code observation]: list refreshes include population-wide summaries and per-user business calculations.** Rows include correlated job counts and payout sums (superAdminController.js:608); summary counts ignore current filters (:632). Search uses several substring predicates, and offset sorting lacks a unique tie-breaker (:553, :621). The bounded 100-row cap is valuable but does not establish acceptable cost at deep pages or six million users. Separate heavy financial/customer detail from routine identity lookup and benchmark representative queries before choosing indexes.
- **T03-F03 - Medium [code observation]: exports are synchronous, capped, and disconnected from list filters.** exportData selects the newest 10,000 rows (:2436) and builds the CSV in memory. It already uses no-store headers, formula escaping, auditing, and a sensitive-operation limiter; those controls should remain. An enterprise investigator needing a complete filtered cohort needs a durable export job and manifest, not an apparently complete partial file.
- **T03-F04 - Medium [code observation]: custom account modals lack full dialog semantics.** The create/reset overlays use plain container elements and backdrop clicks (UsersList.jsx:120, :203). Close buttons have labels, but these inspected structures do not themselves provide a dialog role, focus trap, or focus return. This is a concrete accessibility implementation gap; end-to-end assistive-technology behavior remains unverified.

Expansion scenario

A regional fraud team investigates 80,000 accounts after a credential-abuse campaign while ordinary agents recover customer access. The investigator needs a reproducible filtered export, but a helpdesk agent should not gain financial totals, global bans, or impersonation. Two operators working the same user need a conflict-aware status screen and authoritative history. A keyboard-only supervisor must be able to open, complete, cancel, and leave every recovery modal without losing focus or editing the wrong identity.

Exact recommended controls and UX

Show scope and redacted identity fields until a purpose-based reveal is authorized and logged. Use cursor pagination with (sort_value, id) tie-breakers and separate scoped aggregate summaries. Add filter-preserving asynchronous exports with requester ownership, row counts, checksum, expiry, cancellation, and audit reference. Expand existing deletion and recovery guards rather than adding another account store. Prefer invite/reset links and approved recovery workflows over routine staff-entered passwords; policy and jurisdiction decisions remain with Security and Legal. Use shared accessible dialogs with identity confirmation and conflict refresh.

Dependencies / owner

Identity Engineering and Security own customer lifecycle and privileged recovery; IAM owns workforce delegation; Data Engineering owns query/export capacity; Accessibility and localization owners validate the UI. Financial fields require the existing finance permission model.

Measurable acceptance criteria

Six-million-user lookup and cursor navigation meet p95 below 1.5 seconds with 250 concurrent agents. Regional negative-access tests cover lists, details, counts, exports, and mutations. An 80,000-row export has complete filter metadata and resumes after a worker restart without duplication. Stale status changes return an actionable conflict. All recovery dialogs pass keyboard focus/return and screen-reader checks across five supported languages; no credentials appear in audit logs or artifacts.

4. Jobs - /admin/jobs

Mission / what exists

Jobs provides an implemented searchable, status-filtered list, detail links, pagination, export, and separately granted hard deletion. Abortable loading retains earlier rows after a failed refresh. Server lists use bounded pagination and denormalized category names. Job actions include cancel/remove/flag/unflag; deletion requires a reason and allows only selected statuses, with a status predicate that detects concurrent changes. These existing safeguards deserve credit. The readiness concern is safe intervention across millions of historical jobs, especially where cancellation intersects accepted work, escrow, disputes, and audit durability.

Evidence

UI: client/src/admin/pages/JobsList.jsx:34, :49, :67, :80, :218; detail route: client/src/admin/AdminApp.jsx:232. API grants: server/routes/admin.js:75. List/action/deletion: server/controllers/superAdminController.js:1029, :1045, :1088. Payment relationship: server/migrations/014_phase4_create_payments.sql:7. Existing state/modal regression: client/scripts/adminJobsList.test.mjs:11; pagination helper: server/utils/pagination.js:8.

Findings

- **T04-F01 - High [code observation]: administrative cancellation directly changes job status without a financial recovery workflow in this handler.** forceJobAction updates any noncancelled job to cancelled (superAdminController.js:1056), rather than limiting the source state or calling a lifecycle/escrow coordinator. The reason is optional on the API. Other payment/recovery systems exist; this observation does not prove all funds are lost or all consumers ignore the change. Require a dependency-aware action plan that distinguishes unpublished content, accepted work, in-progress service, completed history, and disputes before committing cancellation.
- **T04-F02 - High [code observation]: hard deletion and audit are not one durable operation.** The status-guarded DELETE is followed by best-effort audit with swallowed failures (:1102, :1106). The original payment FK cascades from jobs (014_phase4_create_payments.sql:7); later schema/runtime behavior must be verified before asserting actual production cascade loss. Replace routine deletion with archival and evidence preservation; gate irreversible purge on dependency checks, policy approval, and a durable audit transaction.
- **T04-F03 - High [confirmed defect]: first-time flagging can incorrectly return a conflict.** The flag UPDATE predicate is NOT (COALESCE(metadata,'{}')->>'admin_flagged') = 'true' (:1067). If the key is absent, extraction yields SQL NULL, the predicate is unknown, and no row matches. A legitimate first flag is therefore reported as already flagged/state changed. Use a null-safe comparison and a targeted absent/false/true-key regression. This is source-level SQL reasoning, not a live reproduction.
- **T04-F04 - Medium [code observation]: paging and export do not support stable large investigations.** List sorting uses only created_at with OFFSET (:1039); UI export ignores current filters (JobsList.jsx:70), and the shared export caps rows at 10,000. Introduce stable cursors and filtered asynchronous exports, preserving existing grant, limiter, and no-store controls.

Expansion scenario

A hurricane response team needs to suspend marketplace exposure for 20,000 affected jobs while preserving already accepted service obligations and payment records. Moderators in several countries must not cancel completed paid work simply because a content report arrives. Two supervisors can act on the same job during a handoff; one should receive a meaningful state conflict. An investigator must reproduce the exact affected cohort, including archived records, instead of exporting a worldwide newest-jobs sample.

Exact recommended controls and UX

Present "hide listing," "cancel service," and "archive record" as distinct, authorized actions with an impact preview. Require reasons server-side, retain before/after versions, and coordinate escrow/refund requests through existing idempotent financial services. Show pending/recovering/failed compensation states and receipt links; do not mark a refund complete from a job status alone. Use durable batch jobs with preview counts, per-item outcomes, checkpoints, and bounded retries for incident cohorts. Currency and provider eligibility must be reviewed by the responsible finance/provider teams for each jurisdiction.

Dependencies / owner

Marketplace Operations owns job semantics; Payments owns escrow and compensation; Trust and Safety owns content hiding; Data Governance owns retention/purge policy; IAM and SRE own scope and throughput.

Measurable acceptance criteria

Absent/false/true flag states produce correct outcomes. Concurrent cancellation and acceptance cannot commit an invalid lifecycle transition. Canceling a paid/completed job requires an explicit allowed policy and recovery receipt. Every destructive action has durable audit evidence even if auditing fails. A 20,000-job isolated batch resumes after restart and reports every item exactly once. Six-million-user-derived job histories support p95 list reads below two seconds; regional API and export denials execute in the authenticated matrix.

5. Reports - /admin/reports

Mission / what exists

Reports is the content-report intake and review queue. It already shows pending/reviewed/escalated/dismissed status tabs, reporter details, descriptions, job/user deep links, and dismiss/warn/remove/escalate actions. Errors render explicitly rather than claiming the queue is clear. The backend uses bounded pagination and locks each report during review, returning 409 for a previously processed report. The richer reports dashboard is linked from the list. Those features form a real review workflow; the enterprise gaps are queue reachability, faithful decision states, ownership, and consistent downstream effects across different target types.

Evidence

UI loading/actions/statuses: client/src/admin/pages/ReportsList.jsx:11, :28, :40, :149, :177. API: server/routes/admin.js:71; list/review: server/controllers/adminController.js:50, :89, :105, :121, :135, :207. Localized report contracts: client/scripts/adminReports.test.mjs:11, :18. The support-report replay test at server/__tests__/adminMessageIdempotency.test.js:225 covers a different workflow and does not prove content-report decision idempotency; that property needs its own behavioral acceptance.

Findings

- **T05-F01 - High [confirmed defect]: the UI cannot navigate the server's paginated report queue.** The request sends only status (ReportsList.jsx:29); the page renders the returned array with no pagination controls. The server defaults to 25 and returns total/page/limit (adminController.js:56, :76). With 500 pending reports, operators can only access the initial 25 through this list until decisions remove them. Reviewed or escalated historical pages remain inaccessible from the status tab. Add cursor navigation, queue position, and a distinct oldest-unowned view.
- **T05-F02 - Medium [confirmed defect]: dismiss decisions do not enter the displayed dismissed state.** The UI requests status=dismissed, but review sets every non-escalate decision to reviewed (adminController.js:129). A newly dismissed report therefore cannot appear in the dismissed tab through this handler. Resolve the contract with an explicit decision outcome and separate lifecycle status, or align the UI taxonomy; migrate/report old outcomes carefully rather than inventing historical decisions from localized strings.
- **T05-F03 - High [confirmed defect]: remove-content can report success without handling the target.** The action is offered for any pending report, but removal handles only jobs and reviews (adminController.js:135). Other target types can be marked reviewed without a corresponding removal or an unsupported-target response. For jobs, removal directly cancels status without the dependency-aware financial workflow described in T04-F01. Resolve supported targets explicitly and delegate to existing content/lifecycle services.
- **T05-F04 - High [code observation]: the durable decision and audit/notification effects are separated.** The report row is transactionally locked, which prevents duplicate decisions; however audit failures after COMMIT are swallowed (:207), and warning notification failure is only represented in the response (:218). The UI discards that response and shows generic success (ReportsList.jsx:47). Preserve existing conflict protection while making notification and audit delivery visible and recoverable.

Expansion scenario

A coordinated spam event produces 50,000 reports across multiple languages. Duplicate reports concern the same job, while one genuine high-risk complaint is older but has fewer submissions. Regional reviewers need ownership, language, due time, and linked evidence; supervisors need to inspect reviewed history even when thousands of cases remain pending. A warning provider failure must appear as an unresolved delivery task rather than a successful warning. A review decision should be understandable independently of the language used by the original operator.

Exact recommended controls and UX

Extend the existing reports and reports-dashboard records with assignment, severity, locale, service region, case grouping, and SLA projections. Preserve FOR UPDATE and add version-aware claim/reassignment. Store machine-readable outcome and policy revision separately from localized display labels. Render supported-target actions only after authorization and capability checks; enforce identical checks on the API. Add a durable outbox for decision notifications and transactional audit, plus explicit delivery pending/failed states. Link evidence snapshots and appeals without creating another reporting database. Bulk triage should run as a previewed, scoped asynchronous job with individual decisions and audit references.

Dependencies / owner

Trust and Safety owns case taxonomy and policy; Marketplace/Payments own job effects; Messaging owns notification recovery; IAM owns regional reviewer scope; Support Operations coordinates escalated customer cases.

Measurable acceptance criteria

An operator can reach all 500 fixture reports and their reviewed history. Dismiss produces the agreed dismissed outcome. Unsupported targets return 422 or the agreed explicit error without changing the report. Two simultaneous reviews produce one decision and one meaningful conflict. Provider/audit failure never appears as complete delivery. A 50,000-report batch resumes without duplicate warnings; five-language keyboard workflows and regional negative-access checks execute without skips.

6. Moderation - /admin/moderation

Mission / what exists

Moderation is a distinct implemented job review queue, derived from admin flags and pending content reports. It shows job descriptions, client information, categories, report counts, and detail links, with dismiss/unflag, flag, and cancel actions. Fetches are abortable and failed reads show a retry panel. It complements Reports, which records individual complaints, and Content Removals, which handles bids/reviews. Enterprise work should connect these existing views into consistent case decisions, not replace them with a new moderation platform or falsely label the page missing.

Evidence

UI: client/src/admin/pages/Moderation.jsx:17, :25, :81, :108. Queue SQL: server/controllers/adminController.js:237, :253, :267. Permissions: server/routes/admin.js:73, :78. Job mutation semantics: server/controllers/superAdminController.js:1045, :1065, :1074, :1083. Related report locking is present at adminController.js:105.

Findings

- **T06-F01 - High [code observation]: the moderation queue has no row bound or server pagination.** Its SELECT joins jobs, users, categories, and reports, aggregates report counts, sorts, and returns every matching job (adminController.js:242). The UI renders the entire array (Moderation.jsx:81). At large abuse-event volumes this can create expensive queries, large payloads, and long browser render work. This is inspected unbounded behavior, not a measured performance failure. Page the authoritative query before rendering and benchmark reporting indexes with real cardinalities.
- **T06-F02 - High [confirmed defect]: "Dismiss" does not resolve report-driven queue membership.** The button invokes unflag (Moderation.jsx:110), whereas the queue includes any job with pending reports (adminController.js:260). A job with pending reports remains in the queue after its admin flag is cleared; a report-only job can return 409 because there was no flag (superAdminController.js:1076). Expose "clear admin flag" accurately and provide a linked report-review action with explicit case outcomes.
- **T06-F03 - High [code observation]: read and write permissions differ without a task-specific UI gate.** Queue read requires global moderate_content; actions require global manage_operations (admin.js:73, :78). The component offers all buttons without checking the latter grant. Server enforcement correctly blocks unauthorized writes, but a moderator can enter an action/reason only to discover that authority is unavailable. Add task-specific rights and matching UI availability, retaining server enforcement as authoritative.
- **T06-F04 - Medium [design requirement]: raw report counts do not establish severity or ownership.** Sorting by count then job creation age (adminController.js:268) favors repeated complaints without representing verified severity, evidence freshness, assigned reviewer, or review deadline. Add policy-based triage and owner claims; do not assume high report volume alone establishes harmful conduct. The first-flag NULL predicate defect is tracked once under T04-F03 rather than counted as another independent bug here.

Expansion scenario

Thousands of reports target legitimate listings during a coordinated campaign while a single dangerous listing receives one report. A bilingual Caribbean team and an Arabic-speaking team review separate territories. The queue must prioritize credible urgency, explain why a job appears, and avoid two reviewers canceling the same active service. Clearing an administrative flag should not make the reviewer think the underlying complaints were dismissed. A language specialist should inspect evidence without inheriting worldwide cancellation or financial intervention rights.

Exact recommended controls and UX

Show queue-entry reasons separately: admin flag, pending report group, automated signal, and reopened appeal. Link each reason to the existing Reports record and show assignment, language, impact region, and due time. Use stable cursors, virtualized bounded lists, and scoped saved filters. Replace prompt-driven interactions with accessible decision dialogs displaying current state, allowed actions, and financial dependencies. Require reason/policy metadata on the API; support conflict refresh and unflag semantics distinct from dismissing complaints. For large incidents, use an asynchronous batch with preview, approval thresholds, individual results, and cancellation of unfinished work.

Dependencies / owner

Trust and Safety owns triage and adjudication; Marketplace owns job effects; IAM owns scoped read/write roles; Payments owns dependent recovery; Accessibility owns dialogs. Existing content reports remain the case source of truth.

Measurable acceptance criteria

A 100,000-job flagged fixture returns at most 100 queue rows per request and meets p95 below two seconds at 150 reviewers. Report-only, flag-only, and combined cases expose correct actions and remain in or leave the queue for explicit reasons. A read-only moderator sees no usable mutation controls and receives 403 on direct API writes. Concurrent reviewers cannot commit contradictory decisions. Urgent single-report cases can outrank low-risk duplicates, with a documented policy explanation and complete localized audit history.

7. Enforcement - /admin/enforcement

Mission / what exists

Enforcement provides an implemented action list with filtering, pagination, issue/deactivation workflows, and confirmation for lifting actions. The backend has a warning/restriction/suspension ladder, writes restrictions and user suspension state transactionally, and records enforcement audit events. Deactivation correctly targets restriction rows by enforcement ID and checks for other active suspensions before lifting the user lock. Appeals already have a schema and indexes. The audit therefore evaluates consistency and delegation of these existing mechanisms, rather than asserting that enforcement or appeals do not exist.

Evidence

UI: client/src/admin/pages/AdminEnforcementDashboard.jsx:14, :27, :89, :380. Routes: server/routes/enforcement.js:14. Enforcement writes: server/controllers/enforcementController.js:24, :48, :93, :150, :173; deactivation: :359, :388, :398. Existing appeals: server/migrations/173_enforcement_appeals.sql:4, :27. Global permission guard: server/middleware/adminAuth.js:126.

Findings

- **T07-F01 - High [confirmed defect]: suspension issuance erases underlying active restrictions.** Issuing a suspension calls liftUserRestrictions, which deactivates all active restriction rows for the user (enforcementController.js:63, :150). Their enforcement-action records are not deactivated by that helper. Later lifting a suspension checks other suspensions but does not restore still-applicable restrictions (:396). A bidding restriction intended to outlast a short suspension can therefore stop applying afterward. Preserve independent sanctions and compute effective permissions from all current actions instead of destroying subordinate effects.
- **T07-F02 - High [code observation]: overlapping issue/deactivate operations lack a common user-level lock.** Both handlers use transactions, but issuance does not serialize the user's sanction set, and deactivation reads the action before BEGIN (:365) then checks other suspensions (:398). Transactions alone do not prove correct concurrent composition. Add a user-scoped lock, atomic version checks, and effective-state recomputation; test an issue racing with deactivation and expiration. This finding does not claim a reproduced production race.
- **T07-F03 - Medium [code observation]: invalid restriction types broaden to "all."** applyUserRestriction converts an unknown type to the broadest restriction (:30). The endpoint validates actionType but delegates restriction normalization to this helper. A typo from an API client should return a validation error, not suspend unrelated capabilities. Require explicit restriction enums, valid future expiry where applicable, and validated target/report relationships before committing a sanction.
- **T07-F04 - High [design requirement]: worldwide delegation and appeals need bounded adjudicator rights.** Current enforcement requires global manage_users; audit read uses a separate grant. Extend the existing appeals/action schema with region/case assignment, reviewer independence, due times, and an escalation route. Existing appeals tables are a dependency, not a missing capability assertion. Eligibility, notices, and retention policy require jurisdiction review; no legal rule is inferred here.

Expansion scenario

A user has a 30-day bidding restriction and receives a two-day suspension from another regional team. At suspension expiry, the original bidding restriction must still apply. During an appeal, one supervisor lifts an incorrect action while another issues a new restriction. The result must be explainable from durable actions and policy versions, without a last-writer-wins user boolean accidentally reopening access. A supervisor should see the necessary history and appeal evidence only within assigned responsibility.

Exact recommended controls and UX

Represent actions as independent records with explicit active/effective/expired/lifted states. Retain the existing transactional audit and enforcement ID links. Recompute effective capability denial under a shared user lock and publish revocation/realtime consequences through the existing authentication system. Require an explicit lifting reason, display remaining actions and resulting access before confirmation, and separate issuance from appeal approval when policy demands it. Add narrow warning/restriction/suspension/lift permissions with regional targets checked on the server. Show expiry timezone, ownership, case links, and whether notification was delivered.

Dependencies / owner

Trust and Safety owns sanction composition; Security owns authentication/session enforcement; IAM owns adjudicator scope; Support owns notices and appeals handoffs. Legal and market operations review policy content independently of implementation.

Measurable acceptance criteria

The 30-day restriction survives the two-day suspension and its lift. Invalid types return 400 without writes. Fifty concurrent issue/lift/expiry attempts preserve the exact effective action set and audit sequence. Expired actions do not incorrectly block deactivation or access. Regional users cannot inspect or sanction foreign cases. Appeal queues expose owner/deadline and retain evidence links; keyboard and localized issue/deactivate dialogs pass executed browser checks.

8. Content Removals - /admin/content-removals

Mission / what exists

Content Removals is an existing administrative workspace for bids and reviews. It has debounced search, status filters, page navigation, reason collection, hide/remove actions, review restoration, retryable fetch errors, and visible restore failures. The server requires moderate_content, validates reasons for removal, conditionally updates the current state, and commits the content change and audit row in one transaction. This is stronger than a purely cosmetic control. Enterprise expansion must preserve those safeguards while distinguishing a harmless visibility correction from a change to accepted marketplace obligations or reputational calculations.

Evidence

UI: client/src/admin/pages/ContentRemovals.jsx:13, :104, :113, :136, :148, :167. Routes: server/routes/admin.js:83. Backend list/actions: server/controllers/adminController.js:389, :447, :509, :571. Conditional update and audit: :466, :475, :528, :537, :588, :596. Initial job/payment dependency: server/migrations/014_phase4_create_payments.sql:7.

Findings

- **T08-F01 - High [code observation]: bid removal accepts any status other than already removed.** The handler fetches bid/job information, checks only status === 'removed', then changes any remaining bid to removed (adminController.js:454, :466). No accepted-bid/job dependency check appears in this handler. Other lifecycle services may exist, so this is not proof that an active job fails in production. Require an explicit policy for accepted/awarded bids, preserve commercial evidence, and route service cancellation or financial effects through existing domain services rather than a status-only removal.
- **T08-F02 - Medium [code observation]: concurrency is detected but reported as a generic server failure.** Conditional updates prevent two operators hiding/removing/restoring the same state, which is valuable. Zero updated rows throw ordinary errors (:470, :532, :592) and outer handlers return 500. An expected conflict during a 3,500-employee handoff should be 409 with current state, actor, and safe refresh guidance. Preserve atomic auditing and distinguish conflicts from actual database failures.
- **T08-F03 - High [verification gap]: visibility, rating aggregates, search, and customer notices need consistency evidence.** Review hide/restore handlers change is_public and audit it (:528, :588); the inspected handlers do not themselves recalculate ratings or issue an owner notice. That does not prove no trigger/consumer handles these effects. Trace and test the full read path, caches, profile ratings, and any notice obligations before claiming completion. Extend the existing review ecosystem with an explicit recovery task where necessary.
- **T08-F04 - Medium [code observation]: restoration lacks decision context and the removal modal lacks full dialog semantics.** Restore sends no reason (ContentRemovals.jsx:150), and its audit contains rating/reviewer metadata but no restoration justification (adminController.js:600). The custom modal begins with plain divs (ContentRemovals.jsx:39). Add explicit policy/reason and accessible focus handling; end-to-end assistive behavior has not been established by source contracts alone.

Expansion scenario

A regional team hides 8,000 fraudulent reviews, another reviews appeals, and an account manager attempts to remove a bid already accepted by a customer. The team must preserve the original records, reverse mistaken hides, and explain rating changes. Two reviewers restoring the same item should get a clear conflict rather than a retry loop that creates confusing audit attempts. No country team should see or act on unrelated tenant content solely because it holds a global moderation grant.

Exact recommended controls and UX

Add a scoped content preview with linked job, accepted-work state, author, policy, evidence snapshot, and visibility history. Separate hiding public content from altering commercial obligations. Keep reason-required removal and transactional audit; extend the same standard to restoration. Make expected-state/version required and return typed conflicts. Use existing notification/recalculation infrastructure with durable delivery receipts and repair queues, not a new ratings database. Provide previewed asynchronous bulk hides/restores with per-item validation and failure reporting. Mask contact details unless needed for the reviewer task and authorize every selected item server-side.

Dependencies / owner

Trust and Safety owns removal policy and appeals; Marketplace owns bid obligations; Reputation Engineering owns rating/search/cache consistency; IAM owns region/tenant scope; Accessibility owns dialog behavior. Legal reviews retention and notification policy by jurisdiction.

Measurable acceptance criteria

Accepted/awarded bid fixtures cannot be removed without the approved dependency-aware action. Concurrent hide/restore attempts yield one accepted transition and typed conflicts, with atomic audit. An 8,000-review isolated bulk job resumes after interruption and identifies every outcome. Public content, search, and rating projections converge within the agreed 60-second target or expose repair status. Restorations require reasons and preserve original evidence; keyboard focus and regional read/write denials pass executed tests.

9. Skills - /admin/skills

Mission / what exists

Skills is an implemented lookup-taxonomy manager, not a missing capability. It supports category filtering, debounced search, paginated rows, add/edit, active/hidden toggles, category autocomplete, and permanent deletion requests. It links to the existing licenses manager. The API bounds list size and offset; permanent deletion is explicitly super-admin-only even though other authorized admins can manage lookup entries. The expansion problem is governing a shared taxonomy used by global profiles and matching, including translations, aliases, referenced IDs, review ownership, and safe concurrent edits.

Evidence

UI: client/src/admin/pages/SkillsManager.jsx:16, :25, :38, :50, :89, :111, :121, :142. API: server/routes/admin.js:649, :688, :715, :736. Initial schema/indexes: server/migrations/037_skills_and_licenses_lookup.sql:5, :11. Related job skill storage: server/migrations/080_job_required_skills.sql:5, :9; business references: server/migrations/079_business_skills.sql:16.

Findings

- **T09-F01 - Medium [confirmed defect]: clearing an existing category does not clear the stored value.** UI edits submit an empty category; the API converts it to null and applies COALESCE($2, category) (admin.js:720, :725). The old category survives even though the form can appear cleared. Distinguish omitted fields from an explicit empty/null update and return the actual persisted state. Apply the same partial-update convention consistently to taxonomy metadata.
- **T09-F02 - High [code observation]: creation can silently overwrite an existing skill and edits have no version precondition.** POST uses ON CONFLICT(name) DO UPDATE (admin.js:695), so an employee adding an apparently new entry can change another curator's category or active status. PUT overwrites fields by ID without checking a version (:719). These paths also lack an audit write in the inspected handlers. Add conflict-aware create semantics, stable IDs, versioned updates, and transactional taxonomy history; do not report mere duplicate-name validation as sufficient governance.
- **T09-F03 - High [design requirement]: the shared schema is a single-name taxonomy rather than a worldwide concept model.** skills_lookup contains one unique name and category; its search index is English full text, while admin search uses ILIKE (037_skills_and_licenses_lookup.sql:7, admin.js:657). This is not proof every skill search is unindexed or that licenses are missing. Extend existing IDs with translated labels, aliases, locale-aware lookup, deprecation/merge mapping, and regional applicability instead of creating one disconnected catalog per language.
- **T09-F04 - Medium [code observation]: UX exposes permanent deletion to users who can only receive a denial.** The component offers the hard-delete request (SkillsManager.jsx:121) without a super-admin visibility check, while the API correctly returns 403 (admin.js:740). Category-fetch failures are also swallowed (SkillsManager.jsx:54), making outage look like no suggestions. Show permitted soft-hide actions, protected deletion, and explicit category-load errors.

Expansion scenario

Curators in France and Haiti add differently worded labels for the same plumbing capability, while a North American team deprecates an outdated skill used by thousands of helpers. A six-million-user matching system needs to retain referenced IDs and map aliases consistently. Two curators editing category/visibility must not lose one another's work. A regional skill may require local qualifications, but engineering should link to the existing license model and jurisdiction-reviewed eligibility policy rather than make unverified legal claims from a category name.

Exact recommended controls and UX

Create a concept-detail screen around the existing skill ID: localized labels, aliases, references, owner, revision, and deprecation destination. Show affected profile/job/business counts before hiding, merging, or deleting. Require expected version and publish review for large taxonomy changes. Use scoped curator permissions distinct from broad manage_operations; allow country-specific applicability while preserving global concept identity. Hard deletion requires zero validated references and policy-approved retention; prefer reversible deprecation. Provide a bounded asynchronous import/merge workflow with dry run, validation errors, resume, and complete audit evidence.

Dependencies / owner

Marketplace Taxonomy owns concepts and aliases; Search/Matching owns query semantics and index plans; IAM owns curator delegation; Localization owns translation review; Legal/Compliance reviews qualification applicability.

Measurable acceptance criteria

Clearing category persists correctly. Duplicate creates return explicit conflict or approved merge preview. Two stale edits cannot overwrite silently. Five-language labels and aliases resolve to one stable concept ID. Referenced skills remain resolvable after deprecation and cannot be permanently purged accidentally. A 20,000-entry isolated import exposes every validation outcome and resumes safely. Scoped curators cannot alter global/foreign policy; all edit dialogs and category-error states pass keyboard and screen-reader checks.

10. Support Tickets - /admin/support

Mission / what exists

Support Tickets is a real inbox with status/priority/ownership/search filters, claim/release, public replies, internal notes, realtime updates, availability, statistics, and ticket deep links. It links to existing agent onboarding, shifts, knowledge base, templates, metrics, SLA, performance, CSAT, QA, escalation, hierarchy, settings, and audit pages. Public request and reply paths already support request/message idempotency. Migration 600 protects terminal ticket transitions. An enterprise audit should strengthen this ecosystem's ownership and queue semantics instead of proposing that the company build an entirely new support suite.

Evidence

UI tools: client/src/admin/pages/SupportTickets.jsx:52; fetch/realtime: :576, :634. API grants: server/routes/admin.js:267, :274. Server list/claim/reply: server/controllers/supportController.js:407, :529, :689. New-ticket fanout: :46, :224. Terminal integrity: server/migrations/600_support_ticket_terminal_integrity.sql:5. Existing request/reply regressions: server/__tests__/supportRoutes.test.js:219, :243; idempotency coverage: server/__tests__/adminMessageIdempotency.test.js:197.

Findings

- **T10-F01 - High [confirmed defect]: competing ticket claims can both succeed and overwrite ownership.** Claim reads status/assigned_to, checks authorization, then performs an UPDATE constrained only by ID (supportController.js:532, :546). Two authorized admins reading an unassigned ticket can both pass; the later write replaces the first owner. Release has a similar read/check/write structure (:575). Reuse the stronger conversation-claim locking approach in the Live Support subsystem; require an atomic owner/version predicate and a typed 409.
- **T10-F02 - High [confirmed defect]: the inbox does not navigate the server's bounded result set.** List supports limit/offset with default 50 (supportController.js:415). The component sends only search/status/priority/assigned (SupportTickets.jsx:583) and exposes no list pagination state in this load path. Operators see a total exceeding reachable rows. The sequence guard already prevents stale response replacement; extend it with stable server cursors and queue-wide summary semantics.
- **T10-F03 - High [code observation]: ticket notifications and reloads fan out to all administrators.** Every new ticket emails every admin/super-admin and broadcasts an admin event (supportController.js:46, :224). The recipient query does not require an active support assignment or active account. Each subscribed inbox reloads both list and stats on every event (SupportTickets.jsx:634). An assumed 100-ticket burst with 200 open inboxes generates up to 20,000 reload triggers, with overlapping requests despite the stale-response guard. Scope notifications and coalesce reloads; this is workload arithmetic, not measured production load.
- **T10-F04 - Medium [design requirement]: global ticket grants do not support regional ownership delegation.** manage_support controls read/claim/status, while reply uses message_users. Existing ownership checks and separate grants are useful, but country/team responsibility, absence coverage, and supervisor reassignment need explicit task-scoped contracts. Integrate existing shifts, hierarchy, and SLA systems rather than claiming those capabilities are absent.

Expansion scenario

After a payment incident, thousands of customers submit tickets while shifts hand off between New York, Paris, and Cairo. Two agents simultaneously claim the same urgent billing ticket. A regional supervisor needs to redistribute work from an absent employee without worldwide super-admin powers. All employees should not receive customer ticket notifications; only eligible queue members need actionable alerts. A filtered urgent queue must expose every case, including cases beyond the first 50 and cases with a failed reply-delivery attempt.

Exact recommended controls and UX

Use atomic claims with owner/version, supervisor-authorized takeover reasons, and explicit transfer receipts. Add cursor navigation, server-side urgency ordering, unread and oldest-unassigned projections, and scope-constrained saved queues. Preserve existing terminal guards and message idempotency. Route alerts to existing teams/on-call schedules through a bounded outbox; notify inactive/off-shift staff only under approved escalation policy. Debounce/coalesce realtime invalidations and fetch lightweight counts separately from full list detail. Show reply authority, delivery state, ownership conflict, and resolution evidence in accessible controls. Bulk reassignment should be a previewed asynchronous task with per-case authorization.

Dependencies / owner

Support Operations owns team coverage and SLA rules; Support Engineering owns claims/inbox; Messaging owns notification delivery; IAM owns territory and reply rights; SRE owns event fanout budgets. Customer privacy and retention policy require business/legal review.

Measurable acceptance criteria

Twenty simultaneous claims produce one owner and nineteen explicit conflicts. All 5,000 filtered fixture tickets are reachable. A 100-event burst creates at most one coalesced reload per viewer per agreed one-second window. Inactive/unassigned staff receive no routine new-ticket emails. Regional supervisors can transfer only permitted cases. A 10,000-ticket reassignment resumes safely. Resolution/reopen paths honor migration 600; keyboard, locale, ownership, and failed-delivery browser scenarios execute without skips.

11. Live Support - /admin/support/chat

Mission / what exists

Live Support embeds the existing SupportConversationWorkspace with availability controls. The workspace implements paginated/load-more conversations, assignments, realtime viewer presence, typing/read states, lifecycle/customer context, SLA information, reminders, attachments, transcript download, merge, redaction, internal notes, AI draft review, external-channel delivery status, and retry controls. Conversation claims already lock capacity and competing work. Email and SMS adapters exist; phone/social outbound explicitly report unsupported. The enterprise issue is strengthening these working components' authorization, query shape, global filters, and provider evidence, not inventing a missing live-support page.

Evidence

Wrapper: client/src/admin/pages/SupportAgentChat.jsx:6. Workspace list/filter/events: client/src/components/support/SupportConversationWorkspace.jsx:187, :518, :614, :795, :984. Platform list API: server/routes/support.js:64, :284, :297, :321. Claim locking: server/controllers/conversationAssignmentController.js:51, :77. Existing tenant checks: server/services/supportEmployerService.js:11, :62; channel capabilities: server/services/supportChannelDeliveryService.js:17; tests: server/__tests__/supportRoutes.test.js:173, server/__tests__/supportChannelDeliveryRecovery.test.js:15.

Findings

- **T11-F01 - High [confirmed defect]: live conversation access bypasses the delegated support grant boundary.** After general authentication, /support/conversations treats any admin role as platform authority (support.js:285, :297), returning global conversations unless a business filter is supplied. The legacy ticket API requires manage_support. Role-level access here does not check that grant or region. A business filter is optional client input, not an authorization boundary. Apply server support-read/write/export scopes consistently while preserving existing business membership checks for nonplatform agents.
- **T11-F02 - High [code observation]: the list aggregates whole message histories despite conversation pagination.** The platform list joins all conversation messages and JSON-aggregates their contents before LIMIT (support.js:328, :355, :359). Fifty conversations can therefore return far more than fifty lightweight records. Move message detail to the existing detail API and use bounded last-message/unread projections; paginate conversations before expensive joins where plans permit. Verify redaction and retention behavior across this raw list projection as well as detail/download.
- **T11-F03 - High [confirmed defect]: several queue filters operate only on loaded conversations.** Requests include search/channel but not status, ownership, priority, or category (SupportConversationWorkspace.jsx:198); those filters run locally over the loaded array (:614). An urgent owned conversation beyond the first page can be absent from an apparently empty filtered view. Keep load-more but make operational filters authoritative on the server and return matching totals/cursors.
- **T11-F04 - Medium [verification gap]: supported adapters and retries do not prove worldwide channel coverage.** Capabilities explicitly mark phone/social unsupported and email/SMS configured conditionally. The UI already respects capability state. No provider delivery/recovery evidence for every target country was obtained; a healthy Redis/worker snapshot or green mocked test is insufficient to claim supported live delivery everywhere.

Expansion scenario

Three thousand long-running conversations span multiple businesses and languages during a provider outage. A regional live agent selects urgent, unassigned SMS cases and must find matching cases beyond the initial load. Another operator views a two-year conversation without downloading its full history on every queue refresh. An Arabic-speaking supervisor reviews a merge and redaction while a provider retry resumes. Country/tenant scope must constrain list, transcript, merge target, socket rooms, customer context, and retries, not only visible filters.

Exact recommended controls and UX

Extend the existing support grant model to every REST and realtime operation; require server-resolved regions/businesses and scoped transcript-export rights. Use lightweight queue rows, cursor pagination, server filters, and lazy history retrieval. Retain existing locked claims and expose assignment/version conflicts. Preserve capability-aware buttons, but add per-market supported/configured/degraded status and delivered/failed/unknown provider receipts. Unsupported channels need an explicit owned fallback case with SLA, not a misleading sent state. Localize operational fallback strings and test bidi names, attachments, transcript redaction, and focus handling.

Dependencies / owner

Support Engineering owns queue/state/realtime; IAM owns scope consistency; Integration Operations owns providers and market evidence; Support Operations owns fallback SLAs; Privacy owns approved redaction/retention behavior. Existing channel outbox/recovery remains authoritative.

Measurable acceptance criteria

An admin without support grants receives 403 on list/detail/download/socket access. Nonmember tenants and foreign regions are denied on all derivatives. Priority/owner filters find cases on every page. Queue payload contains only a bounded preview per row and stays below the agreed 250 KB target for 50 rows. At 250 concurrent agents, p95 list reads stay below two seconds. Demonstrate provider timeout, restart, retry, and duplicate-webhook recovery in isolation; execute Arabic, keyboard, merge, and redaction acceptance without skips.

12. Quote Requests - /admin/quote-requests

Mission / what exists

Quote Requests is an implemented intake inbox with paginated lists, status/search filters, customer contact details, request context, notes, status editing, and guarded detail fetches. Its underlying records are shared with Direct operations, which add assignment, scheduling, payment proof, catalog selections, and customer messages. The server validates statuses, negative amounts, schedule order, assignee eligibility, and some payment transitions. This existing cross-workspace record model is useful; enterprise readiness requires preserving omitted fields, enforcing the same authority on detail reads, and preventing concurrent notes/financial changes from silently overwriting one another.

Evidence

UI: client/src/admin/pages/QuoteRequestsInbox.jsx:72, :83, :118, :159, :198. Routes: server/routes/quoteRequests.js:10, :12. Server list/detail/update: server/controllers/quoteRequestsController.js:530, :596, :620, :742, :811. Direct aliases: server/routes/direct.js:104, :106. Existing catalog-intake coverage: server/__tests__/directQuoteCatalogLink.test.js:48; shared record schema: server/migrations/374_quote_requests.sql:4.

Findings

- **T12-F01 - High [confirmed defect]: direct request detail uses role-only administrative access.** List/update routes require manage_operations, but GET /quote-requests/:id only authenticates and its controller authorizes any admin/super-admin role (quoteRequests.js:12, quoteRequestsController.js:607). Thus a grantless admin denied the inbox list can obtain a known request's full detail. Require the same scoped read permission for the administrative branch while retaining owner access and field-appropriate customer serialization.
- **T12-F02 - High [confirmed defect]: notes/status-only edits can replace an existing quote currency with USD.** UI sends only {status, admin_notes} (QuoteRequestsInbox.jsx:163). The controller defaults omitted currency to USD (quoteRequestsController.js:631) and always writes it through COALESCE (:816). Updating notes on an EUR quote therefore changes its currency label while preserving the amount. Omitted fields must remain untouched; currency changes need a dedicated validated monetary transition with explicit intent and audit.
- **T12-F03 - High [code observation]: preflight checks and final edits are not version-bound.** Payment/schedule checks read current data, then a later UPDATE uses only ID (:746, :831). Admin notes replace the whole text (:814), and the UI has no expected version. Simultaneous intake and dispatch operators can lose notes or change state after validation. Preserve existing schedule guards and add transactionally locked/versioned updates, append-only notes, and conflict-aware UI diff/reload.
- **T12-F04 - High [code observation]: list API accepts unbounded limit, while headline subtotals are page-local.** The list uses a minimum but no maximum for requested limit (:555). UI "new" and "active" counts derive from the 20 returned requests (QuoteRequestsInbox.jsx:205, :211) beside a total from the whole query. The UI list loader also lacks the detail loader's sequence protection (:118). Bound requests, return clearly scoped aggregate counts, and guard list response ordering.

Expansion scenario

A European sales team receives 25,000 requests while a North American dispatcher updates schedules on the same shared records. A Paris agent changes a French note on a euro-denominated quote; the amount/currency must remain EUR. Two operators editing a case should see a meaningful conflict and preserve both notes. A regional manager needs accurate intake aging, ownership, and conversion totals instead of counters that change merely because another page was opened. Read authority must also protect direct deep links.

Exact recommended controls and UX

Separate read, intake-note, quote-price, assignment, scheduling, and payment-proof rights within the existing request model. Enforce territory/business scope in list/detail/update/export. Use partial-update semantics distinguishing absent, clear, and change; freeze monetary revisions once accepted/settled except through approved correction. Store note authors/time and request revisions. Return bounded cursors and server totals; show currency, region, owner, due time, and last edit in the pane. Add an explicit conflict comparison and dirty-draft protection. Asynchronous lead assignment/import can reuse existing Direct ownership and audit infrastructure.

Dependencies / owner

Sales/Direct Operations owns intake and conversion definitions; Payments owns monetary changes; IAM owns request scope; Data Engineering owns aggregates; Support owns contact delivery. Country commercial/legal eligibility remains a separate reviewed policy.

Measurable acceptance criteria

A notes-only patch to EUR preserves EUR and amount exactly. Grantless admins cannot read request IDs; owners retain appropriate access. Stale concurrent edits return 409 with preserved notes. Requests above the agreed 100-row cap are rejected or clamped explicitly. New/active totals reconcile across all pages and delayed old list responses cannot replace newer filters. A 25,000-request synthetic assignment run resumes safely and meets the agreed regional SLA; localized keyboard workflows execute without skips.

13. Direct Ops - /direct/admin

Mission / what exists

Direct Ops is an existing multi-page operational portal with dashboard, inbox, work orders, dispatch calendar, assignment, availability/blackout blocks, calendar export/history, payment-proof review, customers, reports, team, settings, and catalog links. The main application hosts this route outside the admin SPA, as the route manifest explicitly records. The slot engine already uses holds, transactions, row/advisory locks, and post-lock capacity checks. Calendar overlap protection also exists in migrations. Enterprise readiness therefore concerns finer authority, consistent field edits, operational timezones, and trustworthy payment recovery rather than absence of dispatch or scheduling.

Evidence

Route identity: client/src/admin/adminRouteManifest.js:34. Portal calendar/payment: client/src/pages/DirectAdminPortalPage.jsx:702, :826, :1165, :1280, :1319. API: server/routes/direct.js:78, :80, :104, :108. Proof: server/controllers/directController.js:1753; update guards: server/controllers/quoteRequestsController.js:742. Locking: server/services/directSlotEngine.js:428, :562, :604. Schema: server/migrations/420_direct_schedule_conflict_guard.sql:17, server/migrations/589_direct_catalog_availability_capacity.sql:81; tests: server/__tests__/directSlotEngine.test.js:131.

Findings

- **T13-F01 - High [code observation]: operational rights also authorize payment-proof confirmation and payment-status edits.** Proof confirmation requires manage_operations (direct.js:78) and records confirmed status/operator metadata (directController.js:1767). General request patch accepts payment statuses including paid/refunded and guards only selected regressions (quoteRequestsController.js:654, :758, :773). These are administrative records, not proof of a bank transfer or actual refund. Separate dispatch from reconciliation/financial correction rights and tie final monetary status to durable evidence and approved recovery controls.
- **T13-F02 - High [code observation]: dispatch editing uses the browser's local timezone.** Date-key helpers and datetime-local conversions use JavaScript Date; schedule edits convert the browser-entered time to ISO (DirectAdminPortalPage.jsx:702, :731, :1166). No explicit service-location timezone selector appears in these controls. A Cairo operator scheduling an Ohio job can interpret the same wall-clock time differently. Store and display service timezone plus UTC; validate daylight-saving ambiguity independently of the operator's computer timezone.
- **T13-F03 - High [design requirement]: operations grants are global and bundle unrelated responsibilities.** Dashboard, inbox, customer history, team, catalog, scheduling, and proof share global manage_operations; settings and exports have separate grants (direct.js:80-112). This correctly denies regional grants today but does not support country dispatcher, crew supervisor, catalog curator, or finance reviewer delegation. Add domain-specific scoped permissions without promoting routine dispatch staff to super admins.
- **T13-F04 - Medium [verification gap]: existing capacity/overlap guards need one combined concurrency and recovery rehearsal.** Advisory locks and hold checks are implemented; claiming scheduling is wholly race-prone would be inaccurate. What remains unproven is consistent behavior across public holds, administrative reschedules, newly inserted blackout blocks, worker restarts, and 3,500-employee handoffs on a production-like restored dataset. CI's clean-database migration rehearsal does not establish that combined property.

Expansion scenario

A global dispatcher schedules Ohio crews from Cairo while a local supervisor creates a weather blackout. Customers hold booking slots concurrently and a payment reviewer checks externally submitted proof. A disputed payment needs investigation without an operations employee marking "refunded" as if funds were returned. The worker restarts mid-reminder delivery. The system must preserve slot capacity, time interpretation, ownership, payment evidence, and delivery receipts across all these existing paths.

Exact recommended controls and UX

Use a scoped dispatcher workbench with territory, crew pool, timezone, coverage, capability, and owner/SLA indicators. Preserve slot locks and database overlap guards; require expected versions for admin edits and integrate blackout changes into the same coordination boundary. Give financial reviewers a proof-evidence checklist, original currency/amount, external reference, immutable receipt, and correction history; high-impact corrections require maker/checker policy. Reuse existing ledger/provider recovery where actual financial movement occurs. Operational status and settled funds must be separate concepts, with unknown/pending/failed reconciliation visible. Market/provider approvals and payment policy need jurisdiction review.

Dependencies / owner

Direct Operations owns dispatch; Marketplace Scheduling owns holds/capacity; Finance owns proof/reconciliation; IAM owns delegation; SRE owns recovery rehearsals; Localization owns timezone and regional formatting.

Measurable acceptance criteria

A foreign-timezone operator schedules the same service instant as the local crew, including DST repeated/skipped hours. Fifty concurrent holds/reschedules/blackouts never overbook an assignee. Dispatch-only users cannot confirm proof or finalize paid/refunded states. Monetary recovery requires an evidence-linked receipt and replay-safe execution. An isolated restart preserves bookings and reminder outcomes. A production-like restored rehearsal records dataset, image, migration state, timings, and results; full authenticated Direct journeys execute without skips.

14. Direct Service Areas - /admin/direct-service-areas

Mission / what exists

Direct Service Areas manages the coverage records used by public Direct availability and quote intake. The existing page supports search/active filtering, creating/editing an area, ZIP coordinate lookup, radius, notes, activation, and refresh/retry. The service caches geocoding and compares locations to active coverage radii. Direct coverage is different from employee administrative territory: a customer's location being serviceable must not grant an operator authority to manage that market. Worldwide expansion needs to extend this implemented coverage system for international geography, governance, spatial scale, and service-specific promises.

Evidence

UI: client/src/admin/pages/DirectServiceAreas.jsx:11, :64, :75, :83, :112, :177. Routes: server/routes/direct.js:101. Service: server/services/directServiceAreas.js:46, :106, :120, :146, :197, :227, :273. Controller: server/controllers/directController.js:1188. Schema: server/migrations/377_direct_service_areas.sql:5. Existing tests: server/__tests__/directServiceAreas.test.js:55, :81, :105, :118.

Findings

- **T14-F01 - High [code observation]: the managed geography path is US-specific.** The form requires and strips input to five numeric ZIP digits (DirectServiceAreas.jsx:85, :123, :179). ZIP geocoding calls /us/, and address geocoding sets countrycodes=us (directServiceAreas.js:107, :126). A Canadian or UK postal address cannot be managed through this flow as designed. This is a current expansion limitation, not an outage of the supported Ohio coverage. Add explicit country and postal validation with provider capability by market.
- **T14-F02 - High [code observation]: coverage and administration load all areas and compare them in application memory.** listServiceAreas is unbounded (:46), public availability maps/sorts every active area (:201), and the client filters locally (DirectServiceAreas.jsx:75). That may be adequate for a small launch footprint; it is unproven for thousands of country/service polygons and frequent public checks. Use scoped paginated management and indexed spatial matching with bounded candidates, preserving existing distance/cache semantics where appropriate.
- **T14-F03 - High [code observation]: coverage edits have neither version conflicts nor durable actor history in the inspected write path.** Update reads the row and later replaces fields by ID (directServiceAreas.js:273, :285). The controller passes no actor to the service (directController.js:1220). A manager disabling an area can be overwritten by another employee's stale radius edit. Require expected version, actor/reason, and an atomic change history linked to the existing Direct audit system.
- **T14-F04 - Medium [confirmed defect]: coordinate validation accepts finite but geographically invalid values.** Normalization checks finiteness and positive radius (directServiceAreas.js:235), not latitude/longitude ranges. The initial schema constrains radius but not coordinate bounds (377_direct_service_areas.sql:9). Reject latitude outside [-90,90] and longitude outside [-180,180] in API validation; verify any later constraints before migrating existing data. Also distinguish absent coordinates from empty strings that coerce to zero.

Expansion scenario

Operations launches Toronto, London, and Paris while Ohio remains live. A postal lookup must include country and service timezone, and a river or border can make a simple radius an unsuitable service promise. A regional manager disables Toronto coverage during a staffing shortage; a stale editor must not reactivate it accidentally. Thousands of coverage regions and millions of customer availability checks must not require loading the entire worldwide directory for every request. Administrative scope remains independently enforced on every edit.

Exact recommended controls and UX

Extend existing area IDs with country, timezone, service/category applicability, geometry, effective dates, and accountable owner. Support radius and polygon coverage with explicit exclusions and a preview map/test-address tool. Show distance units appropriate to the selected locale and provider supported/configured/unavailable states. Require versioned publish/disable with impact preview on accepted bookings, holds, and quotes; schedule coverage changes through existing Direct lifecycle controls. Paginate management, cache versioned public coverage, and invalidate it after approved changes. Provider usage/market eligibility must be reviewed with the chosen provider; do not assume existing public geocoder endpoints are approved for enterprise worldwide traffic.

Dependencies / owner

Direct Market Operations owns coverage promises; Geospatial Engineering owns spatial indexes/providers; IAM owns country editor scope; Scheduling owns existing bookings/holds; Localization owns postal/address/unit presentation.

Measurable acceptance criteria

Canadian alphanumeric and UK postal fixtures resolve only within the selected country. Invalid coordinate ranges return 400 without persistence. Concurrent disable/radius edits return a typed conflict and preserve actor history. A 10,000-area synthetic directory returns paged admin results and public matches within p95 500 ms under an agreed traffic workload. Coverage changes explicitly preserve or route existing obligations. Regional editors cannot change other markets; map alternatives, keyboard input, Arabic layout, and provider-outage messages pass executed browser checks.

15. Direct Catalog - /admin/direct-catalog

Mission / what exists

Direct Catalog is an implemented catalog/lifecycle editor with categories, services, tools, five-language translations, review flags, search, availability/bookability, price, service area, crew size, assignee pools, bulk lifecycle transitions, archive confirmation, and AI draft preview/apply. Public catalog queries already support bounded localized search, and server publication requires complete translations. Entity updates are transactional with audit/history; optimistic conflicts exist when updatedAt is supplied. Existing manual fallback for an unavailable AI provider is tested. Enterprise enhancement must focus on the administrative payload, bulk completeness, strict concurrency, and scoped publication ownership.

Evidence

UI: client/src/admin/pages/DirectCatalog.jsx:10, :22, :51, :93, :121, :139, :160, :174. API: server/routes/direct.js:88, :98. Service: server/services/directCatalogService.js:514, :571, :604, :663, :742, :804. Schema/history indexes: server/migrations/390_direct_catalog.sql:57, 414_direct_catalog_lifecycle.sql:107, 584_direct_audit_indexes.sql:20. Tests: server/__tests__/directCatalog.test.js:138, :166, :257, :334; performance verifier tests: server/__tests__/directCatalogPerformance.test.js:5.

Findings

- **T15-F01 - High [code observation]: admin catalog returns the whole directory despite public pagination.** getAdminCatalog selects all categories/services/tools and resolves pools/review flags for them (directCatalogService.js:515). The client searches loaded arrays, chiefly English names/descriptions (DirectCatalog.jsx:121). Existing public bounded search should be extended to authenticated scoped management instead of claiming all catalog paths are unpaginated. Require server filters, lightweight summaries, locale-aware search, and detail-on-demand for large country catalogs.
- **T15-F02 - High [confirmed defect]: bulk selection above 100 is silently truncated.** The service deduplicates requested IDs then slices to 100 (directCatalogService.js:808); the client permits arbitrary selected IDs and sends them together (DirectCatalog.jsx:156, :162). The request can report success for 100 while leaving other selected entries unchanged. Reject excessive synchronous batches explicitly or return a durable job covering the entire authorized selection, with requested/processed/failed/skipped totals and a manifest.
- **T15-F03 - Medium [code observation]: optimistic concurrency is optional, not an API invariant.** The UI carries updatedAt, and the parser accepts it as expectedUpdatedAt (DirectCatalog.jsx:63, directCatalogService.js:604). The locked save checks conflicts only if that value exists (:668). A different client omitting the precondition can overwrite a stale record. Preserve the existing implementation and require a valid version on every update; bulk and archive need equivalent expected-state or reviewed-selection semantics.
- **T15-F04 - High [design requirement]: global publishing rights need regional and commercial approval boundaries.** Catalog routes share global manage_operations; records have serviceArea text, currency, and lifecycle, but those fields alone are not curator authorization or market approval. Add scoped curator/drafter/publisher rights and market-specific offer applicability around existing entity IDs. Five complete translations do not establish provider availability, licensing eligibility, price accuracy, or legal approval in every country.

Expansion scenario

A curator prepares 2,000 services for three new markets while local reviewers approve translations and operations verifies crew capacity. Pausing 800 services during a staffing incident must cover all 800, not a hidden first 100. A stale translated description cannot overwrite a newly reviewed price/availability revision. Customers in Arabic should find the same service concept through localized names, while publication in one market must not enable bookings in another without readiness and provider review.

Exact recommended controls and UX

Keep translation tabs, lifecycle rules, review flags, locked transactional saves, audit/history, and manual AI fallback. Add server-backed locale/category/lifecycle/market search and paginated rows with virtualized selection. Provide persistent batch receipts, dry run, full selection counts, per-item failures, cancellation of pending work, and resume after restart. Make versions mandatory and show conflict diffs without discarding drafts. Attach owner/reviewer/market eligibility and price revision to publication; reuse crew-pool/bookability controls rather than duplicate availability systems. Separate AI drafted, human reviewed, operationally ready, and published states; display capability availability accurately without exposing private configuration.

Dependencies / owner

Catalog Product owns entities; Localization owns language review; Direct Operations owns operational capability; Finance owns price/currency revisions; IAM owns scoped publication; SRE owns payload/batch budgets. Legal/providers approve applicable market conditions separately.

Measurable acceptance criteria

A 10,000-service admin fixture returns no more than 100 rows and localized server search finds all supported-language labels. An 800-item transition processes every item or explicitly rejects before writes; replay cannot duplicate history. Updates without valid versions fail, stale versions return 409, and correct versions preserve transactional audit. Publication requires the agreed regional readiness evidence. AI outages retain manual editing. Measure admin and public paths separately; verifier unit tests are not claimed as a completed live capacity test.

16. Messages - /admin/messages

Mission / what exists

Messages is an implemented one-to-one administrator/customer messaging workspace with conversation search, unread counts, thread selection, optimistic sending, visible send errors, and a new-conversation dialog. List/detail fetches have stale-response protection and several controls are localized/accessibility-tested. Server routes require message_users; ordinary admins list their own conversations, while super admins list all. Sending uses sanitized content, transactions, conversation uniqueness recovery, optional message idempotency, audit, and notification infrastructure. The remaining problems are UI/API identity alignment, history reachability, truthful read state, replay from the browser, and team continuity.

Evidence

UI: client/src/admin/pages/AdminMessages.jsx:49, :79, :102, :115, :160. Routes: server/routes/admin.js:639. List/thread/send: server/controllers/adminMessageController.js:36, :110, :146, :153, :188, :219. User lookup contract: server/controllers/superAdminController.js:538. Tests: server/__tests__/adminMessageIdempotency.test.js:99, :121, :142; UI contract: client/scripts/adminMessages.test.mjs:11.

Findings

- **T16-F01 - High [confirmed defect]: new-message user search sends the wrong parameter.** AdminMessages.jsx:176 requests /admin/users?q=..., but getUsers reads search, not q (superAdminController.js:538). The typed query is ignored and the first ten ordinary users can be returned. Also, user lookup requires manage_users while the page requires message_users, so messaging-only staff cannot find recipients through that path. Add a narrow scoped recipient endpoint or align the existing lookup without granting global user-management authority.
- **T16-F02 - High [confirmed defect]: thread loading hides recent history and marks unseen messages read.** UI requests no page/offset (AdminMessages.jsx:88). Server returns the first 50 messages ordered ascending (adminMessageController.js:114, :161) and first marks every unread message in the conversation read (:146). For a 200-message thread, the newest 150 are absent while read state can say they were seen. Fetch the latest bounded page, allow older history navigation, and mark only messages actually displayed through a separate authorized acknowledgment.
- **T16-F03 - High [confirmed defect]: super-admin list and thread selection identify different conversations.** Super list includes other admins' conversations (adminMessageController.js:76), but UI opens by userId alone (AdminMessages.jsx:103) and thread lookup always uses the current admin as customer_id (adminMessageController.js:129). Selecting another admin's case can show the supervisor's different thread or empty history. Use an immutable conversation ID and explicit supervisor read/participation authorization, rather than conflating all contact with a user.
- **T16-F04 - Medium [code observation]: browser sends do not activate existing idempotency support.** The UI posts content only (AdminMessages.jsx:133), while the backend accepts optional UUID message IDs (adminMessageController.js:24, :193). An ambiguous network failure followed by manual resend can duplicate contact despite server replay tests. Persist a message UUID per draft/send attempt and retry the same identity; preserve visible delivery uncertainty and draft recovery.

Expansion scenario

A supervisor takes over a departing employee's 200-message customer thread while a regional agent sends a sensitive billing clarification over an unstable connection. Typing the recipient name must find the intended authorized user, not an unfiltered initial page. The supervisor must see the exact selected conversation and latest message; unread state must not erase unseen evidence. Employee turnover and shift coverage need continuity without copying private threads into a second communication platform or granting every employee all customer messages.

Exact recommended controls and UX

Keep existing message sanitization, uniqueness recovery, audit, and notification queues. Introduce conversation-ID navigation, cursor history, explicit displayed-message read acknowledgment, and scoped recipient search. Add persisted send UUIDs, pending/sent/failed/unknown status, retry receipts, and drafts scoped to the selected thread. Extend ownership/transfer with existing support case links and team delegation; direct customer contact should retain purpose and case context. Paginate the conversation list beyond the current requested 100 rows (AdminMessages.jsx:56) and enforce both region and participant authority on REST/socket operations. Keep accessible dialog/status controls and test the actual send/history behavior, not only source strings.

Dependencies / owner

Messaging Engineering owns thread identity and delivery; Support Operations owns continuity; IAM owns contact/supervisor scope; Privacy owns visibility/retention policy; Localization/Accessibility own authoring and confirmation behavior.

Measurable acceptance criteria

Recipient queries return only matching permitted users. A 200-message thread initially shows the newest page and exposes all older pages; only displayed incoming messages are acknowledged. A supervisor selecting another admin's conversation sees that exact ID or an explicit denial. Twenty retries of one send UUID produce one message and one notification/audit effect. Regional and nonparticipant access is denied, and cross-thread draft/send races preserve recipient identity. Executed keyboard/five-language browser tests validate new conversation, transfer, history, and ambiguous delivery recovery.

17. Search - /admin/search

Mission / what exists

Search is an implemented federated administrative lookup across users, jobs, messages, disputes, payments, tickets, reviews, and bids. The UI has entity results, links, query/page state, and error handling. The server bounds query length and page size, escapes ILIKE wildcards, restricts type names, parallelizes entity queries, and logs searches with a hashed IP. Existing pagination contracts cover its bounds. Enterprise readiness requires treating federated search as an authorization-sensitive data product: a single search permission must not become an alternate route to finance, private conversations, or other territories.

Evidence

UI: client/src/admin/pages/AdminSearch.jsx:29, :36, :107, :127. Route: server/routes/admin.js:281. Engine: server/controllers/adminSearchController.js:16, :27, :36, :52, :105, :153, :268, :303. Initial search-audit indexes: server/migrations/049_admin_search_and_permissions.sql:17, :20. Bounds contract: server/__tests__/adminPaginationContracts.test.js:8. Permission behavior: server/middleware/adminAuth.js:126.

Findings

- **T17-F01 - High [confirmed defect]: search exposes other domains without their permission checks.** The route requires global view_search_logs; the controller then searches requested/default types without finance, support, message, or user-management grants (adminSearchController.js:27, :52). Payment results include amounts, processor identifiers, and payer/payee emails (:153), and message results include content (:105). No participant or tenant/region predicate appears in those queries. This bypasses the intended separate domain permissions. Derive allowed types/fields from current task grants and intersect them with requested scope before any query.
- **T17-F02 - High [confirmed defect]: failed entity queries are returned as zero matches.** Promise.allSettled is sensible for partial results, but extract maps rejection to empty rows/zero total (:269). The response has no per-source failure distinction. An investigator can interpret a broken payment/message query as evidence that no record exists. Return per-entity available/denied/failed/stale status and a correlation reference while retaining successful results; do not reintroduce historic SQL assertions without current schema reproduction.
- **T17-F03 - High [code observation]: bounds limit returned rows, not federated query cost.** Up to eight wildcard queries run concurrently, often with COUNT(*) OVER() and OFFSET (:52, :61, :112, :166). At six million users and much larger message history, two-character substring searches and deep pages need measured index/query budgets. The English full-text index in migration 049 is on the search audit log, not evidence that all entity predicates are indexed. Inspect representative plans and choose exact ID/email/phone lookup, supported full-text/trigram strategies, or authorized search projections based on measured requirements.
- **T17-F04 - Medium [confirmed defect]: out-of-range pages can erase known totals.** Totals are read from the first returned row (:272); when a type has no row at the requested offset, its total becomes zero even if matching records exist on earlier pages. Keep totals independent of the page or return a cursor/hasMore contract. This matters when one shared page number spans entity types with different result counts.

Expansion scenario

A regional investigator searches a common surname during a payment complaint. They should receive permitted customer/case matches without unrelated private conversations or processor identifiers. A finance reviewer searches an exact payment reference and must know if the payment source failed rather than conclude no payment exists. A broad two-character query from hundreds of employees must not monopolize the transactional DB. Navigating the jobs page must not reset user totals simply because that entity has fewer matches at the same offset.

Exact recommended controls and UX

Apply field-level and entity-level rights inside the engine, with server-resolved tenant/region predicates and participant checks where relevant. Add exact-identifier search modes, minimum broad-query rules, cancellation/time budgets, query rate limits, and per-type cursors. Show denied versus unavailable versus zero results distinctly and keep authoritative record links. Use permission-aware lightweight projections or measured indexes; do not choose a separate search cluster without workload evidence. Log purpose/case, authorized scope, selected types, and outcome safely; raw query retention must follow reviewed privacy policy because queries can contain identifying information. Preserve hashed IP behavior and avoid secret/private payload logging.

Dependencies / owner

Search Engineering owns query correctness/performance; IAM and domain teams own authorization projections; SRE owns resource budgets; Privacy owns search-log minimization; Support/Finance define investigation workflows.

Measurable acceptance criteria

An operator with only search-log permission cannot obtain finance, support, private-message, or foreign-tenant content. Injected failure in one entity renders unavailable with a trace ID while other matches remain visible. Empty/out-of-range pages preserve correct totals or valid cursor semantics. Exact identifier p95 is below 500 ms and scoped broad search p95 below two seconds at 250 concurrent operators on representative synthetic/restored data. Search load causes less than 10% transactional latency degradation. Execute cross-domain negative-access, cancellation, keyboard, and five-language result tests without skips.

Completion and evidence boundary

All 17 requested tabs are covered with existing implementation, at least three stable findings, source pointers, expansion scenarios, controls, ownership, and measurable acceptance criteria. Prior local verification is user-reported as 310 suites / 3,339 tests passed, with build/typecheck success; it was not repeated here. Latest e7ac9415 GitHub CI and fresh-PostGIS apply-twice rehearsal succeeded. Both authenticated-browser tests skipped, and optional live DB verification skipped. The user-supplied 08:55 UTC live readiness observation was healthy with Credit enabled. No production-like restored-dataset rehearsal, worldwide provider acceptance, full live browser matrix, or target-scale benchmark is claimed completed. Null release provenance metadata remains an observability gap, not a claim that e7ac9415 is unpushed or that the running service is down.

18. Businesses - /admin/businesses

Mission and what exists

This tab moderates business accounts, allowing operators to locate organizations, inspect owners and activity, change active state, flag concerns, edit notes, and request deletion. It is an implemented screen, supported by business detail and broader business-management views, not a missing capability. The client has status/activity/sort controls, paged loading with request cancellation, preservation of previous results on fetch failure, and reason-entry confirmation. The server implements conditional state changes and soft deletion; these are useful foundations for controlled enterprise moderation rather than a reason to replace the system.

Evidence

- [client/src/admin/pages/BusinessModeration.jsx:229](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/BusinessModeration.jsx:229) implements abortable list loading; :274 sends moderation actions with a reason.
- [server/routes/admin.js:93](/Users/oxsteed/Documents/GitHub/6/server/routes/admin.js:93) protects business endpoints with admin authentication and manage_users.
- [server/controllers/businessModerationController.js:7](/Users/oxsteed/Documents/GitHub/6/server/controllers/businessModerationController.js:7) reads filters, :32 builds the result projection, :79 implements conditional toggling, and :430 requires deletion confirmation.
- [server/migrations/161_business_moderation_enhancements.sql:4](/Users/oxsteed/Documents/GitHub/6/server/migrations/161_business_moderation_enhancements.sql:4) adds moderation notes/deletion fields; :23 adds a supporting index.

Findings

T18-F01 - High [confirmed defect]: deletion confirmation contracts disagree. The client action body at [BusinessModeration.jsx:274](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/BusinessModeration.jsx:274) supplies {reason}; the deletion controller at [businessModerationController.js:430](/Users/oxsteed/Documents/GitHub/6/server/controllers/businessModerationController.js:430) additionally requires confirmed. The normal confirmed UI action therefore lacks a required API field. This is a current source-level mismatch, not an inference that the business screen is absent. A browser-to-controller contract test is needed before declaring the workflow usable.

T18-F02 - Medium [confirmed defect]: activity filtering and displayed metrics misrepresent the query. The controller destructures activity at :7 but does not use it in the predicates. Its projection sets category to null and earnings to zero at :37 and :43; job counts at :42 are attached through the owner rather than a demonstrated business attribution. Operators cannot safely interpret these values as verified business-level commercial activity. Either calculate the intended metrics or label their limits explicitly.

T18-F03 - High [code observation]: reasons and investigation records are weaker than the UI implies. The UI requires a reason, but the API accepts nullable reasons and catches audit failures (:100). Flagging at :352 is an audit action rather than a durable, assigned investigation queue. Notes update unconditionally at :384, permitting stale editors to overwrite each other. The conditional toggle at :79 deserves credit, but it does not provide an expected-version contract for all moderation changes.

T18-F04 - Medium [design requirement]: global moderation needs bounded, delegated work. List limits are parsed without a maximum at :9; the statistics query at :58 adds correlated activity work. The global permission boundary is secure against regional grants, but regional staff cannot use this workflow with target-scoped authority. Performance on millions of businesses and related jobs remains a measurement gap, not a measured outage.

Expansion scenario

A regional fraud team investigates 12,000 newly onboarded businesses after a referral campaign. Two analysts edit the same organization while headquarters reviews a deletion appeal. An ineffective activity filter changes prioritization, zero earnings can imply inactivity, and a lost note destroys handoff context. Granting every analyst global user-management authority would solve access friction by creating excessive reach, not safe delegation.

Recommended controls and UX

Repair the confirmation contract and test it end-to-end. Enforce structured reason codes plus bounded explanatory text on the API. Keep soft deletion, show affected memberships and dependent work, and require a reviewed recovery policy. Add expected versions, conflict diffs, append-only case notes, assigned investigator, appeal state, and due-time escalation to existing moderation records. Expose unavailable metrics as unavailable rather than zero. Bound page size, stabilize ordering with ID, and benchmark filtered counts; use asynchronous scoped exports and reviewed batch operations with per-record outcomes. Display the active regional scope and disable unauthorized actions with clear explanations. Provide labeled dialog controls, focus restoration, localized reason labels, and RTL-safe tables.

Dependencies and owner

Business Operations owns policy and case SLAs; Identity owns scoped permissions; business API owners own attribution and concurrency; Data/SRE owns query plans and job limits. Records management must define deletion recovery separately from engineering implementation.

Measurable acceptance criteria

All UI actions pass authenticated contract tests; missing reasons are rejected; stale notes return 409 without loss; restricted analysts cannot read or mutate another region by direct ID. A representative million-record dataset sustains 100 concurrent moderators with list p95 below one second and limits capped at 100. Every flag has an owner and deadline, every destructive decision has durable audit evidence, and keyboard-only users can complete and cancel every dialog.

19. Tool Rentals - /admin/tool-rentals

Mission and what exists

Tool Rentals is an existing moderation queue for peer equipment listings, not an absent rental system. It displays listing and owner information, supports search and availability filtering, and offers reason-confirmed Hide/Show actions with pagination. The wider application already includes bookings, reviews, favorites, invoices, owner terms, and peer-rental attestation. Enterprise expansion should connect moderation to those capabilities rather than duplicate booking or deposit machinery. The backend's conditional update protects a simultaneous availability change from a simple overwrite, which is a useful existing safeguard.

Evidence

- [client/src/admin/pages/ToolRentalModeration.jsx:87](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/ToolRentalModeration.jsx:87) loads the queue, :229 displays price, and :265 implements pagination.
- [server/routes/admin.js:120](/Users/oxsteed/Documents/GitHub/6/server/routes/admin.js:120) requires global manage_operations for rental moderation.
- [server/controllers/businessModerationController.js:118](/Users/oxsteed/Documents/GitHub/6/server/controllers/businessModerationController.js:118) lists rentals; :155 conditionally toggles availability and :182 records the optional reason.
- [server/migrations/039_user_skills_and_tool_rentals.sql:27](/Users/oxsteed/Documents/GitHub/6/server/migrations/039_user_skills_and_tool_rentals.sql:27) defines listing rates, deposits, local location, and availability.
- [server/__tests__/toolRentals.test.js:48](/Users/oxsteed/Documents/GitHub/6/server/__tests__/toolRentals.test.js:48) verifies owner availability updates preserve monetary fields with mocked queries; it does not test administrator-hide persistence.
- [server/routes/toolRentals.js:20](/Users/oxsteed/Documents/GitHub/6/server/routes/toolRentals.js:20) imports existing booking/review/invoice capabilities; :128 defines the owner-attestation version and :137 validates monetary input.

Findings

T19-F01 - High [confirmed defect]: owners can reverse an administrative hide through ordinary availability. The Hide/Show handler at [businessModerationController.js:155](/Users/oxsteed/Documents/GitHub/6/server/controllers/businessModerationController.js:155) changes is_available, also used by public visibility. The owner update at [server/routes/toolRentals.js:483](/Users/oxsteed/Documents/GitHub/6/server/routes/toolRentals.js:483) checks ownership but permits that same Boolean to be reset at :523; public browse requires availability at :322. Thus a hidden owner listing can be made visible through the normal update API, without a separate moderation hold. This is a current cross-path enforcement defect established from source, not a production exploit performed here. Preserve ordinary availability but separate administrator moderation state.

T19-F02 - Medium [code observation]: list pagination exists but its server budget is unbounded. The controller parses caller-supplied page and limit at :120 without a maximum, applies substring search at :124, and orders by creation timestamp at :141 without a unique tie-breaker. The client's normal 25-row pager does not constrain direct API calls. Deep offsets and count costs need representative query-plan verification before six-million-user readiness is claimed.

T19-F03 - High [code observation]: reason enforcement and audit durability are inconsistent. The dialog requires text, but :158 accepts an absent reason and the audit failure is caught at :184. An operator can therefore change listing availability through the API without the explanation demanded by the screen. Existing moderation does not expose a durable review owner, expiry, appeal outcome, or restoration checklist for this action. These are additions to the current queue, not evidence that rental management is missing.

T19-F04 - Medium [design requirement]: global equipment operations need locale and regional context. The client uses a dollar prefix at [ToolRentalModeration.jsx:229](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/ToolRentalModeration.jsx:229), hard-coded availability wording at :237, and an unlabeled icon close control at :39. Global manage_operations cannot be safely substituted for country-specific recall authority. Currency provenance and booking-impact context must be explicit before operators communicate deposit or rental consequences across markets.

Expansion scenario

A manufacturer recalls a generator model in three countries while 700 affected listings have future bookings. A local team must hide unsafe listings immediately without describing ordinary owner downtime as a recall, changing deposits blindly, or cancelling unrelated rentals. Headquarters needs a reproducible affected-list snapshot, and the next shift needs to distinguish restored equipment from pending evidence review.

Recommended controls and UX

Add a moderation status/reason/version alongside existing owner availability, and calculate effective public visibility from both. Use existing bookings to show affected reservations, settlement state, and contact tasks before action; do not automatically infer refunds from a visibility change. Require API reasons and durable action audit, create scoped recall cases with owners and review deadlines, and support appeal/restoration evidence. Bound limits at 100, add stable cursor ordering and indexed model/category/location filters, and provide asynchronous batch preview, execution, cancellation, and per-item conflict reports. Show currency codes and provider-supported monetary handling, localize dates and text, label dialog controls, and preserve keyboard focus after mutations. Jurisdiction and peer-rental agreement changes require separate specialist review, not an engineering claim of approval.

Dependencies and owner

Rental Trust/Safety owns recall decisions; booking/payment owners determine reservation and deposit effects; Identity supplies target scope; marketplace Data/SRE owns indexed lookup and job throttling; localization/accessibility owners validate operator interactions.

Measurable acceptance criteria

An owner update cannot clear an administrator hold; ordinary availability changes do not erase hold reasons. All moderation calls require structured reasons and generate durable audit entries. Regional operators fail direct-ID cross-region tests. A 100,000-listing recall preview runs as a bounded job with resumable item results, not one browser request. List p95 stays below one second at agreed peak staff load, and five supported locales plus keyboard-only navigation complete both Hide and restore flows.

20. Planned Needs - /admin/planned-needs

Mission and what exists

Planned Needs administers upcoming recurring or prearranged work before publication. The implemented screen lists schedules, exposes status and search controls, and offers cancellation of automatic activation or force cancellation with confirmation. It correctly warns that cancelling a planned need does not cancel an already published job. A date-only parsing helper avoids one common client timezone shift. The existing scheduler publishes work transactionally and checks for an already linked job, so the proper direction is hardening this lifecycle, not inventing another recurring-work engine.

Evidence

- [client/src/admin/pages/PlannedNeedsAdmin.jsx:24](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/PlannedNeedsAdmin.jsx:24) handles date-only values; :74 explains linked-job limitations; :134 fetches the queue and :155 submits reasons.
- [server/routes/admin.js:124](/Users/oxsteed/Documents/GitHub/6/server/routes/admin.js:124) protects the moderation endpoints with manage_operations.
- [server/controllers/businessModerationController.js:194](/Users/oxsteed/Documents/GitHub/6/server/controllers/businessModerationController.js:194) lists needs; :231 cancels auto activation and :268 force-cancels nonterminal needs.
- [server/migrations/156_planned_needs_pause.sql:1](/Users/oxsteed/Documents/GitHub/6/server/migrations/156_planned_needs_pause.sql:1) introduces is_paused and a partial supporting index.
- [server/jobs/plannedNeedsScheduler.js:32](/Users/oxsteed/Documents/GitHub/6/server/jobs/plannedNeedsScheduler.js:32) promotes impending work, :45 selects publication candidates, and :64 locks individual needs during publication.

Findings

T20-F01 - High [confirmed defect]: cancelling automatic activation does not establish a durable pause. The action at [businessModerationController.js:231](/Users/oxsteed/Documents/GitHub/6/server/controllers/businessModerationController.js:231) moves activating_soon back to planned but does not set is_paused. The scheduler at [plannedNeedsScheduler.js:32](/Users/oxsteed/Documents/GitHub/6/server/jobs/plannedNeedsScheduler.js:32) can promote unpaused planned records again, and publication selection at :45 includes planned records when due. An existing pause field therefore is not used by this administrative cancellation contract. The success response is stronger than the durable behavior.

T20-F02 - High [confirmed defect]: cancellation can race a previously selected publication candidate. The scheduler selects candidates before opening each item transaction. Its locked re-read at :64 checks ID and published_job_id, but does not revalidate cancelled status or pause before inserting a job at :98. A force cancellation committed after candidate selection but before that lock can still be followed by publication. The transaction prevents duplicate publication in one important case; it does not protect this lifecycle transition.

T20-F03 - Medium [confirmed defect]: queue errors resemble an empty workload. At [PlannedNeedsAdmin.jsx:134](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/PlannedNeedsAdmin.jsx:134), failed loading logs an error and sets needs to an empty array rather than retaining an actionable error state. Requests lack the cancellation/generation protection found in other moderation pages. Search responses arriving out of order can therefore replace current results, and a failing queue can be read as no pending work. This is an operational visibility defect, not proof of a present server outage.

T20-F04 - Medium [design requirement]: publication throughput needs explicit budgets and ownership. The publication candidate query at [plannedNeedsScheduler.js:45](/Users/oxsteed/Documents/GitHub/6/server/jobs/plannedNeedsScheduler.js:45) has no batch limit; the administrative controller also does not cap caller page size. The queue needs regional authorization, automation ownership, and clear scheduled-time semantics before worldwide delegated operation. Neither the existing per-item transaction nor healthy worker readiness establishes catch-up performance after an outage.

Expansion scenario

Severe weather shuts down outdoor work in six regions shortly before local publication time. Dispatchers cancel thousands of upcoming needs while the scheduler already holds a candidate snapshot. A newly published job can still notify a helper despite a successful cancellation message. At the next timezone boundary, previously reverted but unpaused records become eligible again, creating repeated manual intervention and customer confusion.

Recommended controls and UX

Define separate pause, cancel, and linked-job cancellation semantics using the existing fields. Revalidate status, pause, due time, and version under the same locked transaction that creates the job; use guarded updates and an idempotent publication identity. Return explicit transition results and linked-job IDs, with authorized navigation to the separate job workflow. Require API reasons and durable audit. Preserve stale results with a visible load error and retry, cancel superseded fetches, and display local due time with an explicit IANA zone and UTC detail. Introduce bounded worker batches, leases, retry/backoff, backlog age metrics, and scoped asynchronous emergency actions with dry-run counts and item outcomes. Provide accessible confirmation text distinguishing pause from irreversible cancellation.

Dependencies and owner

Work Planning owns lifecycle policy; Dispatch owns regional shutdown decisions; scheduler/API owners own atomic publication; Jobs owns already-published work; SRE owns backlog SLOs and safe worker concurrency.

Measurable acceptance criteria

An isolated database race test cancels between candidate selection and row lock and proves no job is created. Cancel-auto remains paused across two scheduler cycles unless deliberately resumed. Duplicate workers produce exactly one publication. A 50,000-item catch-up uses bounded batches and reports age, failures, and owner. Queue failures are visible, stale searches cannot replace current results, regional direct-ID mutations are rejected, and date-boundary tests cover DST and non-US timezones.

21. Payments - /admin/payments

Mission and what exists

Payments is an implemented financial investigation list showing payer/payee, job linkage, amount, fees, status, and timestamps. It supports paged status/date filtering, cancellable requests, permission/error messaging, and retry. Its backend caps page size and uses inclusive end-date semantics. Importantly, the surrounding refund system already has durable attempts, idempotency, reserved amounts, provider reconciliation, and monotonic accounting. Those capabilities must be surfaced to operators; describing the system as having no safe refund machinery would be inaccurate.

Evidence

- [client/src/admin/pages/PaymentsList.jsx:34](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/PaymentsList.jsx:34) loads paged filters; :120 renders error/retry; :193 renders financial records; :25 formats USD.
- [server/controllers/paymentsAdminController.js:6](/Users/oxsteed/Documents/GitHub/6/server/controllers/paymentsAdminController.js:6) bounds pagination and :21 selects the payment list projection.
- [server/routes/admin.js:134](/Users/oxsteed/Documents/GitHub/6/server/routes/admin.js:134) requires view_financials; [server/routes/payments.js:33](/Users/oxsteed/Documents/GitHub/6/server/routes/payments.js:33) additionally protects refunds with issue_refunds and limiting.
- [server/services/paymentRefundService.js:57](/Users/oxsteed/Documents/GitHub/6/server/services/paymentRefundService.js:57) reserves refunds under locks, :137 settles monotonic totals, and :213 uses provider idempotency.
- [server/migrations/597_payment_refund_reconciliation.sql:21](/Users/oxsteed/Documents/GitHub/6/server/migrations/597_payment_refund_reconciliation.sql:21) persists attempt state; [server/__tests__/paymentRefundReconciliation.test.js:32](/Users/oxsteed/Documents/GitHub/6/server/__tests__/paymentRefundReconciliation.test.js:32) covers idempotency and :38 cumulative partial refunds.

Findings

T21-F01 - High [code observation]: investigation presentation omits essential recovery state. The list projection at [paymentsAdminController.js:21](/Users/oxsteed/Documents/GitHub/6/server/controllers/paymentsAdminController.js:21) exposes payment/provider references but not durable refund-attempt status, reserved totals, reconciliation age, or a case owner. The UI at [PaymentsList.jsx:193](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/PaymentsList.jsx:193) cannot distinguish a safely reserved pending refund from a settled cumulative refund or a provider outcome awaiting local recovery. This is an operator-contract gap atop existing recovery services, not a claim that reconciliation itself is absent.

T21-F02 - High [code observation]: the list assumes currency without presenting provenance. [PaymentsList.jsx:25](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/PaymentsList.jsx:25) formats every amount as USD, while the list projection does not include currency. Refund state processing at [paymentRefundStateService.js:24](/Users/oxsteed/Documents/GitHub/6/server/services/paymentRefundStateService.js:24) already handles provider monetary evidence. The screen must either prove and label its USD-only product boundary or carry authoritative currency and minor-unit semantics. Expanding to other markets without that contract risks misleading support and reconciliation, even if the current product intentionally charges USD.

T21-F03 - High [design requirement]: worldwide financial delegation needs narrower visibility. The global view_financials grant correctly excludes regional-only grants, but a delegated global reader receives payer/payee details for all matching records. Read-finance, initiate-refund, reconcile, export, and approve-adjustment need independent scoped responsibilities. Display filters must not be mistaken for enforced tenant or regional authorization. Masked search results and just-in-time sensitive detail access would support routine local investigations without broad exposure.

T21-F04 - Medium [verification gap]: bounded pages do not prove large-ledger query capacity. The implementation caps limits at 100 and page at one million, which deserves credit. However, [payments AdminController.js:21](/Users/oxsteed/Documents/GitHub/6/server/controllers/paymentsAdminController.js:21) uses offset paging ordered by creation time alone, and :41 performs a matching joined count. Deep history, timestamp ties, and concurrent inserts require deterministic pagination and realistic cardinality tests. No measured p95 or production query plan is claimed here.

Expansion scenario

A provider timeout interrupts a partial refund during the European support shift; a delayed webhook arrives after a US analyst opens the same payment. The refund service can preserve financial correctness, yet an operator seeing only the original status and amount may retry through the wrong workflow or promise an incorrect resolution time. Regional access, cumulative totals, and recovery ownership matter as much as the provider response.

Recommended controls and UX

Extend the existing payment detail/investigation path with authorized attempt timelines, pending reservations, reconciled totals, provider event identifiers, last error category, and a designated recovery owner. Do not expose raw provider payloads or sensitive credentials. Use stable idempotency keys for retries and make uncertain outcomes read-only until reconciliation determines eligibility. Present currency codes, localized formatting, explicit timezone, and settled versus pending amounts. Add target-scoped finance policies, masked lists, reasoned reveal/export, and dual review for defined high-value adjustments. Adopt cursor ordering, indexed status/date/provider lookup, bounded asynchronous exports, and reconciliation-age alerts. Provider availability, supported markets, refund rules, and legal obligations require separate provider/jurisdiction review.

Dependencies and owner

Finance Operations owns case resolution and adjustment policy; Payments owns provider state and idempotency; Identity owns scoped permissions; Security owns sensitive-data exposure; SRE/Data owns ledger query and reconciliation-age monitoring.

Measurable acceptance criteria

Timeout/retry/webhook integration tests reconcile exactly one financial outcome and show its attempt timeline. Two analysts cannot refund beyond remaining eligible cents. Every displayed amount has verified currency semantics. Regional readers cannot retrieve foreign-region records or exports. At representative retained ledger volume, first-page and provider-reference lookup p95 remain below one second; exports are asynchronous and audited. Recovery cases beyond the agreed SLA have an owner, alert, and documented safe next action.

22. Orders - /admin/orders

Mission and what exists

Orders provides a real marketplace operations screen with list filters, totals, paging, detail dialogs, shipping, cancellation, and refund actions. The client uses cancellable query-backed loading and explicit error presentation. Server list parameters have bounded pagination, parameterized search, and an allowlisted sort. The customer-search count uses matching joins, and its focused regression test exists; this report does not recycle the historical count defect. Marketplace refund recovery and database state-transition guards are also implemented, so recommendations should improve their operational use rather than introduce competing order or refund systems.

Evidence

- [client/src/admin/pages/OrderManagement.jsx:25](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/OrderManagement.jsx:25) loads orders/stats; :70 handles errors and :175 renders pagination.
- [server/routes/adminOrders.js:8](/Users/oxsteed/Documents/GitHub/6/server/routes/adminOrders.js:8) requires manage_operations; :23 additionally requires issue_refunds.
- [server/controllers/adminOrdersController.js:10](/Users/oxsteed/Documents/GitHub/6/server/controllers/adminOrdersController.js:10) bounds list parameters, :116 handles detail, :229 calls the transaction helper, and :339 repeats that call pattern for cancellation.
- [server/utils/transaction.js:39](/Users/oxsteed/Documents/GitHub/6/server/utils/transaction.js:39) defines the helper signature as (pool, operation, name).
- [server/migrations/603_marketplace_order_state_integrity.sql:18](/Users/oxsteed/Documents/GitHub/6/server/migrations/603_marketplace_order_state_integrity.sql:18) guards transitions; [server/__tests__/adminOrdersController.test.js:10](/Users/oxsteed/Documents/GitHub/6/server/__tests__/adminOrdersController.test.js:10) tests repaired customer-search counting.

Findings

T22-F01 - High [confirmed defect]: shipping and cancellation call the transaction helper incorrectly. [adminOrdersController.js:229](/Users/oxsteed/Documents/GitHub/6/server/controllers/adminOrdersController.js:229) supplies only an async callback to the imported helper, while [transaction.js:39](/Users/oxsteed/Documents/GitHub/6/server/utils/transaction.js:39) expects a pool first and calls pool.connect() at :40. Cancellation at [adminOrdersController.js:339](/Users/oxsteed/Documents/GitHub/6/server/controllers/adminOrdersController.js:339) has the same mismatch. With this implementation, the callback is treated as the pool and cannot supply connect. This is deterministic source evidence of an essential mutation-path defect, not a claim that a production shipment was attempted during this audit. The list-focused test does not exercise either mutation.

T22-F02 - High [confirmed defect]: completed refund cases cannot be investigated through normal detail. The GET detail handler at [adminOrdersController.js:116](/Users/oxsteed/Documents/GitHub/6/server/controllers/adminOrdersController.js:116) returns 409 for refunded orders. Blocking new financial mutation may be appropriate, but blocking read-only detail prevents operators from reviewing an already refunded order, its shipment, and the reason for resolution. The list and refund-service safeguards should be accompanied by immutable case inspection, not disappearance of resolved financial context.

T22-F03 - Medium [code observation]: shipment changes lack an explicit stale-editor contract. The shipment handler reads the order at :230 without a row lock and updates it at :245 without an expected version. Migration 603 supplies important terminal-state protection; it does not make two operators' tracking edits conflict visibly or preserve the client's observed state. After repairing the helper, this path needs transactional transition validation and a versioned mutation response rather than assuming the database trigger covers every operational race.

T22-F04 - Medium [design requirement]: operational scale needs scoped, asynchronous exception handling. Existing bounded lists are valuable, but deep offsets, synchronous per-order work, and broad global operations authority are not a regional incident workflow. Thousands of cancellations, address issues, or carrier exceptions require item-level ownership and safe batch processing. Performance,

bulk recovery, and cross-region denial remain acceptance work, not assertions of current downtime.

Expansion scenario

A hurricane closes a fulfillment region with 20,000 pending or partially shipped orders. Local supervisors need to cancel eligible unpaid work, route paid orders to the existing refund process, and preserve carrier evidence for orders already resolved. Headquarters and a night shift inspect the same case. A broken cancellation call halts triage, while refusing refunded detail obscures the financial history needed to explain the outcome.

Recommended controls and UX

Repair and directly test both transaction-helper callers before expanding operational access. Lock the target order, enforce an explicit state transition, accept an expected version, and return a conflict with current state and a readable diff. Keep refund idempotency and terminal guards; make refunded detail readable with all mutations appropriately disabled. Add carrier/tracking validation, durable before/after audit, scoped case assignment, age/SLA indicators, and an authorized timeline joining existing refunds and shipments. Provide asynchronous scoped batch preview with paid/unpaid eligibility, resumable jobs, and per-order outcomes; never implement cancellation as an implicit refund. Use stable pagination, audited exports, localized currency/date semantics, and accessible dialogs. Provider and consumer-policy decisions remain specialist approvals.

Dependencies and owner

Marketplace Operations owns fulfillment exceptions; Orders API owns transaction and version contracts; Payments owns refunds; Identity owns regional delegation; SRE/Data owns large-order query budgets and batch reliability.

Measurable acceptance criteria

Database-backed shipping/cancellation tests pass through the real helper. Invalid and terminal transitions are rejected, concurrent tracking edits return a conflict, and refunded detail remains readable. Two retries cannot duplicate a refund or state change. A 20,000-order incident job resumes after worker restart and reports each outcome. Regional direct-ID and export access tests deny out-of-scope records, and normal filtered lists meet a proposed one-second p95 at retained production-like volume.

23. Templates - /admin/templates

Mission and what exists

Templates is an implemented administration workspace for template preview, pricing, access grants, revocation, and usage statistics. The client includes grant search, paged grant loading, pricing editors, confirmation flows, and template previews. Individual and bulk pricing endpoints already exist; bulk pricing uses a transaction. The server intentionally treats acquisition as monthly access rather than the former lifetime-purchase path. Enterprise controls should consolidate those current entitlement semantics, preserve legitimate existing access, and improve release governance rather than create another template marketplace.

Evidence

- [client/src/admin/pages/TemplateManagement.jsx:60](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/TemplateManagement.jsx:60) implements preview dialogs; :464 uses one-based grant paging and :497 keeps pricing errors actionable.
- [server/routes/templates.js:28](/Users/oxsteed/Documents/GitHub/6/server/routes/templates.js:28) protects pricing with manage_settings; :31 protects stats with manage_operations; :34, :43, and :46 protect grants only with admin authentication.
- [server/controllers/templateController.js:192](/Users/oxsteed/Documents/GitHub/6/server/controllers/templateController.js:192) deprecates lifetime purchase, :201 updates pricing, :246 performs transactional bulk updates, and :290 creates access grants.
- [client/scripts/adminAccessibilityContracts.test.mjs:38](/Users/oxsteed/Documents/GitHub/6/client/scripts/adminAccessibilityContracts.test.mjs:38) checks template refresh/close labels statically, not focus behavior or entitlement mutation integrity.
- [server/controllers/templateController.js:345](/Users/oxsteed/Documents/GitHub/6/server/controllers/templateController.js:345) revokes access and resets active templates; :380 lists grants; :457 aggregates usage.

Findings

T23-F01 - High [confirmed defect]: grant authority is less restricted than pricing authority. Grant list, creation, and revocation routes at [templates.js:34](/Users/oxsteed/Documents/GitHub/6/server/routes/templates.js:34), :43, and :46 omit a permission check beyond admin identity. A general admin without manage_settings can still create or remove valuable entitlements by direct API call. Pricing and statistics have explicit grants, so this is a concrete inconsistency in the existing permission design, not an absence of authentication. Introduce a distinct entitlement-management responsibility with target scope.

T23-F02 - High [confirmed defect]: revoking one access row resets profiles without evaluating remaining access. [templateController.js:349](/Users/oxsteed/Documents/GitHub/6/server/controllers/templateController.js:349) revokes the selected row, then :361 and :366 reset all matching business/helper active templates for that user. It does not check whether another valid grant or subscription still authorizes the same slug, and these separate writes are not one transaction. A promotion revocation can therefore disturb legitimate paid or overlapping access, while a partial failure can leave entitlement and presentation state inconsistent.

T23-F03 - Medium [code observation]: pricing and grant changes lack mature validation/versioning. Pricing updates at :201 accept incoming monetary fields without the controller proving strict integer bounds or an expected version. Grant creation at :290 checks grant types and required dates, but does not establish valid ordered date intervals. Existing database constraints may reject some invalid values; they do not replace actionable field validation. Bulk updates at :246 lack an explicit array budget, preview, or per-item decision report.

T23-F04 - Medium [code observation]: business semantics and discovery can mislead operators. Lifetime pricing remains editable in [TemplateManagement.jsx:223](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/TemplateManagement.jsx:223) although the purchase path at [templateController.js:1

92](/Users/oxsteed/Documents/GitHub/6/server/controllers/templateController.js:192) declares monthly semantics. The grant-entry interface suggests ID/email entry at [TemplateManagement.jsx:310](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/TemplateManagement.jsx:310), while submission at :291 forwards user_id unchanged and [server/controllers/templateController.js:315](/Users/oxsteed/Documents/GitHub/6/server/controllers/templateController.js:315) queries only the user identifier. Server grant paging exists, but usage aggregation at [templateController.js:457](/Users/oxsteed/Documents/GitHub/6/server/controllers/templateController.js:457) counts active rows without demonstrating effective expiry semantics. Labels and metrics must match the entitlement resolver.

Expansion scenario

A global campaign grants 80,000 trial extensions while enterprise customers retain paid access. One coordinator revokes a mistaken trial grant and another adjusts regional pricing. Resetting the customer's active template despite a still-valid paid entitlement creates an avoidable commercial incident; stale price updates can silently overwrite each other. Support needs the effective entitlement explanation, not only a list of grant rows.

Recommended controls and UX

Reuse the existing access model but centralize effective entitlement evaluation. Revoke transactionally, then reset presentation only when no valid remaining entitlement permits it. Require scoped entitlement permissions, reason codes, durable audit, expected versions, and validated date intervals. Distinguish monthly price, historical lifetime terms, and grandfathered access visibly; resolve email searches through an authorized lookup instead of sending them as identifiers. Build price-change preview and scheduled publication with explicit currency, affected cohorts, review approval, and rollback to the previous version. Convert large grants and bulk price changes to bounded asynchronous jobs with deduplication, cancellation, and per-user outcomes. Preserve the current preview dialogs while verifying focus, translated labels, and permission-specific loading rather than failing the whole workspace when one panel is unauthorized.

Dependencies and owner

Product Monetization owns entitlement policy; Templates API owns effective access and transactions; Finance reviews pricing treatment; Identity owns delegation; Customer Success owns campaign exceptions and recovery communication.

Measurable acceptance criteria

An admin lacking entitlement authority cannot grant or revoke through direct calls. Revoking an overlapping trial preserves valid paid access; injected transaction failure leaves no partial reset. Invalid cents/date ranges return field errors and stale prices return 409. An 80,000-grant job resumes without duplicates. Usage metrics match effective entitlements, ID/email searches behave as labeled, and pricing terms are unambiguous across supported locales.

24. Consulting - /admin/consulting

Mission and what exists

Consulting provides an existing operational dashboard with request discovery, status/tier/search controls, manual intake, consultant assignment, notes, and confirmation dialogs. It fetches request, consultant, and aggregate data with cancellation support. The server can filter requests and maintains consultant records with active-work counts. This is a working intake/assignment framework, not a missing consulting module. Enterprise expansion should join this dashboard to the existing scheduling, sessions, and payment capabilities so ownership and delivery evidence follow a case throughout its lifecycle.

Evidence

- [client/src/admin/pages/ConsultingDashboard.jsx:364](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/ConsultingDashboard.jsx:364) requests the latest 200 items and performs local filtering; :213 changes status, :230 assigns, and :243 patches notes.
- [server/routes/consultingAdmin.js:7](/Users/oxsteed/Documents/GitHub/6/server/routes/consultingAdmin.js:7) defines admin authentication; :13 through :19 expose request and assignment operations without additional permission grants.
- [server/controllers/adminConsultingController.js:16](/Users/oxsteed/Documents/GitHub/6/server/controllers/adminConsultingController.js:16) implements server request filters; :62 assigns consultants; :101 lists consultant workload.
- [server/controllers/adminConsultingController.js:296](/Users/oxsteed/Documents/GitHub/6/server/controllers/adminConsultingController.js:296) updates details and :526 handles status changes, including a status allowlist at :534 and audit at :566.
- [server/migrations/103_consulting_all_phases.sql:36](/Users/oxsteed/Documents/GitHub/6/server/migrations/103_consulting_all_phases.sql:36) adds consultant assignment and :46 introduces scheduled appointments.

Findings

T24-F01 - High [confirmed defect]: local filtering conceals older requests. [ConsultingDashboard.jsx:364](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/ConsultingDashboard.jsx:364) fetches only 200 records and filters that array by search, tier, and status. There is no complete queue paging in this dashboard. A pending lead older than the newest 200 requests is invisible even if the server supports the desired filter. At expansion volume this is a queue-completeness failure, not merely slow rendering; the screen can suggest no matching workload while undisplayed work exists.

T24-F02 - High [code observation]: ordinary admin identity grants broad consulting mutation. [consultingAdmin.js:13](/Users/oxsteed/Documents/GitHub/6/server/routes/consultingAdmin.js:13)-:19 require authentication and requireAdmin, but no request-read, assignment, or transition permission. Financial routes elsewhere have grants, showing that granular enforcement exists but is not applied here. Neither UI filtering nor consultant selection establishes tenant/country scope. The issue is least privilege for internal administration, not unauthenticated access.

T24-F03 - High [code observation]: status and notes are vulnerable to stale decisions. The controller reads then updates status without expected-version matching at [adminConsultingController.js:526](/Users/oxsteed/Documents/GitHub/6/server/controllers/adminConsultingController.js:526), despite validating the enum. Notes submitted by the client at [ConsultingDashboard.jsx:243](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/ConsultingDashboard.jsx:243) are patched as a field, and detail update at [adminConsultingController.js:296](/Users/oxsteed/Documents/GitHub/6/server/controllers/adminConsultingController.js:296) replaces the value rather than appending an immutable note event. Concurrent coordinators can overwrite case context or apply a decision based on an obsolete assignment.

T24-F04 - Medium [design requirement]: active-work counts are not staffing availability or SLA control. Consultant listing at [adminConsultingController.js:101](/Users/oxsteed/Documents/GitHub/6/server/controllers/adminConsultingController.js:101) counts pending/scheduled work; assignment at :62 does not use the separate availability/blocked-time booking rules. A global workforce needs language, expertise,

capacity, regional operating hours, backup ownership, and escalation. The existing consultant model is the appropriate foundation, but an active-count badge is insufficient proof that a consultant can accept a specific appointment.

Expansion scenario

A launch campaign generates 50,000 consultation requests over a weekend. Older urgent leads disappear from the dashboard's newest-200 slice. The overnight team assigns apparently underloaded consultants who are unavailable in the client's timezone, while two coordinators replace the same notes. Monday leadership sees aggregate activity but cannot identify which uncontacted clients have exceeded the promised response time.

Recommended controls and UX

Use server-side filter/search with bounded cursor paging and honest total/backlog indicators. Add independent scoped permissions for intake reads, assignment, and status mutation. Accept expected versions, return current-state conflict diffs, and convert notes to append-only attributed events with correction links. Define an explicit transition graph and preserve audit with the mutation. Connect assignment to existing availability, appointment duration, expertise/language, and workload reservations; do not create a second availability engine. Add assigned owner/team, first-response deadline, escalation, workload aging, handoff notes, and reassignment reasons. Offer reviewed asynchronous campaign actions with dry-run counts, bounded jobs, and per-case outcomes. Localize timestamps and statuses and ensure modal focus and keyboard navigation remain usable in a densely populated queue.

Dependencies and owner

Consulting Operations owns intake SLAs and skill routing; scheduling owners provide reservable capacity; Identity owns scoped permissions; consulting API owners provide transition/version contracts; analytics owners define truthful backlog measures.

Measurable acceptance criteria

An old pending case remains discoverable after 50,000 newer requests. Scoped coordinators cannot read or assign another region's case. Simultaneous edits produce 409 rather than losing a note or assignment. Unavailable consultants cannot be reserved through dashboard shortcuts. Every open request has an owner and due time, overdue work escalates within the agreed interval, and server-filtered queue p95 stays below one second at representative retained volume.

25. Consulting Pay - /admin/consulting/payments

Mission and what exists

Consulting Pay combines package management, purchase/invoice history, and manual invoice creation. The UI validates package input, supports active-state toggles, and uses optimistic rollback for some failures. The payment API already validates session ownership, prevents competing active payments, uses deterministic provider idempotency, verifies actual provider amount/currency/success, and integrates durable external-refund recovery. Database migrations and focused payment tests support these repairs. The enterprise question is whether staff can safely administer and investigate those existing financial flows, not whether consulting payments exist.

Evidence

- [client/src/admin/pages/ConsultingPayments.jsx:9](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/ConsultingPayments.jsx:9) defines invoice tier defaults, :36 formats USD, :126 submits manual invoices, and :205 loads the workspace.
- [server/routes/consultingAdmin.js:22](/Users/oxsteed/Documents/GitHub/6/server/routes/consultingAdmin.js:22) permits invoice generation with view_financials; :27 protects package editing only with admin authentication.
- [server/controllers/adminConsultingController.js:619](/Users/oxsteed/Documents/GitHub/6/server/controllers/adminConsultingController.js:619) limits purchases to 100, :638 limits invoices to 200, and :663 generates manual invoices.
- [server/controllers/consultingPaymentController.js:26](/Users/oxsteed/Documents/GitHub/6/server/controllers/consultingPaymentController.js:26) guards payment creation, :150 verifies provider confirmation, and :272 authorizes refunds by role; [server/routes/consultingPayment.js:15](/Users/oxsteed/Documents/GitHub/6/server/routes/consultingPayment.js:15) has authentication/limiting but no refund grant.
- [server/__tests__/consultingPaymentStateMigration.test.js:12](/Users/oxsteed/Documents/GitHub/6/server/__tests__/consultingPaymentStateMigration.test.js:12) checks terminal-trigger SQL text; this is not a restored-data payment rehearsal.
- [server/migrations/599_consulting_payment_terminal_integrity.sql:26](/Users/oxsteed/Documents/GitHub/6/server/migrations/599_consulting_payment_terminal_integrity.sql:26) protects terminal state; [server/__tests__/ws3-consulting-chat.test.js:70](/Users/oxsteed/Documents/GitHub/6/server/__tests__/ws3-consulting-chat.test.js:70) exercises repaired payment creation contracts.

Findings

T25-F01 - High [confirmed defect]: financial responsibilities are not consistently separated. Invoice creation uses the read-oriented view_financials grant, package changes require only admin identity, and consulting refunds rely on a role check at [consultingPaymentController.js:272](/Users/oxsteed/Documents/GitHub/6/server/controllers/consultingPaymentController.js:272) rather than the issue_refunds plus requireAdmin boundary used by ordinary payments. This permits an admin without that refund grant to reach consulting refund logic. Durable refund recovery remains valuable, but correctness of provider execution does not establish authority to initiate it.

T25-F02 - High [confirmed defect]: manual invoice lifecycle loses recipient provenance. The controller receives name/email/tier at [adminConsultingController.js:664](/Users/oxsteed/Documents/GitHub/6/server/controllers/adminConsultingController.js:664), but the INSERT at :689 stores none of those recipient/package fields. The email call uses transient request data and marks the invoice sent before delivery, with failures merely logged at :702. After reload, manual invoices without a linked user cannot reconstruct the intended recipient from the normal user join. The UI at [ConsultingPayments.jsx:126](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/ConsultingPayments.jsx:126) also does not supply the optional idempotency key supported by the controller, leaving routine retries unprotected by that mechanism.

T25-F03 - Medium [confirmed defect]: administrative package discovery excludes inactive products. The workspace at [ConsultingPayments.jsx:205](/Users/oxsteed/Documents/GitHub/6/client/src/a

dmin/pages/ConsultingPayments.jsx:205) loads the public packages endpoint, which returns only active packages ([consultingPaymentController.js:7](/Users/oxsteed/Documents/GitHub/6/server/controllers/consultingPaymentController.js:7)). Once disabled and reloaded, a package disappears from the administrative list and cannot be reenabled there. Additionally, the editor catch at [ConsultingPayments.jsx:227](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/ConsultingPayments.jsx:227) does not propagate save failure like its toggle handler, allowing the dialog to close despite unsuccessful persistence.

T25-F04 - Medium [code observation]: history and invoice amounts are incomplete operational contracts. Purchase/invoice limits of 100/200 have no cursor or complete search, so older financial cases disappear. Manual invoice options use hardcoded tier prices at [ConsultingPayments.jsx:9](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/ConsultingPayments.jsx:9) rather than authoritative current package/version data. USD formatting must be labeled as a supported product constraint or generalized with verified currency; global expansion does not itself authorize another provider or accounting regime.

Expansion scenario

An enterprise onboarding campaign generates 5,000 manual invoices. Email delivery fails for one region, and coordinators retry creation after browser timeouts. The saved records cannot reliably reveal the intended recipients, old invoices drop out of history, and disabled campaign packages vanish from management. A read-finance employee should investigate these outcomes without automatically possessing invoice-creation or refund authority.

Recommended controls and UX

Add scoped create-invoice, manage-package, initiate-refund, and read-finance permissions with consistent active-admin/MFA enforcement. Persist recipient snapshot, package/version, currency, delivery state, and a mandatory client idempotency key. Use a delivery outbox with retry ownership and distinguish generated, queued, delivered, paid, and failed states. Provide an admin package endpoint including inactive entries; retain failed editors and show field errors. Page/search full history and expose existing payment/refund attempt timelines. Resolve invoice pricing from the selected authoritative version, with explicit authorized overrides and reasons. Require provider-supported currency and market review separately from engineering approval, and support accessible localized financial dialogs.

Dependencies and owner

Finance Operations owns invoice/refund authority; Consulting Monetization owns package versions; Payments owns provider recovery; Messaging owns delivery evidence; Identity and Security own consistent grant/MFA enforcement.

Measurable acceptance criteria

Read-only finance cannot generate invoices or refund through direct APIs. Retrying the same invoice creates one record and preserves its recipient after reload. Delivery failures are visible and retryable without recreating the invoice. Disabled packages remain administratively editable. Cases beyond the first 200 rows are searchable. Real-database/provider-sandbox race tests confirm one active payment and monotonic refund state, while browser acceptance verifies failure dialogs, localization, and permission boundaries.

26. Scheduling - /admin/consulting/scheduler

Mission and what exists

Scheduling presents an existing consulting calendar with assignment, scheduling, cancellation, reminder, and invitation actions. It loads requests and consultants with cancellation support and provides manual session creation. Separately, the server already implements consultant availability, blocked dates, exceptions, ownership checks, platform approval, slot validation, and overlap-aware booking. That distinction is important: enterprise readiness requires routing administrative scheduling through the protected booking contract, not building a second scheduler or claiming availability does not exist.

Evidence

- [client/src/admin/pages/ConsultingScheduler.jsx:19](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/ConsultingScheduler.jsx:19) defines fixed hours, :65 renders calendar events, :102 submits schedules, and :340 loads only 200 requests.
- [server/controllers/adminConsultingController.js:526](/Users/oxsteed/Documents/GitHub/6/server/controllers/adminConsultingController.js:526) patches request status and scheduled timestamp through the generic administrative path.
- [server/controllers/consultantSchedulingController.js:207](/Users/oxsteed/Documents/GitHub/6/server/controllers/consultantSchedulingController.js:207) validates booking inputs, :236 checks ownership, :257 checks consultant approval, :280 validates availability, and :293 guards overlap in an update.
- [server/migrations/107_consultant_availability.sql:66](/Users/oxsteed/Documents/GitHub/6/server/migrations/107_consultant_availability.sql:66) defines available-slot computation and the same migration adds timezone/availability infrastructure.
- [server/routes/consultingAdmin.js:38](/Users/oxsteed/Documents/GitHub/6/server/routes/consultingAdmin.js:38) maps send-invite to the reminder handler; [server/controllers/adminConsultingController.js:815](/Users/oxsteed/Documents/GitHub/6/server/controllers/adminConsultingController.js:815) implements that reminder.

Findings

T26-F01 - High [confirmed defect]: administrative scheduling bypasses the existing booking validation. [ConsultingScheduler.jsx:110](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/ConsultingScheduler.jsx:110) PATCHes the generic request-status endpoint with scheduled_at and a selected consultant. [adminConsultingController.js:526](/Users/oxsteed/Documents/GitHub/6/server/controllers/adminConsultingController.js:526) updates that parent record, rather than invoking the availability/blocked-date/approval and scheduled-child booking path at [consultantSchedulingController.js:207](/Users/oxsteed/Documents/GitHub/6/server/controllers/consultantSchedulingController.js:207). A successful calendar operation therefore does not establish a valid reserved appointment. This is a source-level contract divergence, not proof that all public bookings are unsafe.

T26-F02 - High [code observation]: displayed time has implicit operator-local semantics. Fixed 09:00-17:00 options and date construction at [ConsultingScheduler.jsx:19](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/ConsultingScheduler.jsx:19) and :102 lack an explicit scheduling-zone selector and duration control. The separate booking endpoint at [consultantSchedulingController.js:315](/Users/oxsteed/Documents/GitHub/6/server/controllers/consultantSchedulingController.js:315) defaults missing timezone to America/New_York. A UTC timestamp alone is insufficient UX evidence of the consultant/client's intended local appointment, particularly at DST transitions. Preserve the existing timezone field but make its interpretation explicit.

T26-F03 - Medium [confirmed defect]: calendar completeness and invitation labeling overstate behavior. Requests are capped at 200 before calendar placement ([ConsultingScheduler.jsx:340](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/ConsultingScheduler.jsx:340)), rather than queried by visible calendar window. Older records scheduled in the current week can be absent. The send-invite action at :128 reaches a route mapped to sendSessionReminder, not a distinct calendar invitation/response lifecycle. Reminder delivery is a useful feature, but the UI must not imply accepted ICS/provider-calendar synchronization without implementing and verifying it.

T26-F04 - High [verification gap]: overlap SQL is not proven safe under concurrent distinct appointments. The existing conditional NOT EXISTS update at [consultantSchedulingController.js:293](/Users/oxsteed/Documents/GitHub/6/server/controllers/consultantSchedulingController.js:293) rejects a visible overlap, but two transactions updating different target rows may both read no competing committed booking. A consultant-level lock or database exclusion/reservation invariant needs inspection and a real concurrent test. Do not call this a confirmed double-booking incident or dismiss the existing sequential conflict check.

Expansion scenario

A London coordinator schedules a Sydney consultant for a New York client around a US daylight-saving change. Another coordinator books the same consultant through the dedicated booking endpoint. The calendar shortcut writes only the parent time, fixed work hours obscure the consultant's actual zone, and a reminder labeled invite does not reveal whether the client accepted. The worldwide workforce needs one authoritative reservation and an explainable timezone conversion.

Recommended controls and UX

Use the existing availability service for every administrative reservation with target-scoped assignment authority. Reserve consultant/time capacity transactionally, validate duration and both timezone offsets, and require an expected appointment version for moves/cancellations. Add explicit IANA consultant/client/display zones with UTC detail and DST ambiguity choices; derive hours from availability rather than fixed office defaults. Query bounded calendar windows and render all matching appointments through paged/virtualized results. Distinguish reminder from invitation; add provider-supported ICS lifecycle, delivery state, response tracking, cancellation identity, and retries only if that capability is approved. Provide keyboard-operable calendar events, labeled buttons, visible focus, an accessible agenda alternative, and translated time/status labels. Measure slot-query fanout at [consultantSchedulingController.js:167](/Users/oxsteed/Documents/GitHub/6/server/controllers/consultantSchedulingController.js:167) before expanding bulk scheduling.

Dependencies and owner

Scheduling API owns the reservation invariant; Consulting Operations owns capacity and reschedule policy; Messaging owns delivery; provider integration owners approve calendar capabilities; Identity owns scoped assignment; accessibility/localization owners validate agenda/calendar parity.

Measurable acceptance criteria

Every calendar action reserves the same authoritative child appointment as public booking. Two concurrent distinct bookings for overlapping time yield one success and one conflict. DST tests cover nonexistent and repeated local times, and visible calendar windows include records older than the latest 200 requests. Invitations and reminders have accurate distinct labels and delivery states. Keyboard users can schedule, move, inspect, and cancel; regional staff cannot reserve unauthorized consultants or sessions.

27. Business Launch - /admin/consulting/business-launch

Mission and what exists

Business Launch is an implemented consulting intake and referral workspace, with country/state/status/package/urgency/search filters, pagination, request detail, notes, messages, consultant assignment, and CSV export. The client uses abortable loading and explicit confirmations. Server assignment is transactional, checks contractor eligibility and referral limits, and records referral history; status changes include audit. These are meaningful capabilities. Expansion should strengthen query correctness, delegated ownership, capacity reservation, and founder communication rather than label the screen missing or create a separate launch CRM.

Evidence

- [client/src/admin/pages/BusinessLaunchAdmin.jsx:96](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/BusinessLaunchAdmin.jsx:96) loads filtered pages; :124 handles confirmations and :144 exports the current page.
- [server/routes/businessLaunch.js:49](/Users/oxsteed/Documents/GitHub/6/server/routes/businessLaunch.js:49) protects administration with manage_operations; :109 requires message_users for custom messaging.
- [server/controllers/businessLaunchController.js:150](/Users/oxsteed/Documents/GitHub/6/server/controllers/businessLaunchController.js:150) lists requests, :204 projects joined rows, :358 transactionally assigns contractors, and :517 changes status.
- [server/controllers/businessLaunchController.js:396](/Users/oxsteed/Documents/GitHub/6/server/controllers/businessLaunchController.js:396) checks referral capacity; :534 reads existing state and :557 writes audit in the status transaction.
- [server/migrations/155_consulting_admin_improvements.sql:59](/Users/oxsteed/Documents/GitHub/6/server/migrations/155_consulting_admin_improvements.sql:59) expands launch-related statuses; [server/migrations/103_consulting_all_phases.sql:226](/Users/oxsteed/Documents/GitHub/6/server/migrations/103_consulting_all_phases.sql:226) establishes consulting referral records.

Findings

T27-F01 - High [code observation]: the current joined list SQL contains unqualified shared columns. The projection at [businessLaunchController.js:204](/Users/oxsteed/Documents/GitHub/6/server/controllers/businessLaunchController.js:204) selects id, email, first_name, last_name, and timestamps without the cr prefix, while joining users cu at :218. Those names are shared by the request/user models and are therefore an ambiguity risk in the actual current statement, independent of previously repaired SQL elsewhere. This is source evidence requiring an isolated database regression test; no production list failure was reproduced and it must not be described as an observed live outage.

T27-F02 - High [verification gap]: contractor capacity is checked without proving a shared reservation lock. Assignment locks the request and validates active contractor acceptance, minimum price, and referral count at :358-:406. Two different requests can compete for the same contractor's final capacity slot because locking one request does not serialize the contractor-level aggregate. Existing transactional referral logging deserves credit; its capacity invariant needs a concurrent Postgres test and a contractor-level reservation strategy if it fails.

T27-F03 - Medium [code observation]: stale status decisions and export completeness need clearer contracts. Status mutation reads then writes without expected-version matching at [businessLaunchController.js:534](/Users/oxsteed/Documents/GitHub/6/server/controllers/businessLaunchController.js:534), allowing stale decisions to overwrite a newer phase while audit references an old observation. The CSV at [BusinessLaunchAdmin.jsx:144](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/BusinessLaunchAdmin.jsx:144) exports only the displayed page and uses quoting without demonstrated spreadsheet-formula neutralization. The UI should label page-only export or offer a scoped full-result job, particularly for founder-supplied text.

T27-F04 - High [design requirement]: country filters are not jurisdiction-scoped delegation. Global manage_operations guards the endpoints, but selected country/state filters are caller input rather than authorization scope. Launch coordinators need bounded regional access, a responsible consultant/team, phase due dates, required evidence, and backup ownership. Setup-item labels are operational intake choices; they do not establish local legal authorization or completion of jurisdiction-specific obligations.

Expansion scenario

Two launch managers in Kenya and Haiti compete for a contractor's last referral slot during a holiday surge. One changes urgency while another completes the case from stale detail. A regional coordinator exports founder names containing spreadsheet control characters to send an approved campaign. Correct referral reservations, complete export labeling, safe text handling, and an accountable local escalation chain are necessary even while the deployment remains healthy.

Recommended controls and UX

Qualify every joined column and test all combined list filters against the actual migrated schema; do not reopen unrelated repaired SQL without evidence. Serialize capacity reservations on the contractor or a dedicated capacity row, preserving current transactional eligibility/referral checks. Add expected versions, permitted phase transitions, durable reasoned audit, owner/backup team, first-response and phase SLA, and overdue escalation. Enforce region/tenant predicates server-side for list, detail, assignment, messaging, and exports. Retain custom-message permission separation. Provide an asynchronous filtered export with immutable scope, safe CSV cell encoding, progress, expiration, and download audit. Show jurisdiction-dependent evidence checklists as pending specialist review, not automatic legal approval. Localize urgency/date labels and ensure detail dialogs are keyboard accessible.

Dependencies and owner

Business Launch Operations owns phase outcomes; contractor operations owns capacity; API/Data owns query and version correctness; Identity owns target scope; Messaging owns delivery; jurisdiction specialists define required approvals and evidence independently of engineering.

Measurable acceptance criteria

Migrated-schema list tests cover all joins and combined country/status/search filters. Concurrent assignments to one remaining capacity slot produce one reservation. Stale phase updates return 409. Regional users cannot access another country by changing filters or IDs. Full exports include every authorized match, neutralize formula-leading text, and run as bounded jobs. Every active launch case has an owner and due time, with escalation and evidence state visible to both shifts.

28. Consulting Sessions - /admin/consulting/sessions

Mission and what exists

Consulting Sessions is a real paginated operations view with search/filter controls, session detail, manual creation, editing, completion, cancellation, no-show, reminders, follow-ups, and page export. Requests are cancellable and confirmation targets are explicit. The schema already distinguishes intake/session parents from scheduled appointment children and follow-up records. Payment creation checks those child appointments. Enterprise hardening must make this existing lifecycle consistent across screens and controllers, rather than invent an unrelated appointment or consulting-delivery system.

Evidence

- [client/src/admin/pages/ConsultingSessionsAdmin.jsx:65](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/ConsultingSessionsAdmin.jsx:65) builds manual intake, :107 posts creation, :224 pages sessions, :249 completes them, and :282 exports displayed rows.
- [client/src/admin/pages/ConsultingSessionDetail.jsx:50](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/ConsultingSessionDetail.jsx:50) updates assignment; :71 completes, :92 marks no-show, and :129 changes notes.
- [server/controllers/adminConsultingController.js:191](/Users/oxsteed/Documents/GitHub/6/server/controllers/adminConsultingController.js:191) lists parent sessions, :397 completes a scheduled child, :442 marks child no-show, and :488 creates a parent manually.
- [server/migrations/102_consulting_sessions.sql:2](/Users/oxsteed/Documents/GitHub/6/server/migrations/102_consulting_sessions.sql:2) defines parents; [server/migrations/103_consulting_all_phases.sql:46](/Users/oxsteed/Documents/GitHub/6/server/migrations/103_consulting_all_phases.sql:46) defines the scheduled child with a unique parent link; migration 106 adds follow-up records.
- [server/routes/consultingAdmin.js:30](/Users/oxsteed/Documents/GitHub/6/server/routes/consultingAdmin.js:30)-:38 exposes these actions with admin authentication but no separate session-operation grants.

Findings

T28-F01 - High [confirmed defect]: manual creation and completion target different required records. Manual creation at [adminConsultingController.js:488](/Users/oxsteed/Documents/GitHub/6/server/controllers/adminConsultingController.js:488) inserts the parent, including optional scheduled time, but not the child. Completion at :403 searches consulting_sessions_scheduled and returns 404 when absent. A manually scheduled row shown in the parent list can consequently lack the object required for its offered completion action. The schema and payment path already provide the distinction; the administrative contract must respect it.

T28-F02 - High [code observation]: completion/no-show do not update the state being listed. The completion UPDATE at [adminConsultingController.js:415](/Users/oxsteed/Documents/GitHub/6/server/controllers/adminConsultingController.js:415) and no-show UPDATE at :460 modify only the child. The list at :191 reads parent status. No synchronization mechanism was identified in the inspected lifecycle controllers/migrations. Thus the source path does not establish that the list reflects completion, and the unguarded child update can overwrite a terminal state. A real-database lifecycle test must verify all coupled states rather than assume an unseen deployment mechanism repairs them.

T28-F03 - High [code observation]: assignment and note editing have weak identity/version contracts. Detail assignment submits selected consultant IDs at [ConsultingSessionDetail.jsx:50](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/ConsultingSessionDetail.jsx:50), while the model contains both legacy consultant-record IDs and user UUID assignments. This requires explicit mapping tests, not an unsupported claim that every assignment fails. Generic details at [adminConsultingController.js:296](/Users/oxsteed/Documents/GitHub/6/server/controllers/adminConsultingController.js:296) replace notes/fields without an expected version. Separate read, schedule, complete, no-show, and edit-note grants are absent from [consultingAdmin.js:30](/Users/oxsteed/Documents/GitHub/6/server/routes/consultingAdmin.js:30).

T28-F04 - Medium [design requirement]: package delivery and follow-up need measurable case ownership. Migration 103's scheduled child has a unique parent link; packages can represent multiple units of consulting service. The operational model needs an explicit relationship between entitlement, consumed appointment, completion evidence, and follow-up owner. This is a design/verification requirement, not a claim that multiple scheduled children already exist for one parent. Page-only export and hard-coded detail labels further limit global handoff.

Expansion scenario

A customer purchases a multi-session package. A coordinator manually schedules the next consultation, a specialist marks the previous appointment complete, and a night-shift agent records a no-show dispute. The parent queue, child appointment, payment eligibility, and remaining service entitlement must agree. Otherwise a completed session can remain actionable or a newly scheduled manual row cannot be completed at all, undermining SLA and customer explanations.

Recommended controls and UX

Define one lifecycle service that atomically creates required appointments, validates transitions, synchronizes parent summary, and records completion/no-show evidence. Keep the existing tables but give each appointment a clear identity and entitlement-consumption policy. Map consultant records to user identities explicitly. Require scoped action permissions, expected versions, append-only notes, audit, and reviewable reversal reasons. Display appointment, parent case, remaining entitlement, follow-up owner/deadline, and delivery state separately. Make reminders/follow-ups retryable through existing delivery infrastructure without declaring them sent before evidence. Clearly label current-page export or supply an audited asynchronous full-result export. Translate detail statuses, show explicit zones, and test modal focus and confirmation wording for keyboard users.

Dependencies and owner

Consulting Delivery owns completion evidence and no-show policy; session/scheduling API owners own atomic lifecycle; Payments owns entitlement/financial linkage; Messaging owns follow-up reliability; Identity owns scoped action grants.

Measurable acceptance criteria

Manual scheduling followed by completion works against the migrated database and updates the list consistently. Cancelled/completed appointments cannot be overwritten by stale no-show calls. Consultant IDs map deterministically to the authorized user. Concurrent note edits do not lose history. Every package unit has an auditable appointment/consumption outcome, each follow-up has an owner and deadline, and multilingual keyboard-driven session workflows pass authenticated browser acceptance.

29. Org Chart - /admin/corporate/org-chart

Mission and what exists

Org Chart is an implemented corporate directory view grouped by department and title seniority, with country selection and expandable department sections. The server already returns active assignments, title information, regional fields, reporting-manager references, and a country/global filter. Corporate hierarchy data therefore exists. The enterprise task is to render and delegate that information safely for 3,500 employees, including people with multiple assignments, rather than describe the org chart or corporate structure as absent.

Evidence

- [client/src/admin/pages/super/CorporateOrgChart.jsx:35](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/super/CorporateOrgChart.jsx:35) groups assignments and expands departments; :108 loads data; :147 accepts country input.
- [server/routes/corporateRoles.js:21](/Users/oxsteed/Documents/GitHub/6/server/routes/corporateRoles.js:21) and :44 require manage_admin_accounts for catalog/org-chart access.
- [server/controllers/corporateRolesController.js:771](/Users/oxsteed/Documents/GitHub/6/server/controllers/corporateRolesController.js:771) builds the org chart; :796 sorts assignments; :819 returns grouped and flat representations.
- [server/migrations/246_corporate_hierarchy.sql:474](/Users/oxsteed/Documents/GitHub/6/server/migrations/246_corporate_hierarchy.sql:474) establishes corporate assignments, including country/region and reporting relationships.
- [server/services/corporateRolesService.js:105](/Users/oxsteed/Documents/GitHub/6/server/services/corporateRolesService.js:105) resolves active title assignments, demonstrating an existing corporate authorization layer separate from basic user roles.

Findings

T29-F01 - High [design requirement]: directory reads are coupled to account-management authority. Org-chart access requires manage_admin_accounts, a broad administrative responsibility, rather than a dedicated directory-read permission. Ordinary employees cannot simply use the admin-only endpoint even though corporate assignments can include employees. A global workforce needs selective directory visibility without encouraging account-management grants for staff who merely need an escalation contact. Public-versus-private fields, tenant boundaries, and regional emergency access must be explicit.

T29-F02 - Medium [code observation]: the response and rendering are full-dataset oriented. The controller at [corporateRolesController.js:771](/Users/oxsteed/Documents/GitHub/6/server/controllers/corporateRolesController.js:771) returns all matching assignments without pagination and includes email/role detail. It returns the same records grouped and flat at :819, increasing payload and exposing unnecessary duplication. The client initially expands all departments at [CorporateOrgChart.jsx:35](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/super/CorporateOrgChart.jsx:35). Multiple titles per employee can increase cardinality well beyond headcount. This is a measured-load verification gap, not a demonstrated memory or latency incident.

T29-F03 - Medium [confirmed defect]: country changes can display stale directory results. Loading at [CorporateOrgChart.jsx:108](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/super/CorporateOrgChart.jsx:108) has no abort or generation guard, while the country text input at :147 can start requests as characters change. A slower previous response can overwrite the latest country selection. The result is particularly misleading for an incident operator trying to locate the correct regional accountable owner. Server filtering does not prevent this client response-order error.

T29-F04 - Medium [code observation]: hierarchy and accessible navigation are underused. The API returns reporting references, but the UI at [CorporateOrgChart.jsx:35](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/super/CorporateOrgChart.jsx:35) presents seniority groups rather than a manager tree or validated escalation chain. Expand buttons at :51 do not expose aria-expanded/control relationships. Title seniority is not equivalent to incident command, acting ownership, or availability; those

must be represented as operational context rather than inferred from position order.

Expansion scenario

During a regional service disruption, support staff need the acting logistics owner and backup for the affected country. Several employees hold global and local titles. A stale country response and a seniority-only list send the escalation to the wrong executive, while giving account-management access to everyone who needs a directory would broaden privileges far beyond that task.

Recommended controls and UX

Add a dedicated scoped directory-read policy with minimal fields, masked contact details where appropriate, and separate privileged relationship editing. Reuse reports_to_user_id to render a validated manager hierarchy plus a flat searchable directory, explicitly marking multiple assignments, acting dates, vacant posts, and backup/on-call contacts from their authoritative systems. Abort superseded requests, debounce country selection, validate ISO codes, and show active scope while loading. Remove duplicate payload representations unless requested; page searchable people and virtualize large departments. Provide an accessible tree or table alternative with labeled expand controls, keyboard navigation, localized title descriptions, RTL layout, and explicit timezone/availability. Directory export should be authorized, audited, scoped, and asynchronous rather than a global employee-data download.

Dependencies and owner

People Operations owns reporting accuracy; Identity owns directory versus management authority; corporate API owners own hierarchy validation and response shape; incident management owns on-call references; frontend/accessibility owners own navigation.

Measurable acceptance criteria

An employee directory reader cannot manage admin accounts or retrieve restricted fields. Regional readers cannot change country parameters to escape their scope. A delayed prior-country response never replaces the current selection. A 3,500-person, multi-assignment fixture yields a bounded initial payload and proposed sub-one-second directory p95. All reporting chains resolve or show explicit vacancies/cycle errors, and keyboard/screen-reader users can find an acting owner and backup without expanding every department.

30. Corporate Titles - /admin/corporate/assignments

Mission and what exists

Corporate Titles manages actual staff title/department assignments with country, region, reporting manager, notes, editing, and ending assignments. The UI loads staff/catalog choices, has assignment filters, uses abortable main loading, and confirms ending an assignment. The API supports employees as well as admins, validates active staff and titles, applies seniority/C-suite restrictions, audits transactional creation, and locks a row during editing. These controls should be preserved. Titles also feed a separate permission resolver; they must not be confused with automatic basic admin-role or delegated-grant membership.

Evidence

- [client/src/admin/pages/super/CorporateAssignments.jsx:50](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/super/CorporateAssignments.jsx:50) loads staff choices; :98 submits scoped assignment fields; :283 loads catalog/assignments; :321 confirms ending.
- [server/controllers/corporateRolesController.js:332](/Users/oxsteed/Documents/GitHub/6/server/controllers/corporateRolesController.js:332) lists assignments with LIMIT 500 at :355; :388 creates and :519 updates them.
- [server/controllers/corporateRolesController.js:438](/Users/oxsteed/Documents/GitHub/6/server/controllers/corporateRolesController.js:438) checks hierarchy restrictions, :460 validates region existence, and :469 validates a reporting user.
- [server/services/corporateRolesService.js:31](/Users/oxsteed/Documents/GitHub/6/server/services/corporateRolesService.js:31) implements a process-local cache, :105 selects active assignments, and :227 invalidates cached entries locally.
- [server/routes/corporateRoles.js:21](/Users/oxsteed/Documents/GitHub/6/server/routes/corporateRoles.js:21) applies account-management permissions; [server/migrations/246_corporate_hierarchy.sql:474](/Users/oxsteed/Documents/GitHub/6/server/migrations/246_corporate_hierarchy.sql:474) establishes assignment relationships.

Findings

T30-F01 - High [confirmed defect]: assignment discovery truncates the enterprise workforce. The API at [corporateRolesController.js:355](/Users/oxsteed/Documents/GitHub/6/server/controllers/corporateRolesController.js:355) silently caps results at 500 with no pagination or continuation metadata. At 3,500 employees, especially with multiple titles, staff assignments outside that slice are undiscoverable through a broad view. Filter controls mitigate some searches but cannot replace a complete administration contract. The staff chooser at [CorporateAssignments.jsx:50](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/super/CorporateAssignments.jsx:50) also loads a broad list rather than a scoped searchable staff lookup.

T30-F02 - High [code observation]: relationship existence does not establish hierarchy validity. Creation verifies a region exists and a reporting user exists at [corporateRolesController.js:460](/Users/oxsteed/Documents/GitHub/6/server/controllers/corporateRolesController.js:460) and :469; it does not establish a country/region match or a cycle-free reporting graph. Editing at :519 locks the current row but accepts changed relationship fields without an expected client version. This can serialize writes yet still overwrite decisions based on stale UI state. Self-reference, multi-node cycles, and cross-scope reporting need explicit graph and policy validation.

T30-F03 - High [code observation]: effective dates are not part of active permission selection. The resolver at [corporateRolesService.js:105](/Users/oxsteed/Documents/GitHub/6/server/services/corporateRolesService.js:105) selects is_active = TRUE without filtering started_at or ended_at, despite returning those dates. An assignment marked active can therefore contribute permissions regardless of its recorded time interval. Product policy might define dates as descriptive only, but that must be stated clearly; treating them as scheduled authority without changing resolution is unsafe.

T30-F04 - High [verification gap]: distributed revocation and regional delegation are not demonstrated. Cache TTL and invalidation at [corporateRolesService.js:31](/Users/oxsteed/Documents/Git

Hub/6/server/services/corporateRolesService.js:31) and :227 are process-local. Multiple application replicas may retain different effective permissions until expiry; acceptable revocation latency needs explicit tests. Seniority checks and manage_admin_accounts do not themselves supply target-country authority. No proof is claimed that this overlay controls every route, so its integration with admin grants must be documented rather than assumed.

Expansion scenario

A reorganization moves 900 employees and creates next-week acting assignments across regional teams. People Operations needs to discover every affected record, prevent two departments reporting to each other, schedule authority precisely, and revoke an outgoing manager immediately. A 500-row slice, descriptive dates interpreted as enforcement, and replica-local cache invalidation produce different organizational answers depending on the reader or application instance.

Recommended controls and UX

Add complete bounded cursor paging and scoped typeahead for staff, manager, and region selection. Validate manager graph, active manager eligibility, country/region consistency, and allowed cross-region exceptions transactionally. Require expected versions with relationship diffs; keep existing seniority/C-suite protections and durable audit. Define effective-from/until semantics and enforce them in permission resolution, or visibly label dates non-authoritative. Publish cache invalidation across replicas with a stated maximum revocation delay, and test overlay precedence against delegated grants and basic roles. Introduce dedicated scoped People Operations privileges instead of universal account-management authority. Provide reviewed asynchronous reorganization previews, impact reports, and per-assignment conflicts. Localize dates instead of fixed en-US formatting and label modal close/relationship controls for assistive technology.

Dependencies and owner

People Operations owns hierarchy and acting-role policy; Identity owns effective authority and distributed revocation; corporate API owners own graph/version validation; SRE owns replica consistency; Security approves privileged assignment review.

Measurable acceptance criteria

All 3,500 employees and multi-title assignments are discoverable through paging/search. Self and multi-node cycles fail validation; unauthorized country/region mismatches are rejected. Future/expired assignments cannot grant effective authority under a time-bound policy. Cross-replica revocation meets a proposed five-second maximum. Concurrent edits return 409, a 900-person reorganization has reviewed item outcomes, and direct API tests verify seniority, scope, and permission-overlay precedence.

31. Logistics Ops - /admin/logistics

Mission and what exists

Logistics Ops is an existing dashboard into a substantial logistics system: lanes, suppliers, sourcing/quotes, pooled purchases, purchase orders, shipments, warehouses, allocations, claims, compliance/customs, pricing, profitability, and privileged control. The dashboard exposes multiple operational links, hides super-only control, and uses abortable statistics loading. Backend state-machine helpers and lane/country/product access policies already exist, including fail-closed behavior. Internal operations/compliance workspaces also recheck current approval for non-admin staff. Expansion should connect those safeguards consistently, not duplicate logistics infrastructure or confuse engineering evidence with legal customs clearance.

Evidence

- [client/src/admin/pages/logistics/LogisticsAdminDashboard.jsx:17](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/logistics/LogisticsAdminDashboard.jsx:17) defines operational navigation; :68 loads stats and :101 renders summaries.
- [server/routes/logisticsAdmin.js:39](/Users/oxsteed/Documents/GitHub/6/server/routes/logisticsAdmin.js:39) exposes stats, :88 shipments, :122 super-only pricing changes, and :131 compliance workspaces.
- [server/middleware/requireInternalLogisticsWorkspace.js:54](/Users/oxsteed/Documents/GitHub/6/server/middleware/requireInternalLogisticsWorkspace.js:54) bypasses workspace checks for admin roles; :58-:70 validates non-admin role/session/live approval.
- [server/controllers/logisticsAdminController.js:33](/Users/oxsteed/Documents/GitHub/6/server/controllers/logisticsAdminController.js:33) performs six global aggregate queries; [server/controllers/logisticsShipmentsController.js:85](/Users/oxsteed/Documents/GitHub/6/server/controllers/logisticsShipmentsController.js:85) updates tracking directly.
- [client/scripts/adminLogisticsLocalization.test.mjs:45](/Users/oxsteed/Documents/GitHub/6/client/scripts/adminLogisticsLocalization.test.mjs:45) and [:64](/Users/oxsteed/Documents/GitHub/6/client/scripts/adminLogisticsLocalization.test.mjs:64) verify localized contracts/catalog keys statically across en/es/fr/ht/ar; they do not prove live keyboard use or target authorization.
- [server/services/logisticsStateMachine.js:9](/Users/oxsteed/Documents/GitHub/6/server/services/logisticsStateMachine.js:9) defines transitions; [server/services/logisticsAccessService.js:39](/Users/oxsteed/Documents/GitHub/6/server/services/logisticsAccessService.js:39) enforces existing lane/access eligibility and :163 fails closed on errors.

Findings

T31-F01 - High [confirmed defect]: shipment tracking mutation lacks target authorization. [logisticsShipmentsController.js:85](/Users/oxsteed/Documents/GitHub/6/server/controllers/logisticsShipmentsController.js:85) updates carrier, booking reference, tracking, and ETA with only `WHERE id=$5`. The operations-workspace route admits approved internal operators but does not pass a shipment-owner/region predicate into that direct SQL. Unlike access-aware detail/service paths, this mutation does not demonstrate that the caller may administer the target shipment. Global admin reach may be intentional; unscoped internal-operator updates require correction and direct-ID denial tests.

T31-F02 - High [code observation]: workspace bypass broadens ordinary admin logistics access. [requireInternalLogisticsWorkspace.js:54](/Users/oxsteed/Documents/GitHub/6/server/middleware/requireInternalLogisticsWorkspace.js:54) accepts any admin role without route-specific internal approval. Several administrative routes at [logisticsAdmin.js:43](/Users/oxsteed/Documents/GitHub/6/server/routes/logisticsAdmin.js:43) also rely on role rather than separate capability grants. This is authenticated behavior, not unauthenticated exposure, and super-only pricing is correctly stricter. Enterprise delegation needs supplier, shipment, claims, warehouse, compliance, and profitability responsibilities distinct from merely possessing an admin role.

T31-F03 - Medium [confirmed defect]: dashboard receipt wording does not match counted state. The UI labels warehouse receipts as active, while [logisticsAdminController.js:43](/Users/oxsteed/Documents/GitHub/6/server/controllers/logisticsAdminController.js:43) counts all joined inventory receipts without an active/open predicate. Historical receipts can inflate a current-workload impression. The six aggregates at :37 are global and have no time/scope budget; their scale is a verification gap. Counts should disclose whether they represent backlog, lifetime volume, or snapshot state.

T31-F04 - Medium [design requirement]: exceptions need accountable ownership and safe concurrent/provider updates. Tracking updates at [logisticsShipmentsController.js:90](/Users/oxsteed/Documents/GitHub/6/server/controllers/logisticsShipmentsController.js:90) lack expected versions and explicit mutation audit. The dashboard emphasizes totals rather than oldest exception, missing documents, carrier freshness, or handoff owner. Existing states and sourcing assignment should be extended into an operational control tower with carrier/provider capability evidence, not assume integrations or jurisdiction approvals from the presence of a tracking number or compliance screen.

Expansion scenario

A thousand shipments encounter customs holds across several lanes. A regional coordinator corrects ETAs while a carrier import updates the same records; another team reallocates warehouse receipts. Operators must see which holds exceed SLA, who can modify each shipment, and whether carrier evidence is stale. Global lifetime totals and last-writer tracking updates cannot support a reliable night-shift handoff.

Recommended controls and UX

Apply target-aware lane/region/organization policies to every direct shipment mutation and separate action grants from general admin role. Preserve existing access-service fail-closed decisions and super-only pricing. Add versioned tracking changes, normalized carrier references, source/event timestamps, durable audit, and idempotent ingestion with reconciliation for late/out-of-order events. Define provider-supported integrations and show manual versus verified tracking honestly. Replace ambiguous metrics with scoped aged backlog and snapshot timestamps; benchmark aggregates and use precomputed summaries where justified. Reuse existing state machines for owned exception queues, due dates, document completeness, escalation, and backup assignment. Add bounded asynchronous carrier imports, bulk recalls, and allocation jobs with preview and per-item results. Compliance/customs milestones remain evidence workflows subject to specialist jurisdiction review, not automated legal determinations.

Dependencies and owner

Logistics Operations owns lane SLAs and handoffs; warehouse/sourcing teams own inventory evidence; Identity owns workspace/action scope; API owners own tracking integrity; integration/SRE teams own carrier reconciliation and query budgets.

Measurable acceptance criteria

Approved operators cannot mutate an unauthorized shipment by ID. Stale tracking writes conflict and duplicate carrier events do not regress state. Dashboard active/backlog counts match fixture definitions and display scope/freshness. A thousand held shipments have owners, deadlines, and visible escalation. Large imports resume after restart with item results, and read-only carrier/provider tests verify supported capabilities before claiming automated tracking or clearance.

32. Learn Submissions - /admin/learn/submissions

Mission and what exists

Learn Submissions is an existing moderation screen with status-filtered pagination, expanded review cards, approve/reject actions, reviewer notes, video queues, and flagged-video workflows. The client has cancellable list loading and disabled actions while review is pending. The server validates decisions, publishes approved submissions transactionally with a savepoint, and runs AI tagging afterward. Video publication, provider allowlists, creator flags, and focused tests are real capabilities. Enterprise readiness depends on reviewers seeing authoritative content and resolving publication failures rather than adding a second learning-content system.

Evidence

- [client/src/admin/pages/LearnSubmissionsAdmin.jsx:76](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/LearnSubmissionsAdmin.jsx:76) expects full review content; :169 renders tools; :290 loads paged rows; :724 passes those rows to expanded review.
- [server/controllers/learnController.js:389](/Users/oxsteed/Documents/GitHub/6/server/controllers/learnController.js:389) lists summary fields; :436 already provides submission detail; :460 reviews and :493 attempts publication.
- [server/controllers/learnController.js:1293](/Users/oxsteed/Documents/GitHub/6/server/controllers/learnController.js:1293) lists video work and :1394 processes eligible publication batches.
- [server/routes/learn.js:91](/Users/oxsteed/Documents/GitHub/6/server/routes/learn.js:91) defines video moderation controls/limits and :139 provides flag workflows.
- [server/__tests__/learnGuideVideos.test.js:203](/Users/oxsteed/Documents/GitHub/6/server/__tests__/learnGuideVideos.test.js:203) tests publication, :277 conflicts, and :412 flags; [server/__tests__/learn.test.js:73](/Users/oxsteed/Documents/GitHub/6/server/__tests__/learn.test.js:73) tests submission validation.

Findings

T32-F01 - High [confirmed defect]: reviewers receive summary rows rather than the full submission. The list projection at [learnController.js:405](/Users/oxsteed/Documents/GitHub/6/server/controllers/learnController.js:405) omits intro, steps, skills, tools, video, and license evidence. Expansion at [LearnSubmissionsAdmin.jsx:724](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/LearnSubmissionsAdmin.jsx:724) passes that same list row into a component expecting those fields, without fetching the existing detail endpoint. Reviewers can approve/reject while the screen lacks the actual instructional payload. This is a current UI/API mismatch, not a claim the detail capability is missing.

T32-F02 - High [confirmed defect]: review decisions do not enforce pending/version state. The transaction at [learnController.js:472](/Users/oxsteed/Documents/GitHub/6/server/controllers/learnController.js:472) reads without locking and updates at :484 by ID alone. Two reviewers, or a stale direct API request, can overwrite approved/rejected state and reviewer attribution despite UI actions being shown only for pending rows. The transaction groups writes but does not establish an exclusive moderation decision. Appropriate re-review must be a separately authorized transition with preserved history.

T32-F03 - High [confirmed defect]: approval can succeed without publication and without actionable recovery UX. A publication failure rolls back only to the savepoint at [learnController.js:502](/Users/oxsteed/Documents/GitHub/6/server/controllers/learnController.js:502); approval commits and responds with published_guide_id: null at :508. The client review handler at [LearnSubmissionsAdmin.jsx:305](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/LearnSubmissionsAdmin.jsx:305) reloads without turning that null outcome into an assigned publication-recovery case. Savepoint resilience is intentional and useful, but approved/unpublished must not be indistinguishable from published success.

T32-F04 - Medium [confirmed defect]: linked-tool rendering shadows the translation function. In [LearnSubmissionsAdmin.jsx:169](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/LearnSubmissionsAdmin.jsx:169), the tool variable is named t, then :175 calls that object as a translation function when buy_url exists; rental links have the same pattern. Once full detail is properly loaded, linked tools can trigger a TypeError. This currently latent branch needs a direct component fixture, not simply a build-success claim.

Expansion scenario

A multilingual campaign produces 5,000 how-to submissions, including instructions requiring specialist review. Two moderators open the same item, one approves while another rejects, and catalog publication temporarily fails. A summary-only card and lost decision ownership allow weak review; approved-but-unpublished content becomes an invisible backlog. Timed video publication also needs capacity planning so specialist holds are applied before automated eligibility.

Recommended controls and UX

Fetch authoritative detail before enabling a decision and show attachment/license/content version plus a reviewed-content acknowledgement. Lock or conditionally update pending/version state, append decisions to immutable history, and require a reason for rejection/reversal. Keep the savepoint but persist publication state, error category, retry identity, owner, and SLA; expose a retry/recovery queue. Rename the tool iterator and test links with real detail fixtures. Add scoped moderation roles and specialist routing by language/category/risk, assigned workload, oldest pending age, and explicit auto-publication holds using existing video controls. Bound bulk decisions as reviewed asynchronous jobs rather than unchecked mass approval. Ensure video/content alternatives, captions, translated statuses, keyboard review controls, and RTL presentation. Professional/legal review requirements need specialist definitions, not unsupported blanket claims.

Dependencies and owner

Learn Trust/Content owns review standards and specialist escalation; Learn API owns decision/publication integrity; frontend owners own complete review payloads; video workers own timed backlog; localization/accessibility owners validate global review.

Measurable acceptance criteria

A reviewer sees complete steps/tools/video evidence before approval. Concurrent decisions yield one committed result and a 409 for the stale reviewer. Injected publish failure produces visible approved/unpublished state and idempotent retry. Linked-tool cards render without TypeError. A 5,000-item workload shows owners, oldest age, and held-video deadlines, while authenticated multi-locale keyboard tests verify the complete review flow rather than relying on the skipped CI matrix.

33. Learn Product Links - /admin/learn/product-links

Mission and what exists

Learn Product Links is an implemented curation workspace for tool/material links, including affiliate retail, owned inventory, partners, and internal rentals. It supports global or guide-specific overrides, search, editing, activity toggles, and confirmed deactivation. The server exposes public active links and admin CRUD, while AI tagging reuses curated matches rather than inventing arbitrary purchase URLs. Guide-specific priority and case-insensitive tool lookup already exist. Enterprise expansion should improve this curated registry and publication governance, not create a competing link-management system.

Evidence

- [client/src/admin/pages/LearnProductLinksAdmin.jsx:35](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/LearnProductLinksAdmin.jsx:35) defines link types; :47 validates URLs; :327 loads links and :365 filters them locally.
- [server/routes/learn.js:162](/Users/oxsteed/Documents/GitHub/6/server/routes/learn.js:162)-:165 protects CRUD with moderate_content.
- [server/controllers/learnController.js:600](/Users/oxsteed/Documents/GitHub/6/server/controllers/learnController.js:600) returns public active links; :619 lists admin links; :651 creates, :696 partially updates, and :746 deletes/deactivates.
- [server/migrations/290_learn_product_links.sql:12](/Users/oxsteed/Documents/GitHub/6/server/migrations/290_learn_product_links.sql:12) defines override scope, attribution fields, and active state; :30 adds lookup indexes.
- [server/services/learnTagging.js:102](/Users/oxsteed/Documents/GitHub/6/server/services/learnTagging.js:102) resolves curated links and :119 gives guide-specific overrides priority; [server/__tests__/learn.test.js:64](/Users/oxsteed/Documents/GitHub/6/server/__tests__/learn.test.js:64) tests fallback availability.

Findings

T33-F01 - High [confirmed defect]: retail-domain validation accepts lookalike hostnames. Client validation at [LearnProductLinksAdmin.jsx:47](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/LearnProductLinksAdmin.jsx:47) and server validation at [learnController.js:662](/Users/oxsteed/Documents/GitHub/6/server/controllers/learnController.js:662) use a prefix regex with no hostname boundary. A string such as https://amazon.com.evil.invalid/item matches the allowed prefix despite its actual host being unrelated. This is a demonstrable string-validation defect, not an assertion about any live stored URL. Other link types also lack a unified server URL-protocol/host policy in these handlers. Source comments describing compliance are not proof of provider approval.

T33-F02 - High [confirmed defect]: partial updates do not validate the effective record. [learnController.js:713](/Users/oxsteed/Documents/GitHub/6/server/controllers/learnController.js:713) checks a changed buy URL only when one is supplied. Changing an existing row to the affiliate type without sending buy_url skips validation of its retained destination. Neither a client-only tag-presence check nor the generic allowed-field list establishes that the final combined type, destination, affiliate identifier, and guide scope are valid. This requires merged-record validation under a versioned transaction.

T33-F03 - Medium [code observation]: discovery and public refresh are unbounded. Public active links at [learnController.js:600](/Users/oxsteed/Documents/GitHub/6/server/controllers/learnController.js:600) and administrative links at :619 load all matching rows without paging. The UI at [LearnProductLinksAdmin.jsx:365](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/LearnProductLinksAdmin.jsx:365) searches locally after loading them. With tens of thousands of overrides, repeated public refreshes and staff discovery need indexed, scoped lookup and cache/version budgeting. Existing indexes deserve credit, but they do not reduce a full-table response by themselves; no production latency claim is made.

T33-F04 - High [design requirement]: global overrides need release and regional governance. The current schema supports guide-specific/global scope but no explicit country/locale program selection or

immutable edit history. Partial updates have no expected version, and permanent deletion at [learnController.js:746](/Users/oxsteed/Documents/GitHub/6/server/controllers/learnController.js:746) shares the same content-moderation grant as ordinary deactivation. Duplicate normalized tool/scope records are not prevented by migration 290's indexes, making tie resolution important. Add reviewed commercial-link responsibilities rather than treating all content moderators as global affiliate administrators.

Expansion scenario

A seasonal rollout updates 80,000 learning links for local inventory, supported retail programs, and rental alternatives. Two regional editors change the same global override, and a mistyped destination passes prefix validation. The platform needs to roll back the exact publication version without losing historical click attribution or forcing every user refresh to download the entire registry. Region-specific eligibility cannot be inferred from one global affiliate tag.

Recommended controls and UX

Parse URLs with a standard parser, require HTTPS for external destinations, compare exact normalized hostnames against an approved allowlist, and validate internal paths separately. Reject credentials, unsupported schemes, and lookalike subdomains; normalize then validate the merged record on every update. Add scoped commercial-link grants, expected versions, durable before/after history, reviewed publication versions, and rollback. Define normalized uniqueness and deterministic guide/global/locale priority using the existing override model. Prefer deactivation with retained attribution; require stronger authority and dependency preview for physical deletion. Replace full-list loading with bounded server search and guide-specific public lookup/cache versions; use asynchronous bulk preview/jobs with item errors. Show link origin, region/locale, verification age, and provider-program status accurately. Program terms and jurisdiction-dependent disclosures require specialist/provider review, not assurances from regex checks.

Dependencies and owner

Learn Commerce owns curation and release review; Security owns URL validation; API/Data owns deterministic lookup and cache versions; Identity owns regional commercial grants; partnership specialists own provider eligibility and program approvals.

Measurable acceptance criteria

Lookalike hosts, unsafe schemes, and invalid merged updates fail server tests. Duplicate normalized scope records are rejected or resolve by documented deterministic precedence. Concurrent edits return 409. A published version can be rolled back while retaining click attribution. An 80,000-link update is a resumable bounded job; public guide lookup returns only relevant links with a proposed sub-300-ms p95 cache path. Regional editors cannot alter global or foreign-region overrides without explicit authority.

34. Learn AI Suggestions - /admin/learn/ai-suggestions

Mission and what exists

Learn AI Suggestions is an existing human-review queue for AI-proposed tools, materials, and services, with guide/status filters, expandable suggestions, analysis triggering, approval, and rejection. The client uses abortable loading. The server deduplicates pending suggestions, locks the suggestion and guide during approval, prevents repeat decisions, validates service categories, and uses curated product/rental matches. Tagging has a timeout and fail-soft reasons for missing configuration, throttling, network, and invalid JSON. These are real safeguards; readiness requires scalable job execution, truthful capability reporting, and provenance, not unreviewed automatic publication.

Evidence

- [client/src/admin/pages/LearnAiSuggestionsAdmin.jsx:93](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/LearnAiSuggestionsAdmin.jsx:93) displays confidence, :232 loads guide choices, :243 loads suggestions, and :263 runs analysis.
- [server/controllers/learnController.js:939](/Users/oxsteed/Documents/GitHub/6/server/controllers/learnController.js:939) awaits tagging; :956 lists a fixed 200 suggestions; :986 locks and approves; :1066 conditionally rejects pending rows.
- [server/services/learnTagging.js:18](/Users/oxsteed/Documents/GitHub/6/server/services/learnTagging.js:18) defines timeout/result limits, :36 builds prompts, :82 matches rentals, :140 deduplicates, and :160 returns typed failure reasons.
- [server/migrations/628_learn_ai_tagging.sql:10](/Users/oxsteed/Documents/GitHub/6/server/migrations/628_learn_ai_tagging.sql:10) defines suggestion provenance/status/reviewer fields; :36 establishes pending deduplication.
- [server/__tests__/learnTagging.test.js:63](/Users/oxsteed/Documents/GitHub/6/server/__tests__/learnTagging.test.js:63), :89, and :97 test unavailable/throttled/timeout behavior; [server/__tests__/learnMarketplace.test.js:123](/Users/oxsteed/Documents/GitHub/6/server/__tests__/learnMarketplace.test.js:123) tests approval and :183 decision conflict.

Findings

T34-F01 - High [confirmed defect]: review history and backlog are silently truncated. [learnController.js:970](/Users/oxsteed/Documents/GitHub/6/server/controllers/learnController.js:970) fixes the list at 200 with no cursor or total. The UI at [LearnAiSuggestionsAdmin.jsx:247](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/LearnAiSuggestionsAdmin.jsx:247) falls back to the returned row count as total and has no continuation mechanism. After a tagging campaign, older pending suggestions are invisible even if they are the oldest SLA breach. Existing guide/status filters help narrow subsets but do not establish complete queue access.

T34-F02 - Medium [confirmed defect]: analysis failures are presented as duplicate counts. Tagging returns {skipped: 'no-key'}, 'timeout', 'rate-limited', and other reasons at [learnTagging.js:160](/Users/oxsteed/Documents/GitHub/6/server/services/learnTagging.js:160)-:217. The successful HTTP handler passes that result through, while the UI at [LearnAiSuggestionsAdmin.jsx:270](/Users/oxsteed/Documents/GitHub/6/client/src/admin/pages/LearnAiSuggestionsAdmin.jsx:270) formats skipped as "already existed." Missing provider capability or a timeout can therefore look like a completed analysis with duplicate results. Fail-soft backend behavior is useful, but it needs a typed capability/error UX and accountable recovery.

T34-F03 - High [code observation]: review lacks source/model version evidence. Migration 628 stores raw JSON, timestamps, and reviewer identity, but not a guide-content hash/version, model identifier, prompt revision, or catalog snapshot. Prompt building at [learnTagging.js:36](/Users/oxsteed/Documents/GitHub/6/server/services/learnTagging.js:36) uses title/category/intro rather than full instructional steps. Approval locks current guide content, but does not show whether that content changed after analysis. Rental confidence at :82 is a name-match heuristic, not demonstrated calibrated probability across languages; the UI must explain that distinction.

T34-F04 - Medium [design requirement]: synchronous tagging cannot serve large controlled campaigns. The controller at [learnController.js:939](/Users/oxsteed/Documents/GitHub/6/server/controllers/learnController.js:939) awaits a provider call with a 25-second timeout; [learnTagging.js:224](/Users/oxsteed/Documents/GitHub/6/server/services/learnTagging.js:224) then performs per-item matching/insertion. A six-million-user content platform needs bounded concurrency, quota/cost budgets, resumable jobs, and provider-failure ownership. That is a scale requirement, not proof of current provider outage. Numeric confidence also needs an explicit API type contract: schema NUMERIC at migration 628 line 21 does not by itself guarantee the JavaScript number expected by the client.

Expansion scenario

After a Direct catalog revision, editors retag 12,000 guides across five languages. Specialists concurrently review suggestions while guide authors correct content. Provider throttling must pause jobs visibly, not report duplicates; old suggestions must remain discoverable, and approval must reveal stale source/catalog evidence. English-oriented name matching cannot silently become a universal confidence claim.

Recommended controls and UX

Keep human approval, locking, service validation, and deduplication. Add complete bounded queue paging, oldest-age counts, assigned reviewer/team, and scoped moderation authority. Return typed outcomes distinguishing created, deduplicated, unavailable, throttled, and failed. Move campaigns to durable idempotent jobs with content/catalog hashes, bounded concurrency, backoff, retry ownership, cancellation, quota/cost ceilings, and operational metrics. Store model/prompt revision and provenance with each suggestion; require reanalysis or explicit informed override when source changes. Show confidence as a explained heuristic with normalized numeric serialization, and evaluate precision/recall by language/category before expanding thresholds. Reuse curated destinations and retain the no-arbitrary-AI-URL boundary. Provider/data-processing approval and jurisdiction review remain separate from engineering implementation; no credentials or live provider calls are needed for these controls.

Dependencies and owner

Learn AI/Product owns suggestion quality; Content Operations owns review SLAs; API/worker owners own jobs and provenance; SRE owns provider budgets; partnerships/privacy specialists own external-provider approvals; localization owns multilingual evaluation.

Measurable acceptance criteria

Every pending item beyond 200 is reachable and counted. Missing configuration, timeout, and throttling render distinct non-success outcomes. Stale guide/catalog versions cannot be silently approved. A 12,000-guide job resumes without duplicate pending suggestions and obeys configured provider/concurrency budgets. Concurrent approval produces one result and a conflict, confidence serializes consistently, and language/category evaluations meet an agreed quality threshold before wider automatic matching is enabled.

Super-admin tab reviews

35. Super Dashboard - /admin/super/dashboard

Mission / what exists. This is an executive and incident overview, combining user growth, jobs, revenue, verification, payouts, activity, geography, health, and durable security incidents. The current page offers period selection, refresh, revenue charts, incident acknowledgement and resolution. It is implemented, with separate auxiliary requests that allow partial rendering. Its role restriction is enforced by the API, not only navigation. It should become the entry point for accountable global operations while allowing regional managers to see only authorized business information through an explicitly scoped mode.

Evidence. client/src/admin/pages/super/Dashboard.jsx:141 distinguishes super administrators; :150 fetches primary statistics; :160 independently loads activity, geography, monitoring health, and security events; :202 and :216 invoke incident actions. server/routes/admin.js:161 protects dashboard/chart/activity/geo endpoints with requireSuperAdmin. server/controllers/superAdminController.js:172 constructs statistics, :190 performs multiple user-count subqueries, :235 aggregates jobs, :268 aggregates the platform ledger, and :399 returns statistics. client/scripts/adminRouteContracts.test.mjs:9 checks dashboard support links, but is a source contract test rather than a rendered executive workflow test.

Findings. T35-F01 - High [confirmed defect]: completion-rate cohorts disagree. At server/controllers/superAdminController.js:235, cancelled jobs are counted over all history, while completed jobs for the period are counted separately. At :356, the denominator combines period completions with lifetime cancellations. A seven-day performance card can deteriorate solely because the platform has accumulated old cancellations. This is a direct arithmetic-definition defect, independent of traffic or deployment health. Use the same event-time cohort for numerator and denominator and expose the definition in the card.

T35-F02 - High [code observation]: query failure can look like a valid zero. The core aggregation at :199 catches failure into an empty object; :362 then converts absent fields to zero. Optional panels also use empty activity/geography fallbacks at client/src/admin/pages/super/Dashboard.jsx:182. Consequently, a dependency-specific failure can resemble no business activity even while the endpoint returns 200. Preserve partial availability, but return per-metric validity, error code, source watermark, and last successful observation; render unavailable values distinctly from zero.

T35-F03 - Medium [code observation]: refreshed dashboards repeatedly request history-wide aggregation. Counts at server/controllers/superAdminController.js:192, job aggregates at :235, and ledger totals at :268 are not supplied by an inspected snapshot cache. This is a scale exposure, not proof that current queries are slow. Six million users and years of jobs make refresh fan-out worth benchmarking. Incremental daily aggregates and bounded freshness would let executive refreshes avoid competing with checkout and support writes.

T35-F04 - Medium [design requirement]: shared incident visibility needs regional accountability. The super-only route protects global data, but global operators cannot safely delegate this whole view to shift leads. Extend existing incident and grant mechanisms with scoped summaries, an incident owner/team, acknowledgement deadline, escalation clock, and handover history. Do not create a second incident system. Distinguish security-event samples from the authoritative incident queue and link every actionable card to its filtered operational workspace.

Expansion scenario. During an Asia-Pacific launch, a local holiday increases cancellations while a European shift is investigating an unrelated queue failure. An executive sees both in a global card; a regional lead needs the affected market, valid comparison period, source freshness, owner, and deadline. Wrong cohorts or silent zero fallbacks would misdirect staffing even though /ready remains green.

Recommended controls and UX. Add explicit UTC/reporting-zone selection, consistent cohort definitions, cached aggregates with drill-through, partial-data banners, and accessible textual chart equivalents. Retain translated labels and verify Arabic direction, keyboard incident controls, focus restoration, and reduced motion in a configured authenticated matrix.

Dependencies / owner. Analytics owns definitions; SRE owns freshness and incident handover; IAM owns scoped access; admin frontend owns rendering. Coordinate with Money and Regions rather than separately inventing scopes.

Measurable acceptance criteria. A fixture containing 100 historic cancellations and ten current completions must not alter the current-period rate unless current cancellations change. Inject one failed source and verify an unavailable marker with no fabricated zero. At six million seeded users and declared transaction history, 100 concurrent operator refreshes should meet p95 two seconds, with less than five-minute aggregate lag and recorded query plans. Every critical incident must have an owner and an escalation deadline; the authenticated matrix must execute with zero fixture-related skips.

36. Money - /admin/super/money

Mission / what exists. Money is an existing cross-business commercial map, grouping direct revenue, marketplace volume, partner flows, and configured programs. It includes period switches, recent movements, fee settings, and links to deeper operating pages. This distinction is valuable: marketplace GMV and an affiliate payout are not automatically platform revenue. The endpoint already attempts to account for subscriptions, service fees, consulting, marketplace orders, API products, affiliates, franchises, and launch applications. The scale objective is to make the map an explainable, scoped management instrument with a consistent reporting basis.

Evidence. client/src/admin/pages/super/MoneyPage.jsx:22 defines channel categories and :49 defines tracking-status badges; :68 formats amounts as USD; :270 renders period controls. server/routes/admin.js:247 requires view_financials; server/middleware/adminAuth.js:126 and :171 restrict delegated permission checks to global grants. server/controllers/superAdminController.js:1633 builds the overview, :1659 checks optional tables, :1676 defines recognized ledger sources, :1690 aggregates movements, :2021 combines ledger revenue and consulting receipts, and :2039 constructs channel definitions. client/scripts/adminRouteContracts.test.mjs:62 checks server authorization wiring for Money.

Findings. T36-F01 - High [code observation]: the report lacks a currency grouping at its aggregation boundary. The ledger query groups by source at server/controllers/superAdminController.js:1699, while client/src/admin/pages/super/MoneyPage.jsx:68 renders a USD amount. Consulting and marketplace additions are likewise represented as cent totals in the overview. The inspected endpoint does not establish an exchange-rate snapshot or explain conversion. This does not prove existing rows contain mixed currencies, but it makes worldwide reporting unsafe until the currency invariant is demonstrated. Return native-currency totals and separately identified reporting-currency conversions with rate source, timestamp, and rounding policy.

T36-F02 - High [design requirement]: operational classification is not yet a finance-approved reporting contract. The "tracked revenue" calculation at server/controllers/superAdminController.js:2021 adds selected positive ledger sources to completed consulting payments, while other channels are explicitly volume or pipeline signals. Retain those distinctions. Add a versioned source mapping, provenance, elimination rules, and a cross-tab bridge explaining why Money, Revenue, and Financials differ. A possible duplicate consulting posting is a verification question, not a confirmed duplicate. Finance must approve the bridge before employees use these numbers for a close or management forecast.

T36-F03 - Medium [code observation]: live global aggregation has substantial request fan-out. Schema probes at :1659 precede lifetime sums at :1692, then service, consulting, commerce, affiliate, and franchise queries. The code handles optional tables, but no inspected consistent snapshot encompasses all those reads. Under sustained business writes, channel totals can reflect different instants. Precompute a coherent reporting snapshot, expose its watermark, and benchmark query amplification; do not claim a present performance outage without timings.

T36-F04 - High [code observation]: regional financial delegation remains all-or-nothing. requirePermission deliberately rejects regional-only grants on this global API. That is a correct defensive boundary. However, a regional business lead has no scoped Money request in the inspected page. Introduce entity/market filters backed by server-resolved business ownership and a reporting permission limited to that scope. Merely adding a country dropdown would not create authorization.

Expansion scenario. A regional general manager compares consulting receipts, marketplace sales, and franchise commitments across Canada and Japan during a monthly business review. The existing badges help separate concepts, but a single USD presentation and different observation times could overstate a market or obscure unapplied pipeline value. The manager must drill from each figure to its records without gaining access to another region's customers or affiliate earnings.

Recommended controls and UX. Offer native versus reporting currency, market/legal-entity scope, accounting-basis badges, source freshness, and a reconciled bridge to the ledger. Preserve the existing channel map. Add accessible table equivalents for summary graphics, locale-aware numbers, saved filters, and an explicit "configured, no recognized receipts" state rather than implying provider readiness.

Dependencies / owner. Finance analytics owns mappings and FX policy; IAM owns server scope; business-domain owners validate source semantics; data engineering owns snapshots; admin frontend owns drill-through and accessibility.

Measurable acceptance criteria. USD/CAD/JPY fixtures must remain separated until a declared FX snapshot is applied, with no implicit two-decimal assumption for native JPY. Each channel total must reproduce from an identified source set and watermark. A regional admin must receive 403 for unauthorized entities on list and drill-through APIs. Benchmark 100 concurrent reads against declared multi-year history with p95 two seconds and a published freshness budget. A finance-approved bridge must explain every difference between Money and the other financial tabs.

37. Restaurant Credit - /admin/super/restaurant-credit

Mission / what exists. Restaurant Credit is an existing global oversight console for isolated restaurant wallets. It has search, status/restaurant filters, server pagination, wallet details, balances/reservations, load requests, hold release, suspension/reactivation, operator history, and restaurant conversion-policy controls. The important boundary is restaurant wallet isolation: this tab should oversee that existing ledger rather than introduce a competing wallet. Its global super-admin restriction is explicit on both read and write routes, and financial mutations carry a limiter.

Evidence. client/src/admin/pages/super/RestaurantCreditAdmin.jsx:413 manages page/filter/detail state, :429 loads paginated overview data, :449 fetches a wallet, and :483 dispatches actions. :243, :256, :269, :281, and :289 slice detail histories to small samples. server/routes/admin.js:166 applies no-store responses; :168-:175 require super-admin access and limit mutations. server/controllers/restaurantCreditAdminController.js:180 parses pagination, :189 runs overview queries, :307 returns pagination, and :504 locks status changes inside a transaction with operator audit at :527. server/__tests__/restaurantCreditAdmin.test.js:89 checks the overview contract and :115 validates status-filter rejection.

Findings. T37-F01 - Medium [code observation]: the UI protects against mixed-currency display, but the overview lacks an actionable currency breakdown. At server/controllers/restaurantCreditAdminController.js:197, balance, reserved, and available amounts are summed without currency grouping; the response at :322 supplies totals plus a currency count. Importantly, client/src/admin/pages/super/RestaurantCreditAdmin.jsx:521-:522 masks mixed totals with a localized "Multiple" label, and :541-:544 uses that guard. This is not a misleading mixed-currency amount displayed by the current UI, nor evidence of cross-wallet spending. Nevertheless, finance cannot inspect each currency's aggregate liability from these overview cards. Return grouped native balances and render a currency breakdown; deprecate unqualified scalar totals for multi-currency API consumers unless a separate, documented conversion is applied.

T37-F02 - Medium [code observation]: wallet history is sampled twice without historical navigation. The server limits detailed histories at :389 and :396; the client further slices load requests, holds, transfers, ledger entries, and audit events at the lines above. This is not a missing detail page. It is an incomplete investigation workflow for an old receipt or hold. Add independent cursor pagination, date filters, provider/order reference search, and an export tied to the account and restaurant scope. Clearly label the first screen as recent activity and disclose completeness.

T37-F03 - High [code observation]: locking serializes status updates but does not identify stale operator intent. At :511 the account is locked, closed wallets cannot be reactivated at :517, and unchanged targets become no-ops at :518. Those controls are valuable. The write accepts a desired status without an expected status or version, however. A stale operator can reactivate a wallet that another shift just suspended for an investigation. Require an expected state/version and a reason for reactivation, present the intervening change, and route exceptional overrides through the existing audit and approval mechanisms.

T37-F04 - High [design requirement]: global super-admin operation cannot be the workforce model. Routes at server/routes/admin.js:168 remain global even when the UI filters one restaurant. Introduce restaurant/tenant-scoped reviewer and operator grants, with thresholds and independent approval for material releases or loads. Preserve existing merchant ownership checks on ordinary wallet services. Test the oversight route's scope itself rather than assuming restaurant filtering enforces it.

Expansion scenario. A restaurant group has hundreds of outlets and seasonal wallet promotions. A Canadian finance investigator must examine a months-old transfer while a Caribbean operator suspends a suspicious wallet. Current recent-history samples and stale reactivation intent make this handover fragile. Filters, conflict feedback, and scoped delegation must support the investigation without weakening the existing ledger isolation.

Recommended controls and UX. Extend the current console with currency-separated totals, historical cursors, reasoned reactivation, conflict dialogs, case ownership, pending-request age, and bulk review jobs with previews and per-row results. Financial exports and bulk approvals must reauthorize each restaurant.

Preserve localized credit formatters; test keyboard dialog focus, Arabic layout, and accessible status announcements.

Dependencies / owner. Restaurant payments owns wallet invariants and recovery; IAM owns restaurant scope; risk operations owns review thresholds and SLAs; frontend/data engineering own history navigation and aggregates.

Measurable acceptance criteria. A two-currency fixture produces two totals. A wallet with 10,000 entries must expose its earliest entry through bounded requests. Racing suspension/reactivation with the same starting version must yield one success and one 409. An operator assigned restaurant A cannot read or mutate B, including direct account IDs. A 10,000-request review job must be resumable, idempotent, and produce complete row results with no duplicate ledger effects.

38. OxSteed Credit - /admin/super/oxsteed-credit and all 13 subpages

Mission / what exists. OxSteed Credit is a substantial existing control plane, not an absent page or a placeholder-only feature. It separates its shared credit journal from restaurant wallets and includes maker/checker approvals, append-only financial history, reservations, disputes, reconciliation, per-currency overview, partner agreements, health, customer accounts, withdrawals, and cards. client/src/admin/adminRouteManifest.js:110-:123 declares the overview and thirteen child routes; client/src/admin/pages/super/oxsteedCredit/shared.jsx:420 defines their navigation. All inspected administrative Credit routes use requireSuperAdmin (server/routes/admin.js:347 onward), with no-store responses, export limits, and mutation limiters. The worldwide requirement is to operate these capabilities safely through many scoped teams, not duplicate the accounting rail.

Evidence / complete subpage inventory. Each child route below is relative to /admin/super/oxsteed-credit; source pointers identify real workflows and their next scale requirement.

1. **Approvals - /approvals:** client/src/admin/pages/super/oxsteedCredit/ApprovalsPage.jsx:28 loads pending actions; :55 issues step-up; :71 approves; :88 rejects with a reason. The page fetches only 50 pending actions. Add older-page navigation, assignment, age sorting, and execution-result drill-through.
2. **Disputes - /disputes:** client/src/admin/pages/super/oxsteedCredit/DisputesPage.jsx:65 loads paginated disputes and :118 resolves with documented outcomes. Migration 640_oxsteed_credit_governance.sql:72 adds dispute type, SLA, and compensation references. Extend existing SLA state with accountable ownership and cross-shift reassignment rather than another dispute store.
3. **Settlements - /settlements:** client/src/admin/pages/super/oxsteedCredit/SettlementsPage.jsx:149 loads history; :215 creates; :237 transitions with expected state; :264 reverses; :287 reconciles. Retain these recovery controls. Add a bounded settlement-cycle work queue and receipt-linked provider ambiguity handling.
4. **Reconciliation - /reconciliation:** client/src/admin/pages/super/oxsteedCredit/ReconciliationPage.jsx:86 loads internal and external history; :127 triggers external reconciliation. server/services/oxsteedCreditExternalReconciliation.js:534 fails a run when Stripe is unavailable rather than declaring a match. Extend the existing run history with asynchronous progress, ownership, and checkpointed large-period processing.
5. **Compliance - /compliance:** client/src/admin/pages/super/oxsteedCredit/CompliancePage.jsx:52 loads identity, sanctions, and case queues; :69 decides items using idempotency headers at :88. server/services/complianceCaseService.js:211 combines three queues, but fixes verification and sanction offsets at zero. Add independent complete paging and counts; jurisdiction review determines policy, not this audit.
6. **Reserves - /reserves:** client/src/admin/pages/super/oxsteedCredit/ReservesPage.jsx:29 loads currency policies and :44 queues updates. The reserve-policy system exists. Add market/entity views, required evidence freshness, and scenario previews of liability coverage; keep policy changes in existing approval flow.
7. **Partners - /partners:** client/src/admin/pages/super/oxsteedCredit/PartnersPage.jsx:203 transitions agreements with expected state; :230 exports partners; :389 displays a searchable, paginated directory. Enrollment, agreements, entitlement controls, and partner ledger functionality already exist. Add scoped onboarding ownership and scalable searchable selectors for the bounded supporting lists.
8. **Health - /health:** client/src/admin/pages/super/oxsteedCredit/HealthPage.jsx:67 loads overview and rail state; :88 toggles using expectedEnabled; :103 freezes; :117 unfreezes. client/src/admin/pages/super/oxsteedCredit/shared.jsx:24 lists workers. Add last-good watermark, incident assignment, and controlled regional incident views while retaining global freeze authority.
9. **Audit - /audit:** client/src/admin/pages/super/oxsteedCredit/AuditPage.jsx:37 loads paginated events and :70 exports filtered history. server/services/oxsteedCreditAdminService.js:562 implements bounded audit filtering. Add asynchronous evidence bundles, scoped redaction, and independent integrity-verification status; do not replace the existing journal/audit chain.

10. **Customers - /customers:** client/src/admin/pages/super/oxsteedCredit/CustomersPage.jsx:46 lists filtered accounts with offset paging; :79 loads value; :98 adjusts; :119 changes status with stable idempotency keys. Add scoped customer search, case context, and reproducible action outcomes without granting regional operators global money authority.
11. **Money Out - /money-out:** client/src/admin/pages/super/oxsteedCredit/MoneyOutPage.jsx:20 embeds the existing component; client/src/admin/pages/super/OxsteedCreditMoneyOut.jsx:78 and :89 load withdrawals/conversions with offset zero. Approval integrates pending actions (:156). Add complete queue pagination and separate reconciliation/return handling per provider outcome.
12. **Cards - /cards:** client/src/admin/pages/super/oxsteedCredit/CardsPage.jsx:23 fetches overview and embeds the existing card workspace. client/src/admin/pages/super/OxsteedCreditCards.jsx:72 paginates and filters; :145 advances physical fulfillment. server/services/oxsteedCreditCardService.js:5 explicitly models cards as revocable credentials resolving to existing credit accounts. This is evidence of an internal credential-card ecosystem, not evidence of external network acceptance or provider-issued bank cards. Add fulfillment integration proof, shipping exceptions, and market capability badges before representing a supported external channel.
13. **Ledger - /ledger:** client/src/admin/pages/super/oxsteedCredit/LedgerPage.jsx:63 pages transactions, :95 inspects a transaction, :108 submits an idempotent refund, and :142 exports. Extend with cursor/snapshot consistency, recovery-case ownership, and large export jobs; preserve compensating entries rather than editing posted history.

Findings. T38-F01 - High [design requirement]: global super-admin restriction prevents least-privilege workforce delegation. The server routes at server/routes/admin.js:365, :391, :436, and :441 protect customer, mutation, approval, and compliance operations globally. This is secure fail-closed behavior, but supporting many market teams by making each a super-admin is not an acceptable enterprise operating model. Add granular scoped Credit read/review/initiate/approve permissions to the existing IAM system, resolve tenant/partner/entity scope on the API, enforce amount thresholds, and preserve independent checker identity.

T38-F02 - High [confirmed defect]: compliance paging cannot advance two queue kinds. server/services/complianceCaseService.js:215 and :216 explicitly pass offset zero, even when the composite endpoint receives another offset at :217. Counts at :225 are returned page lengths, not complete queue counts. The client at client/src/admin/pages/super/oxsteedCredit/CompliancePage.jsx:57 requests no pagination at all. Operators cannot reliably establish workload completeness beyond the first verification/sanction page. Return independent cursors, totals, SLA aggregates, and assigned queues.

T38-F03 - High [code observation]: approvals and Money Out hide older work behind first-page requests. client/src/admin/pages/super/oxsteedCredit/ApprovalsPage.jsx:32 asks for the newest 50 actions, while the service supports offsets at server/services/oxsteedCreditPendingActionService.js:568 and orders newest first at :580. Money Out always requests offset zero at the pointers above. Older pending actions could remain unseen under continuous intake; that scenario is not a measured current backlog. Provide age-first queues, complete navigation, assignment, expiry sweeps, and explicit replay/result states.

T38-F04 - Medium [code observation]: bounded exports still execute synchronously in application memory. server/controllers/oxsteedCreditController.js:1292 accumulates partner pages before constructing CSV at :1304, and rejects oversized exports at :1299. This is a real safeguard, unlike Revenue's unbounded export. At six million users it also means a legitimate large investigation needs a job with immutable filter/scope snapshot, bounded chunks, checksum, download expiry, and cancellation. Retain existing row caps for interactive downloads.

T38-F05 - High [verification gap]: provider execution and recovery are not proven by internal readiness. Existing external reconciliation explicitly records failure and incidents at server/services/oxsteedCreditExternalReconciliation.js:514; its provider-less path fails safely at :536. Verify late webhooks, ambiguous transfers, returned payouts, duplicate delivery, provider unavailable, and account restriction in isolated supported provider environments. Demonstrate that each unsupported market is visible and blocked before money moves. Do not describe provider approvals or jurisdiction permissions as already obtained.

T38-F06 - Medium [code observation]: repeated global summaries and offset pages require a scale plan. server/controllers/oxsteedCreditController.js:850 sums all accounts by currency; many child pages fetch that overview. server/services/oxsteedCreditAdminService.js:824 and :995 use offset lists. Existing indexes and bounded limits matter, but do not establish six-million-user latency. Introduce cached scoped summaries and immutable ordered cursors where concurrent inserts would otherwise shift pages. Benchmark partner coverage aggregation at server/controllers/oxsteedCreditController.js:863 separately.

Expansion scenario. A partner payout disruption generates 20,000 disputes, pending approvals, and withdrawal inquiries across several currencies. Night-shift compliance officers need oldest-first assigned work, treasury needs reconciliation progress, and regional support needs safe read access. Existing reserves, freezes, journal integrity, and approval machinery should coordinate that response. First-page queues or global-only roles would force central administrators to become a bottleneck despite healthy workers.

Recommended controls and UX. Extend existing systems with scoped role bundles, entity/currency selectors, assignment and SLA handovers, independent queue pagination, durable jobs, provider capability states, and accessible outcome receipts. Reuse client/src/admin/pages/super/oxsteedCredit/shared.jsx:47 dialog focus trapping and credit currency/localization utilities. Avoid blanket claims that accessibility or currency precision is absent; the remaining requirement is executed coverage across every child workflow.

Dependencies / owner. Credit engineering owns accounting and replay; treasury owns reconciliation/reserves; risk owns compliance/disputes; IAM owns scope; fulfillment/provider integration teams own cards and money-out evidence; SRE owns queue capacity. Qualified market reviewers and providers decide external approval requirements.

Measurable acceptance criteria. Browse and assign the oldest of 10,000 items in each queue; counts must include unseen pages. A maker must receive 403 for self-approval and concurrent checkers must produce one execution, preserving the existing tests in server/__tests__/oxsteedCreditGovernance.test.js:4. Export one million authorized records through a resumable job with bounded memory. Demonstrate six million seeded accounts, p95 scoped-list latency under two seconds, and zero cross-tenant records. Provider failure drills must leave an auditable unresolved/reconciled outcome without duplicate credit. Run all thirteen child routes and overview in authenticated locale, keyboard, and regional-negative tests with zero fixture skips. client/scripts/creditAdminIdempotency.test.mjs:5 already tests key reuse; retain and extend it rather than reimplementing idempotency.

39. Revenue - /admin/super/revenue

Mission / what exists. Revenue is an implemented period-based view of platform-ledger receipts, subscriptions, job fees, negative movements, growth, and recent transactions. It includes CSV export, translated transaction/status labels, loading/error handling, and retry. The endpoint joins user information to ledger entries and returns a deliberately small recent list. The enterprise mission is a reproducible commercial report that finance analysts can explore and export safely, with transparent source semantics. It should remain distinct from Money's cross-business map and Financials' ledger inspection.

Evidence. client/src/admin/pages/super/Revenue.jsx:13 requests the selected period; :26 formats a dollar amount; :31 downloads a blob; :107 identifies the table as recent transactions. server/routes/admin.js:248 and :249 protect both summary and export with view_financials. server/controllers/superAdminController.js:1522 builds the report, :1541 counts all positive amounts as total revenue, :1544 labels all negative amounts refunds, :1574 caps the table at 50, and :1593 implements export. client/scripts/adminRouteContracts.test.mjs:63 checks summary-route permission wiring; this does not test large exports, financial classification, or browser behavior.

Findings. T39-F01 - High [code observation]: the export has no inspected row or memory bound. The query at server/controllers/superAdminController.js:1605 selects the entire chosen period without LIMIT, and :1618 maps all rows into strings before joining a second CSV representation at :1623. client/src/admin/pages/super/Revenue.jsx:34 then receives a complete blob. A year of receipts for millions of users can consume substantial database, server, and browser resources. This differs from capped Credit exports. Use queued export jobs with immutable period/scope snapshots and bounded chunks; retain a small interactive export limit.

T39-F02 - High [code observation]: sign-based labels overstate their accounting meaning. server/controllers/superAdminController.js:1541 includes every positive ledger source in total revenue; :1544 treats every negative amount as a refund, and :1567 generates transaction status from amount sign. Those are current software definitions, not validated accounting rules. A negative adjustment or outgoing fee is not proven to be a customer refund. Introduce a finance-approved versioned classification using source type, original transaction, and accounting purpose. Preserve the original ledger amount and show an unmapped category explicitly.

T39-F03 - High [design requirement]: currency, entity, and reporting-zone controls are required. client/src/admin/pages/super/Revenue.jsx:28 renders $ plus two decimals, and the selected-period query at :1545 has no currency/entity grouping. Do not infer that the production ledger already contains mixed currencies. Establish that invariant or group currencies before worldwide rollout, and distinguish native and converted figures. The current global grant boundary correctly blocks regional-only grants; add scoped reporting APIs instead of treating a client market filter as an access control.

T39-F04 - Medium [code observation]: CSV quoting is conditional only on a comma. server/controllers/superAdminController.js:1620 quotes a field only when it contains a comma; quotes and newlines alone are not sufficient, and spreadsheet formula prefixes are not neutralized. The exported user-email field is source data, so the consumer contract must handle untrusted cells. Use a tested CSV serializer and explicit spreadsheet-safe text handling. This finding describes the serializer's behavior, not proof of a current malicious email record.

Expansion scenario. A revenue analyst exports a year for a regional board review while support teams are busiest. A large synchronous request competes for resources, and the downloaded file may conflate adjustments with refunds. The analyst needs a scoped, complete export that survives a browser reconnect, carries its classification version, and gives another analyst the same result from the same reporting cutoff.

Recommended controls and UX. Add asynchronous export progress/cancellation, bounded preview, saved date/entity/currency filters, source definitions, original-transaction drill-through, and snapshot provenance. Explicitly retain the "recent" label for the 50-row sample. Provide a complete paginated drill-down rather than turning the current overview into an unbounded table. Use locale-aware money and date formatting, accessible filter names, and status announcements for export completion.

Dependencies / owner. Finance owns classifications; data engineering owns snapshot/export processing; IAM owns scope; admin frontend owns filtering and accessible progress; application security reviews

exported-cell handling.

Measurable acceptance criteria. Export one million synthetic records without application memory growing with total file size, with resumable progress and a complete row count/checksum. CSV fixtures containing commas, quotes, newlines, and formula prefixes must remain valid and inert in approved consumers. Negative adjustments must be distinct from refunds. Two analysts using the same cutoff must reproduce identical totals. Regional direct API and export attempts outside scope must return 403; authenticated browser tests must execute rather than skip.

40. Fee Config - /admin/super/fees

Mission / what exists. Fee Config already supports individual fee edits, required numeric/range validation, confirmation dialogs, optional reason capture, change history, reset, and subscription-price synchronization. It uses the existing fee_config store and fee calculator. The server makes writes super-admin-only while allowing financially authorized admins to read and preview. This is a useful global policy surface, but a worldwide workforce needs controlled releases, accurate before/after evidence, and provider-state visibility for each change.

Evidence. client/src/admin/pages/super/FeeConfig.jsx:82 submits key/value/reason; :175 confirms; :188 displays history; :253 loads configuration/history; :277 resets. server/routes/feeConfig.js:6-:10 enforce read/write separation. server/controllers/feeConfigController.js:13 performs strict numeric parsing, :92 checks stored min/max, :101 inserts history, :116 updates configuration, :141 synchronizes subscription prices, and :225 resets rows. server/migrations/017_phase6_fee_config.sql:9 defines numeric values, limits, and timestamps. Its schema provides a foundation; no new fee database is needed.

Findings. T40-F01 - High [code observation]: a fee change and its history are not atomic or versioned. The read at server/controllers/feeConfigController.js:67, audit INSERT at :101, and UPDATE at :116 are separate pool queries. There is no transaction or expected revision in this controller. Two operators can log the same old value and then overwrite each other; an UPDATE failure can leave a history entry claiming a change that never took effect. Use a transaction, row lock or compare-and-swap revision, and transactional audit. Expose conflicts instead of silently replacing stale intent.

T40-F02 - High [code observation]: provider synchronization failure still returns success. syncSubscriptionToStripe catches errors at :184, logs them, and returns; the outer handler responds success at :134. The store can therefore hold the new fee while the plan/provider state remains old. Add a durable provider-sync operation with status, retry key, receipt, and visible "pending/failed synchronization" outcome. Do not claim Stripe is unavailable now; this is the implemented failure path. The USD provider currency at :155 also needs explicit applicability before regional subscription policy expands.

T40-F03 - High [code observation]: reset can partially apply without equivalent synchronization. The reset loop at :233 separately logs and updates each changed key, then invalidates only the fee cache at :254. It does not call the subscription synchronization path or the trial-cache invalidation used by individual writes. An interrupted reset can leave a mixed policy revision. Release a complete policy version atomically, with one operation receipt and the same downstream propagation rules as ordinary updates.

T40-F04 - Medium [confirmed defect]: preview substitutes defaults for valid zero values. At :203, :206, and :207, JavaScript || replaces zero percentage inputs with nonzero defaults. A requested zero protection/broker percentage cannot be represented faithfully in this preview. Whether every zero rate is allowed in production is a policy question; the preview must either validate/reject it explicitly or calculate it exactly. Reuse the authoritative fee calculator so preview and checkout do not drift.

Expansion scenario. Treasury schedules a regional promotion while another operator restores ordinary rates after a provider incident. Concurrent saves and partially propagated subscription changes could produce different prices across checkout and administrative screens. A release must show market applicability, affected products, expected net payouts, and propagation status before the shift hands over.

Recommended controls and UX. Extend existing edit/history/preview controls with revisions, required change tickets, impact previews, effective dates, independent approval for material changes, and staged rollout. Display old/new values and units clearly, including whether they are percentages or minor units. Add complete history paging beyond the current LIMIT 100 at :48, a diff view, and a governed rollback operation. Verify labeled numeric inputs, keyboard confirmation, translated units, and locale-specific decimal input.

Dependencies / owner. Payments owns authoritative calculation; billing owns Stripe synchronization; finance owns release thresholds; SRE owns cache propagation; admin frontend owns conflict and status UX. Market rules and approvals require jurisdiction/provider review.

Measurable acceptance criteria. Two saves from one revision yield one success and one 409, with exactly one matching audit row. Inject failures between audit/update and verify no partial commit. Zero-rate previews must match calculator output or return explicit validation. Provider timeout must show

pending/failed rather than unconditional success. Reset must apply one policy revision and propagate trial/subscription changes consistently. Verify all serving replicas converge within an agreed 30-second budget.

41. Pricing Config - /admin/super/pricing

Mission / what exists. Pricing Config is a real six-field editor for public tier prices and labels. It loads /config/pricing, allows edits, asks for confirmation, and reports save failures. The server stores allowed keys transactionally and writes an admin audit record using the same transaction. It is separate from Fee Config's runtime calculations and Pricing Overrides' country subscription prices. The enterprise mission must make that separation explicit: employees should know whether they are editing customer-facing copy, a chargeable numeric amount, or the actual billing policy.

Evidence. client/src/admin/pages/super/PricingConfig.jsx:11 seeds the six fields, :23 loads public pricing, :36 sends the form, :68 defines editable fields, and :106 confirms immediate change. server/routes/config.js:6 exposes public reads and :10 restricts writes by super-admin role. server/controllers/configController.js:38 defines defaults, :48 loads current values, :75 validates and updates, :88 requires numeric _price values, and :96-:117 transactionally stores and audits them. This transaction is a positive existing control, unlike the inspected fee update path.

Findings. T41-F01 - High [confirmed defect]: the default form cannot pass its own server validation unchanged. Both client/src/admin/pages/super/PricingConfig.jsx:16 and server/controllers/configController.js:43 use tier3_price: 'up to 17%'. The client submits the entire form at :40; the server requires each _price entry to match a numeric-only regex at server/controllers/configController.js:91. Therefore changing only a label or tier-two price while retaining the default tier-three display string causes validation failure. This is a directly observed contract inconsistency, not a historic SQL claim. Separate display copy from typed charge values or define a consistent explicit schema for percentage-range display fields.

T41-F02 - High [code observation]: read failures masquerade as valid defaults. server/controllers/configController.js:62 catches read errors and responds 200 with DEFAULT_PRICES at :65. The public page may need a fallback, but the administrative editor cannot distinguish a real stored policy from dependency failure. In combination with a whole-form save, an operator could overwrite valid values using fallback content. Return freshness/provenance and an administrative degraded state that blocks publication until the canonical revision is available.

T41-F03 - Medium [code observation]: transactionality does not prevent lost updates. The upsert at server/controllers/configController.js:101 updates each allowed key without an expected version or read revision. Two regional communications teams can replace each other's labels; the existing audit captures the new state at :112 but no full before-state comparison. Add a policy revision, compare-and-swap publication, a diff, and a conflict view that retains unsaved edits. Do not duplicate the existing transactional audit machinery.

T41-F04 - Medium [design requirement]: pricing copy needs market/locale applicability and charge consistency. Default labels at :40-:44 are free text, while the UI uses English defaults and untyped text fields. The confirmation at client/src/admin/pages/super/PricingConfig.jsx:109 says changes take effect immediately. A worldwide enterprise needs localized copy, currency/billing-period context, effective dates, previews, and a link to the charge policy used by the relevant plan. It is not established that this editor updates Stripe or runtime fees; expose the intended contract rather than imply it does.

Expansion scenario. A French communications lead updates a public tier label while billing publishes a new regional offer. The editor should clearly identify approved French copy and its billing revision. Today, the default tier-three string can reject an otherwise simple save, and a transient database failure can silently load fallback text. Neither issue is evidence that the running deployment is unhealthy overall.

Recommended controls and UX. Define typed price/display fields, locale-specific labels, publication preview, draft revision, effective time, and rollback to an audited prior revision. Show public-versus-charge policy explicitly and warn on mismatches supported by actual linked data. Associate each label with its input using htmlFor/ID; the current labels at client/src/admin/pages/super/PricingConfig.jsx:85 lack that explicit connection. Reuse shared accessible confirmation dialogs and validate all supported locale layouts in executed browser tests.

Dependencies / owner. Pricing product owns semantics; billing owns charge linkage; localization owns translated copy; IAM verifies the administrative authentication contract; frontend owns publication UX. The

general role middleware in server/routes/config.js:10 should be checked against the admin authentication standard instead of assuming equivalent MFA enforcement.

Measurable acceptance criteria. Save the default form unchanged and edit only a label successfully under the chosen schema. A failed canonical read must not enable a destructive fallback publication. Concurrent same-revision saves must conflict deterministically. Every displayed tier must identify its locale, currency, applicability, and linked billing revision or explicitly state that it is presentation-only. Keyboard and screen-reader tests must identify every field and confirmation outcome; fixture-dependent tests must not skip.

42. Pricing Overrides - /admin/super/pricing-overrides

Mission / what exists. Pricing Overrides already implements country-level premium pricing, currency comparison statistics, inline edits, reason fields, confirmation, server history, batch updates, and real Stripe synchronization. The client displays configured countries and grouped currency comparisons; the service resolves country prices with a US fallback and local caching. This is an existing regional pricing system, not a missing capability. Enterprise readiness depends on consistent numeric/provider revisions, controlled market delegation, and reproducible rollouts rather than an additional pricing engine.

Evidence. client/src/admin/pages/super/PricingOverrides.jsx:18 loads overrides/comparison, :31 opens an edit, and :54 saves. server/routes/pricingOverrides.js:6-:12 require super-admin access for list/history/write/batch/sync. server/controllers/pricingOverrideController.js:43 validates writes, :112 runs batch updates, and :179 checks provider configuration before syncing. server/services/pricingService.js:101 updates prices, :198 loads history, and :224 synchronizes real Stripe prices. server/migrations/160_location_based_pricing.sql:6 defines country, currency, price, optional floors/ceilings, and provider ID. server/__tests__/pricingServiceStripeSync.test.js:4 tests synchronization and :63 rejects absent configuration.

Findings. T42-F01 - High [confirmed defect]: changed overrides retain a provider ID that synchronization then skips. The upsert at server/services/pricingService.js:147 updates numeric price/currency but does not reset or version stripe_price_id. The synchronization loop at :240 skips any existing ID as already synchronized. Thus an override previously synced at amount A can be saved at amount B and still retain provider price A through another sync call. The current test at pricingServiceStripeSync.test.js:57 deliberately covers skipping an existing ID, but does not establish equivalence after editing. Bind provider IDs to amount/currency/revision and resynchronize changed revisions idempotently.

T42-F02 - High [code observation]: pricing history and mutation are separate operations. server/services/pricingService.js:125 reads an old value, :133 inserts history, and :141 upserts the price using independent pool queries. Concurrent operators can record misleading before-values; failed writes can leave misleading history. Country-level editions also have no expected revision in client/src/admin/pages/super/PricingOverrides.jsx:56. Apply transactionality, a revision check, and atomic audit, with a conflict diff that preserves the proposed regional price.

T42-F03 - Medium [code observation]: batch work has no inspected workload cap or durable result handle. server/controllers/pricingOverrideController.js:116 requires a nonempty array, then :123 processes updates in one HTTP request. Each row can succeed independently, and :164 returns overall success with a separate errors array. Partial success is reasonable when explicit, but retried requests need an operation ID, per-row idempotency, progress, and resumable failed rows. The country count is finite; thousands of duplicate updates in an unrestricted request still create needless audit/provider workload.

T42-F04 - High [code observation]: numeric and currency contracts need stronger validation. The controller at :50 checks truthiness and negativity, floors input at :66, and checks only a three-character currency length at :53. Client edit/display logic divides by 100 at client/src/admin/pages/super/PricingOverrides.jsx:33 and :111 regardless of currency exponent. Migration floors/ceilings at 160_location_based_pricing.sql:11 exist, but are not consulted by this update service. Require supported country/currency pairings, safe integer minor units, valid exponent formatting, and enforced policy bounds.

Expansion scenario. A Japanese market operator proposes a new price while European billing prepares another launch. The country comparison is useful, but a changed amount with an old provider price can make administration and checkout disagree. An interrupted batch also needs a precise receipt showing which markets published, which still await approval, and which failed synchronization.

Recommended controls and UX. Extend the existing table with scoped market ownership, scheduled effective revisions, maker/checker release thresholds, provider-sync state, complete history, and rollback to a verified price revision. Preserve the existing real Stripe implementation; add changed-price and ambiguous-provider-result coverage. Make fallback-to-US explicit to operators and customers where product policy requires it, instead of silently treating fallback as local configuration.

Dependencies / owner. Billing owns provider revision binding; pricing owns supported currencies/bounds; regional finance owns proposals; IAM owns market scope; admin frontend owns conflict/partial-result UX. Provider currency support and market approvals require provider/jurisdiction review.

Measurable acceptance criteria. Sync A, edit to B, and sync again: the active override must reference a provider price matching B and its currency. Race two revisions and obtain one conflict with accurate audit. Test JPY and a three-decimal currency without dividing every amount by 100. Enforce floors, ceilings, integer bounds, and unsupported currency rejection. A bounded batch must return durable row receipts and replay without duplicate price creation. Regional operators must never publish outside assigned markets.

43. Financials - /admin/super/financials

Mission / what exists. Financials is an existing platform-ledger inspector with type tabs, page controls, gross/job-fee/subscription/refund summary cards, and CSV export. The client uses /admin/financials; the super-prefixed path is the SPA navigation route. Server reads require view_financials, while the separate generic export route requires export_data. It should support an accountant tracing a movement to its business event, with a stable filtered population and an unambiguous export, rather than pretend to be a complete general-ledger or close product today.

Evidence. client/src/admin/pages/super/Financials.jsx:21 fetches type/page/limit 30, :31 exports using /admin/export/revenue, :78 renders totals, and :119 changes type/reset page. server/routes/admin.js:242 protects ledger reads; :261 protects generic exports separately. server/controllers/superAdminController.js:1297 implements ledger paging and totals, :1303 filters rows by type, :1305 computes unfiltered totals, and :2427 exports generic data with a bounded limit at :2440. client/scripts/adminRouteContracts.test.mjs:60 verifies the read permission contract. Existing Financials must not be described as absent because its API path differs from its browser route.

Findings. T43-F01 - Medium [code observation]: filtered rows and summary cards describe different populations. A type condition is used in the row query at server/controllers/superAdminController.js:1304, but totals at :1312 scan the entire ledger. Global totals may be intentional context; the response has no explicit summary-scope contract distinguishing them from the visible subset. Label these cards lifetime/global or return filtered and global totals separately. Provide a visible count and selected date range so an accountant can reconcile the population rather than infer it from a tab name.

T43-F02 - High [code observation]: export does not preserve the view's filter. client/src/admin/pages/super/Financials.jsx:33 calls the generic revenue export without type or page/date scope. server/controllers/superAdminController.js:2440 exports a newest-first bounded ledger selection. The export can therefore differ from the table and be incomplete for large history. Bounded extraction is safer than an unbounded response, but the UI must show the export scope and completeness. Introduce a full filtered snapshot job with a result count, cap disclosure, and checksum; keep quick samples explicitly labeled.

T43-F03 - Medium [code observation]: offset paging lacks a stable snapshot and total contract. server/controllers/superAdminController.js:1304 orders only by created time and offsets into live rows; :1314 returns page/limit without a total or immutable cursor. New entries can shift pages and equal timestamps do not provide a deterministic tie-breaker. Add (created_at,id) ordering and snapshot/keyset pagination, with totals computed against the same cutoff. This is a query design exposure, not proof that entries were lost from the ledger itself.

T43-F04 - High [design requirement]: worldwide oversight needs currency/entity and financial-close context. Card amounts at client/src/admin/pages/super/Financials.jsx:84 explicitly render US dollars using en-US; totals at server/controllers/superAdminController.js:1307 are not grouped by currency or entity. Establish the existing ledger's currency invariant before treating these as worldwide consolidated numbers. Add period status, classification version, source reference, original-versus-compensating entry links, and reconciliation status. A complete tax or statutory accounting product is not asserted as a legal requirement by this engineering audit.

T43-F05 - High [confirmed defect]: related manual financial recovery omits a successful HTTP response. The Financials page itself is a ledger viewer; the associated job-refund API is server/routes/admin.js:250. server/controllers/superAdminController.js:1355 delegates to the existing refund-attempt service, and :1363 responds 202 for unresolved provider reconciliation. For a succeeded refund it proceeds through audit handling at :1369-:1394 and ends without res.json, res.send, or another response. The caller can time out after money has already been refunded. Fix the success receipt using the existing attempt ID/status/request ID; retain reconciliation and replay controls. The source-only assertion at server/__tests__/ws5-audit-evidence.test.js:162 checks audit wording, not response completion. Require an HTTP behavior test of succeeded, pending, replayed, and audit-failure outcomes.

Expansion scenario. A quarter-end investigator selects job fees, receives a list of those entries, and downloads a file that also includes subscriptions and adjustments but excludes older records beyond the generic cap. Another investigator enters new movements during paging. The team needs reproducible evidence showing exactly which entries were examined and whether the extract is complete for that period.

Recommended controls and UX. Preserve read/export permission separation, but hide or explain an export action when its grant is absent. Add date/entity/currency filters, a stable snapshot badge, scope labels, reference search, complete export jobs, and drill-through to source transactions. Make summaries and table footers explicit about global versus filtered totals. Use locale-aware currency formatting and accessible type selection, table headings, and pagination announcements.

Dependencies / owner. Finance controls owns reporting definitions and close snapshots; ledger engineering owns immutable references; data engineering owns pagination/export; IAM owns scoped exports; frontend owns scope clarity. Coordinate with Revenue's export and Money's reporting bridge instead of adding a second ledger.

Measurable acceptance criteria. A job-fee filter must return a matching filtered total or visibly labeled global total. An export must reproduce the selected cutoff and population exactly, with truncation never implicit. Insert rows during paging and confirm each snapshot record appears once. Benchmark deep history at declared production-like cardinality with p95 two-second pages. A view-only admin must be unable to export by direct API; regional and currency fixtures must preserve their authorized populations and formatting.

44. Payouts - /admin/super/payouts

Mission / what exists. Payouts is an existing helper-payment oversight page with pending, processing, completed, and failed tabs, monetary summary cards, a detail drawer, and a retry confirmation. Its API derives the list from payment records and connected-account information. Failed retries are super-admin-only, guarded by a conditional state update, and completed payments are replay-safe no-ops. This is meaningful recovery functionality. Enterprise operation should make the distinction between gross payment, helper entitlement, transfer, and bank payout explicit and expose complete exception queues.

Evidence. client/src/admin/pages/super/Payouts.jsx:17 renders the drawer, :60 fetches a status list, :76 requests retry, :174 identifies failed rows, and :178 displays p.amount. server/routes/admin.js:243 grants read access and :246 restricts retries. server/controllers/superAdminController.js:1398 bounds paging, :1404 maps tab names to payment states, :1414 returns gross and helper amounts, :1430 sums helper payouts, and :1460 implements retry. Database safeguards also exist in server/migrations/601_payout_transaction_state_integrity.sql:17 and the payout idempotency index in 620_money_criticals.sql:14; these protect payout_transactions, not automatically every payments operation.

Findings. T44-F01 - High [confirmed defect]: displayed row amount is the gross payment, not the helper payout used by summaries. The API returns both amount and helper_payout at server/controllers/superAdminController.js:1414. Summary totals use helper_payout at :1432, but the table at client/src/admin/pages/super/Payouts.jsx:178 and drawer at :22 show amount. With a 100-unit payment and 83-unit helper entitlement, the page presents a 100-unit row against an 83-unit aggregate. Show net helper payout as the principal amount and gross/platform fee as separately labeled detail.

T44-F02 - Medium [confirmed defect]: successful database states receive the wrong visual treatment. The completed tab selects released records at server/controllers/superAdminController.js:1407; the client at client/src/admin/pages/super/Payouts.jsx:182 assigns the green success style only to completed. Likewise, captured processing rows do not match the processing branch at :184. These records fall into the red fallback. Normalize presentation states in one response adapter, preserve raw provider/payment states in details, and test every mapped state rather than comparing incompatible vocabularies.

T44-F03 - High [code observation]: the UI cannot navigate the bounded API population. The request at client/src/admin/pages/super/Payouts.jsx:63 sends status but no page; the page has no list-page state or next-page control. The server defaults to 25 at server/controllers/superAdminController.js:1401. A finite list size is appropriate, but millions of users require complete paging, oldest-pending search, provider reference filters, and queue counts. A first-page retry tool cannot constitute a complete global recovery queue.

T44-F04 - High [code observation]: retry mutation has incomplete durable operating evidence. The guarded UPDATE at server/controllers/superAdminController.js:1485 prevents simultaneous retries from both changing a failed row, and :1474 safely handles already-released payments. Retain those controls. However, the audit call is standalone at :1502, its failure catch is empty at :1513, and the retry accepts no operator reason. It clears failure_reason at :1487 without an inspected attempt-history receipt in this handler. Make requeue, reason, prior failure, and audit atomic or backed by a durable event, with explicit downstream processing ownership. This is not a claim of double payment today.

Expansion scenario. A provider disruption affects 8,000 helper payments. Regional recovery teams need to distinguish blocked accounts, unknown transfer outcomes, refunded payments, and genuinely retryable failures. The current "failed" filter includes refunded/disputed rows (:1408), while only actual failed rows can retry. Keep that guard, but make exception types and next actions intelligible and assign work across shifts.

Recommended controls and UX. Add complete cursor paging, gross/net breakdown, normalized status badges, accountable exception ownership, age/SLA filters, and provider receipt links. Scoped retry permissions should include amount limits and a reconciliation prerequisite for ambiguous outcomes. Bulk recovery must be a previewed asynchronous job rechecking each payment state, not an unchecked mass status reset. Make the drawer keyboard-accessible with dialog semantics, focus trapping, Escape, and focus restoration; current custom markup at client/src/admin/pages/super/Payouts.jsx:30 needs executed accessibility verification.

Dependencies / owner. Payments and provider integration own state mapping/retry safety; treasury owns reconciliation and threshold policy; IAM owns regional initiation rights; SRE owns worker pickup and alarms; frontend owns amounts/status/complete navigation.

Measurable acceptance criteria. A gross 100/net 83 fixture shows net 83 consistently. Released and captured rows receive the correct successful/processing labels. All 8,000 exception records must be reachable without unbounded requests. Concurrent retries yield one requeue; completed/refunded/disputed records cannot be replayed incorrectly. Inject audit and provider failures and retain a durable reasoned receipt plus a visible unresolved state. Run keyboard drawer tests and regional direct-API negative tests with zero fixture skips.

45. Conversations - /admin/super/conversations

Mission / what exists. Conversations is an existing privileged investigation console: searchable/status-filtered customer/helper threads, message history, linked quotes, proof of payment, agreements, bookings, and block/archive/reactivate controls. Opening a thread requires a documented break-glass reason. The client sends that reason in a POST body, and the server records a dedicated access row before returning messages. This is a meaningful privacy safeguard. The enterprise mission is controlled case investigation with complete evidence, narrowly delegated authority, and audited decisions across regions and support shifts.

Evidence. client/src/admin/pages/super/ConversationsAdmin.jsx:45 loads thread detail; :51 documents POST-body break-glass; :268 renders linked artifacts; :336 requests the list; :353 changes status; :515 renders the access-reason modal. server/routes/admin.js:215-:221 require super-admin access for all these operations. server/controllers/conversationsAdminController.js:12 writes the dedicated access log, :96 lists conversations, :167 validates thread reason, :193 logs access before reading, :200 fetches messages, and :238 changes status. server/__tests__/chatPopAgreementsAdmin.test.js:143 tests break-glass reason normalization; this is not a complete authorization/browser investigation test.

Findings. T45-F01 - High [code observation]: complete history retrieval is unbounded while list navigation is incomplete. Thread messages at server/controllers/conversationsAdminController.js:200 have no limit or cursor. The list uses raw parsed page/limit values at :98 and :136, without the bounded parser used by finance. The client request at client/src/admin/pages/super/ConversationsAdmin.jsx:337 sends search/status but no page, and its footer at :478 only states showing count. Add bounded list and chronological message cursors; ensure an old thread is reachable without fetching its entire history.

T45-F02 - High [code observation]: API status writes do not enforce the UI's reason or stale-state expectations. The client requires a reason before confirmation at :183; the server allows reason || null at server/controllers/conversationsAdminController.js:258. It reads status and performs an unconditional UPDATE at :249, then treats audit as best effort at :261. An API caller can bypass the reason prompt, and competing shifts can reverse each other's moderation decisions. Enforce reason, expected status/version, and transactional audit; preserve the existing allowed status list.

T45-F03 - Medium [code observation]: linked-evidence failures resemble absence. Artifact queries at server/controllers/conversationsAdminController.js:44-:81 catch failures into empty arrays. The client displays only the first eight items per category at client/src/admin/pages/super/ConversationsAdmin.jsx:283, :293, :303, and :313. This is a real artifact view, but it does not establish investigation completeness. Distinguish unavailable from empty, expose total counts/cursors, and show whether payment requests returned by the API are represented in the investigation UI. An investigator should not infer "no agreement" from an unavailable source.

T45-F04 - High [design requirement]: reason-only global break-glass is insufficient workforce delegation. The super-admin restriction prevents ordinary agents from indiscriminate reads, but global access should remain exceptional. Add assigned-case access, scoped investigative permissions, purpose/ticket reference, expiry, read audit review, and supervisor escalation for cross-region cases. Retain the dedicated access log's fail-before-read behavior at :193. Do not route every normal support interaction through an all-powerful account or recreate the existing support inbox.

Expansion scenario. A cross-border marketplace dispute spans a year of messages, several payment requests, and dozens of agreement versions. One team archives the thread for retention review while another blocks it for safety. Investigation requires a stable case record, earliest messages, complete linked artifacts, and reasoned conflict handling. Current first-page navigation and unrestricted message retrieval make this difficult at a six-million-user scale.

Recommended controls and UX. Extend this investigation view with case assignment, scoped access, expiring break-glass sessions, complete evidence cursors, and explicit unavailable-source badges. Provide a sanitized asynchronous evidence bundle only under authorized scope, with retention settings reviewed by qualified stakeholders. Add debounced search, reference lookup, redacted previews, original-language display, and accessible transcript navigation. The custom modal at client/src/admin/pages/super/ConversationsAdmin.jsx:515 needs dialog role, accessible naming, focus

confinement/restoration, and executed screen-reader/keyboard tests.

Dependencies / owner. Trust/support operations own investigation workflow; privacy engineering owns access review and redaction; IAM owns purpose/scope; messaging owns bounded history and artifact availability; frontend owns accessibility and evidence completeness.

Measurable acceptance criteria. Navigate a 100,000-message synthetic thread using bounded pages and reach the earliest message. A missing reason on direct status API must return 400; concurrent same-version decisions must yield one 409. Deny read when access-log persistence fails. Inject an artifact-source error and render unavailable, not absent. Every privileged read must identify actor, purpose, case, scope, and time. Execute translated modal/transcript tests and regional-negative authorization checks without matrix skips.

46. Admin Accounts - /admin/super/admin-accounts

Mission / what exists. Admin Accounts implements creation, role selection, activation/deactivation, activity inspection, and forced logout. The account-management grant delegates ordinary administrator management, while creating or disabling a super-admin remains restricted to a super-admin. Server admin authentication checks active status, token revocation, and TOTP enrollment. Disabling an account updates sessions and sends a live logout signal. The enterprise mission is controlled joiner/mover/leaver administration for authorized employees without converting the entire 3,500-person workforce into privileged platform administrators.

Evidence. client/src/admin/pages/super/AdminAccounts.jsx:23 creates an account, :83 selects role, :210 changes status, and :222 forces logout. server/routes/admin.js:224-:228 enforce manage_admin_accounts; :188 makes force logout super-admin-only. server/controllers/superAdminController.js:2885 lists accounts, :2929 creates, :2973 toggles status, and :3014 pages activity. server/middleware/adminAuth.js:29 requires TOTP, :52 checks active users, and :19 checks revocation. server/__tests__/adminPermissionScope.test.js:23 covers scoped-grant rejection; server/__tests__/adminAuthSecurity.test.js:41 covers revoked admin tokens. Existing role and revocation safeguards must be credited accurately.

Findings. T46-F01 - Medium [code observation]: account discovery loads all admins and historical activity aggregates. server/controllers/superAdminController.js:2887 returns the entire administrative population, with correlated COUNT/MAX audit subqueries at :2891 and :2892, and no pagination. This may be acceptable for a small privileged group, but auditors and regional managers need bounded search, manager/department scope, status/MFA filters, and aggregate activity summaries. The expected workforce count does not mean all employees should appear as admins; preserve the separate employee/internal-staff model at :2905.

T46-F02 - High [verification gap]: the promised first-login change is not established by account creation. The response at server/controllers/superAdminController.js:2965 says users must change their password on first login. The INSERT at :2950 stores no mandatory-change or invitation state; the inspected authentication/migration searches did not establish a linked first-login enforcement mechanism. This is a missing proof, not a claim that password hashing is absent: hashing at :2948 is present. Prefer a user-owned enrollment invitation tied to identity, expiry, and MFA. If temporary enrollment remains, demonstrate enforced rotation before privileged action and avoid administrator-mediated distribution as the scalable onboarding model.

T46-F03 - High [code observation]: status changes have a race guard but incomplete transactional lifecycle. server/controllers/superAdminController.js:2986 compares the old active state and returns 409 on concurrent change, which is a positive control. Session invalidation at :2991, live notification at :2995, and standalone audit at :2996 occur after the account update. A later failure can leave an uncertain UI outcome; reactivation can also expose old grants that remain active. Use an explicit lifecycle operation receipt, durable audit, and reactivation review of grants/assignments. Do not describe inactive users as retaining admin API access: the middleware independently rejects them.

T46-F04 - High [design requirement]: delegated management needs organizational bounds and separation of duties. The global manage_admin_accounts grant can create/manage ordinary admins across the platform; the server protects super-admin targets at :2982, but no departmental management scope is resolved in these handlers. Add team/entity-scoped account managers, privileged-role approval, sponsor ownership, access-review due dates, and HR/identity-provider lifecycle integration where the enterprise selects that model. These enhancements should feed the existing account/grant stores rather than create shadow identities.

Expansion scenario. A 200-person regional support center joins, transfers to a contractor organization, and later offboards. Their managers need to approve roles, confirm MFA readiness, remove privileges promptly, and retain audit evidence. Unbounded lists and manually administered enrollment cannot make that chain reproducible; an inactive flag alone does not show whether all assigned access has been reviewed before later reactivation.

Recommended controls and UX. Add sponsor/team metadata, search/pagination, enrollment state, MFA/device/session indicators, expected-state actions, expiring elevated roles, and a reviewed reactivation

checklist backed by actual data. Show that force logout is unavailable to a delegated manager until authorized, rather than inviting an action that the API correctly rejects. Make custom creation/activity modals accessible with focus management and explicit field labels; verify supported locales rather than claiming translations do not exist.

Dependencies / owner. IAM/security owns lifecycle and approval; people operations owns employment signals/sponsors; regional managers own access reviews; admin frontend owns discoverability; SRE verifies prompt revocation across sockets and sessions.

Measurable acceptance criteria. Onboard 200 synthetic employees through individual invitation/enrollment records without granting super-admin by default. Prove first-use rotation/enrollment gates or correct the promise. Offboarding must block fresh admin API calls and revoke live sessions within 60 seconds. Reactivation must show reviewed grants, not silently resume expired employment authority. Same-version status races must conflict. Unauthorized team managers must receive 403 on cross-team creation/status/activity APIs; executed keyboard and locale tests must cover custom dialogs.

47. Permissions - /admin/super/permissions

Mission / what exists. Permissions implements a scope catalog, time-limited grants, active/expired/revoked displays, creation, and revocation. The API validates supported permissions, future expiry no more than one year, active regular-admin grantees, and optional region IDs. Only super-admins can create/revoke; delegated account managers can inspect according to the server's narrower visibility rule. Global APIs explicitly refuse regional-only grants. This is an existing permission system to expand, not a reason to construct another role database.

Evidence. client/src/admin/pages/super/PermissionGrants.jsx:61 creates the modal, :63 defines its form, :73 loads active admins, :276 fetches grants/scopes, and :292 revokes. server/routes/admin.js:295 provides self-lookup and :300-:303 protect catalog/list/write operations. server/controllers/adminPermissionController.js:7 defines scopes, :71 lists grants, :126 creates, :143 bounds expiry, :166 supports region scoping, and :202 revokes. server/migrations/049_admin_search_and_permissions.sql:42 defines grants and :54 their indexes. server/__tests__/adminPermissionScope.test.js:23 verifies a regional grant cannot authorize a global financial endpoint.

Findings. T47-F01 - High [confirmed defect]: the UI cannot express existing region-scoped grants. The creation form at client/src/admin/pages/super/PermissionGrants.jsx:63 includes grantee, permissions, expiry, and notes, but no region ID or region selection. It posts that form at :94; the server defaults absent region to null at server/controllers/adminPermissionController.js:129. Administrators using this screen therefore create global grants even though the API and schema support a region. Add an explicit scope picker with a clearly marked global option, resolved eligible regions, and confirmation of the resulting reach. This is not a missing region API.

T47-F02 - High [code observation]: list caps obscure review completeness. server/controllers/adminPermissionController.js:114 limits grant inspection to 200 rows with no pagination metadata; self-lookup at :55 limits active grants to 50. client/src/admin/pages/super/PermissionGrants.jsx:306 computes active counts from the returned slice. An access reviewer can mistake the visible population for the whole workforce, and the client guard can omit older effective grants while the server still recognizes them. Add complete paginated review and effective-permission computation on the server, preserving fail-closed client behavior.

T47-F03 - High [code observation]: authorization changes and audit are not committed together. Grant insertion at server/controllers/adminPermissionController.js:176 precedes a standalone audit call at :183; revocation at :216 likewise precedes standalone audit at :223. The audit helper's default failure behavior is explicitly best effort (server/services/auditService.js:12 and :60). Add transaction-bound audit and idempotent request receipts, so reviewers never have to infer whether a permission change committed after an error. Use conditional revoke/state checks to identify simultaneous review decisions.

T47-F04 - High [design requirement]: the catalog needs an effective-capability contract and organizational delegation. Some catalog entries promise operations that remain super-admin-only, such as verify/delete user routes at server/routes/admin.js:183 and :185; catalog text at server/controllers/adminPermissionController.js:13 and :14 should not imply that granting those scopes authorizes those routes. Conversely, financial global-only checks are deliberately restrictive. Show which endpoints each scope currently enables, which require extra constraints, and which are reserved. Extend the existing catalog with team/tenant/region constraints, approval thresholds, incompatible-role rules, and recurring access attestation.

Expansion scenario. A global access review includes several grants per employee, temporary incident permissions, and regional holiday coverage. The latest 200 grants are insufficient to find a long-standing high-risk grant. A reviewer must see effective permissions, overlaps, sponsor, expiry, business scope, and the exact operations enabled, then revoke in bounded batches without accidentally elevating every grant to global.

Recommended controls and UX. Add explicit scope, searchable assignees, role templates built from the existing catalog, effective-permission previews, conflict checks, sponsor/reason, and time-zone-aware expiry. Use a dry-run bulk grant/revoke job that previews affected users and reauthorizes each action.

Preserve API expiry validation and super-admin boundaries until constrained delegation is implemented. Replace native confirmation-only review with accessible context showing which grant and reach will be revoked.

Dependencies / owner. IAM owns effective capability and grant transactions; security owns incompatible access and review cadence; department owners sponsor privileges; admin frontend owns scope visibility and complete lists.

Measurable acceptance criteria. Create a regional grant through the UI and verify persisted region plus denied global access. Inspect 20,000 grants with complete totals and locate the oldest active record. An effective-permission endpoint must include all valid grants beyond the current 50-row slice. Inject audit failure and verify atomic rollback. Every catalog scope must map to tested route behavior. Bulk review must return per-user receipts without privilege expansion; translated expiry controls and keyboard dialogs must pass configured authenticated tests.

48. Regions - /admin/super/regions

Mission / what exists. Regions implements geographic territory CRUD, search/type filtering, assignment counts, and boundaries for zip ranges, city/state, radius, country/region, and whole countries. The API validates known country/region values and blocks ordinary deletion while active grants reference a region. Regional matching exists on the server, including a retryable fail-closed response when region lookup fails. The enterprise mission is stable geographic authorization with clear ownership, international identifiers, and change previews; a geographic region must not be confused with a tenant or legal entity.

Evidence. client/src/admin/pages/super/RegionsManager.jsx:266 loads regions, :283 filters client-side, and :322 confirms deletion. server/routes/admin.js:340-:343 protect CRUD with super-admin checks. server/controllers/adminRegionController.js:18 validates boundaries, :54 supports country/region, :95 lists, :155 updates, and :221 deletes. server/middleware/regionAuth.js:91 dispatches matching, :119 loads active grants, :146 marks lookup unavailability, and :163 enforces target-user access. server/__tests__/regionAuthFailClosed.test.js:25 tests lookup failure, while :34 deliberately preserves unrestricted legacy behavior for an empty successful region result.

Findings. T48-F01 - High [code observation]: expiry can remove a geographic restriction instead of denying all scope. Region hydration at server/middleware/regionAuth.js:135 selects only active grants; the target check at :178 allows an empty region list. Where an administrator has another valid global action grant, an expiring region restriction can therefore broaden their geographic reach. This is the explicit legacy contract, not a database lookup failure bypass-the latter was repaired and fails closed at :168. Replace the empty-array interpretation with explicit unrestricted versus restricted/empty authorization states, and bind regions to the permission being exercised rather than collecting every regional grant indiscriminately.

T48-F02 - High [code observation]: deletion safety relies on a nontransactional check and a nulling foreign key. server/controllers/adminRegionController.js:230 counts active references, then :240 deletes separately. server/migrations/115_super_admin_regions.sql:32 makes the grant-region foreign key ON DELETE SET NULL, repeated in migration 119. Null means global in server/middleware/adminAuth.js:145. An interleaved grant creation and region deletion could broaden the new grant if not otherwise blocked. This is a concurrency exposure requiring a real-database test, not a claim that ordinary protected deletions currently succeed. Use retirement, restrictive references, or a transactional locking protocol shared by grant creation/deletion; never convert restricted authority into global through deletion.

T48-F03 - High [code observation]: boundary edits instantly change existing authority without a revision. The update at server/controllers/adminRegionController.js:193 applies a new definition to the same ID, with no expected version or affected-user preview; audit at :198 is best effort. An accidental expansion changes every active grant referencing that ID. Add immutable boundary revisions, an impact preview, independent approval for large expansions, effective dates, and transactional audit. Retain duplicate-name handling and existing boundary validation.

T48-F04 - Medium [code observation]: legacy boundary types are geographically ambiguous. Zip matching at server/middleware/regionAuth.js:24 parses the first five characters as a number and does not bind country; city/state at :35 also compares no country. New country-specific types already exist and should be preferred. Radius validation at server/controllers/adminRegionController.js:46 checks numeric coordinates but not latitude/longitude bounds. Add explicit country/CRS metadata, coordinate ranges, postal normalization, overlap analysis, and address-quality states. These are source observations, not claims about a specific misassigned production user.

Expansion scenario. An employee receives country-specific authority plus a temporary region restriction during a launch. Their assignment expires overnight while another administrator edits a postal territory shared by multiple teams. Support managers need to predict which customers and businesses move into scope, identify overlap, and transfer ownership without changing privileges accidentally.

Recommended controls and UX. Extend the existing region editor with country-qualified definitions, boundary revision history, preview counts/maps, overlap warnings, owner/sponsor, and retirement instead of privilege-changing deletion. Add server-side search/pagination when territory counts grow; current list at server/controllers/adminRegionController.js:97 is unbounded with per-region grant counts. Implement scoped action routes using matching and capability checks together: server/routes/admin.js:325 still

requires a global manage_users grant, so regional-only delegation is not complete. Tenant isolation needs an additional ownership boundary.

Dependencies / owner. IAM owns explicit scope semantics and deletion safety; geography/data owns normalized boundaries; regional operations owns territory ownership; frontend owns previews and accessible editing. Verify current database constraints in a restored isolated rehearsal before changing historical assumptions.

Measurable acceptance criteria. Expire the last scoped assignment while another action grant remains and prove authority does not broaden. Race grant creation/deletion repeatedly in PostgreSQL and prove no scoped grant becomes global. Same-revision edits must conflict. Reject out-of-range coordinates and cross-country postal collisions. Preview affected identities for a million-user territory using bounded work. Geographic and tenant-negative tests must pass independently, including missing-address and lookup-outage cases.

49. Internal Access - /admin/super/logistics-internal-access

Mission / what exists. Internal Access is a real staff access-management interface, listing eligible employees and administrators, showing corporate departments and eligible workspaces, searching/filtering setup state, and editing operations/compliance/admin-portal access. It extends existing internal workspace and corporate assignment models. The underlying gate has hashed access material, active-user eligibility checks, route eligibility, lockout handling, login-history records, and recovery logic. Enterprise readiness requires reliable UUID-based provisioning, accountable access changes, and a scalable joiner/mover/leaver workflow without treating gate enablement as complete business-data authorization.

Evidence. client/src/admin/pages/super/LogisticsInternalAccess.jsx:95 loads records, :137 filters them, :209 updates the selected user's record, and :623 confirms save. server/routes/admin.js:230-:234 require super-admin access for self/list/other-user provisioning. server/controllers/logisticsInternalAccessController.js:84 saves access, :139 checks route eligibility, :281 lists corporate-linked staff, :377 provisions a target user, and :813 implements one-use recovery. server/migrations/276_logistics_internal_access.sql:3 declares the user foreign key UUID and :15 creates case-insensitive name uniqueness; migration 277_logistics_internal_access_admin_portal.sql:1 adds portal eligibility. server/utils/internalWorkspaces.js:63 checks active corporate department assignments. server/__tests__/accessGateRateLimit.test.js:39 covers lockout checks, not this full provisioning flow.

Findings. T49-F01 - High [confirmed defect]: other-user provisioning rejects UUID identities. The client at client/src/admin/pages/super/LogisticsInternalAccess.jsx:209 puts selectedRecord.user_id into the URL. The listing returns u.id at server/controllers/logisticsInternalAccessController.js:285; users are UUIDs (server/migrations/001_initial_schema.sql:122 and 276_logistics_internal_access.sql:3). Yet server/controllers/logisticsInternalAccessController.js:379 converts the path value with Number() and rejects nonpositive/noninteger values at :380. Normal UUID staff IDs therefore return 400 before saveAccessRecord is reached. Self-setup uses a separate handler at :254, so this does not establish that every access flow is broken. Correct and test the shared ID contract before scaling provisioning.

T49-F02 - High [code observation]: administrative access updates lack durable actor/reason evidence in the inspected save path. saveAccessRecord performs the update at :143 and returns updated_by_user_id only in response data at :183. The provisioning handlers at :254 and :377 do not persist an admin audit record or change reason for enablement, disablement, or route changes. Successful gate-login history is a different event. Add transactional access-change evidence with actor, subject, before/after route state, purpose, and operation ID. Keep sensitive access material out of these records.

T49-F03 - High [code observation]: concurrent updates have no expected revision or proven active-session invalidation. The UPDATE at :154 selects only by user ID; there is no revision check. A stale operator can restore route enablement or active state after another shift disables it. The save path does not establish revocation of previously issued sessions/gate tokens. Whether every downstream workspace rechecks eligibility is a verification gap, not a confirmed bypass. Add compare-and-swap state and a tested revocation/version mechanism covering already-issued access, while preserving the ordinary admin MFA gate.

T49-F04 - Medium [code observation]: full staff enumeration and per-user eligibility work need a bounded workflow. The list at :283 joins users, access records, and corporate assignments, has no LIMIT through :336, then derives eligible routes. The client filters all records at client/src/admin/pages/super/LogisticsInternalAccess.jsx:139. With 3,500 employees, this may still function, but has no inspected scoped manager view or durable bulk onboarding. Add indexed server search, bounded pages, department eligibility totals, and previewed asynchronous enrollment/revocation jobs. Reuse corporate assignments; do not create a second employee directory.

Expansion scenario. A warehouse opens with 300 staff; some require operations, others compliance, and finance staff require a corporate workspace. The workforce administrator currently encounters the UUID mismatch when provisioning another user. After that defect is fixed, concurrent access changes and shift handover still need durable receipts, department ownership, and timely revocation. A "ready" badge must describe valid route eligibility, not imply unrestricted tenant/warehouse authority.

Recommended controls and UX. Correct UUID handling, add access revision/reason/audit, sponsor/expiry, department-scoped management, and lifecycle synchronization. Show exact workspace eligibility and downstream data scope separately. Link existing lockouts and recovery without exposing private access material. Provide bulk dry-run, per-row conflicts, and rollback/revoke outcomes for the authorized workforce. Validate names and labels with associated form controls, keyboard confirmation, Arabic layout, and accessible result announcements.

Dependencies / owner. IAM owns provisioning IDs, lifecycle, and revocation; people/corporate operations owns employment and assignment changes; logistics owns warehouse data scope; security owns access-change audit and gate policy; frontend owns complete search and safe bulk UX.

Measurable acceptance criteria. Provision and disable a real-format UUID staff fixture through the actual route and obtain the correct persisted result. Reject invalid IDs before database writes. Same-version concurrent edits must produce one 409. Every route enable/disable change must persist a redacted actor/reason/before/after receipt atomically. Previously issued access must stop authorizing revoked workspaces within 60 seconds in executed tests. Onboard 300 synthetic staff through bounded jobs with complete row outcomes, and prove an assigned manager cannot provision outside their department/warehouse scope.

50. Gate Lockouts - /admin/super/access-gate-lockouts

Mission / what exists. This tab restores employee access after repeated failures at the internal gate while retaining evidence of suspicious attempts. It is an implemented, localized super-admin page, not a missing security feature. Operators see IP/name pairs, failure counts, first-failure time, expiry, relative time remaining, and recent attempts. Refresh, failed-only filtering, error recovery, and a reason-oriented confirmation dialog already exist. The database rate limiter is shared through PostgreSQL; recovery-email and CLI paths also exist. Admin API authentication rechecks active users, token revocation, and configured two-factor authentication, so a recovery proposal should preserve that boundary rather than create a new authentication system.

Evidence. client/src/admin/pages/super/AccessGateLockouts.jsx:22 loads lockouts and 100 attempts; :43 performs the lift. server/routes/admin.js:237 restricts all three endpoints to super-admin. server/controllers/logisticsInternalAccessController.js:663 fixes the lockout list at 200, :673 handles lifts, and :702 accepts attempt filters. server/services/accessGateRateLimitService.js:281 bounds and offsets list queries; :302 records lift actor/reason/source with a conditional update. server/migrations/278_access_gate_rate_limit.sql:34 supplies attempt indexes and :65 a partial unique active-pair index. server/__tests__/accessGateRateLimit.test.js:39 covers lockout checks, including expiry; these are source-inspected tests, not a concurrent production trial.

Findings. T50-F01 - P1 [confirmed defect]: the server permits a reasonless lift. The controller trims the reason but passes reason || null at logisticsInternalAccessController.js:679; the service stores null at accessGateRateLimitService.js:313. A direct request therefore bypasses the reason-oriented UI expectation. This is an accountability mismatch, not proof of unauthorized access: super-admin authorization still applies.

T50-F02 - P2 [code observation]: the visible incident queue is truncated. The controller returns at most 200 active rows with no continuation metadata, while the page has no lockout pagination and loads only 100 recent attempts. Service-level offset support does not make older cases reachable from this tab. A widespread gate incident can hide affected staff behind the newest rows without a visible "more results" warning.

T50-F03 - P1 [design requirement]: worldwide recovery cannot depend on unrestricted super-admin staffing. Both client route and API require that role. Regional help desks have no scoped lockout-review capability in this route chain. Existing global permission middleware intentionally rejects regional grants (server/middleware/adminAuth.js:126); that is protective behavior, not a regional-scope bypass. Safe delegation must identify the staff account and its assignment before authorizing a lift, since a claimed name and IP alone are weak scope identifiers.

Expansion scenario. At an Asia-Pacific shift change, 280 staff behind several shared egress addresses are locked out while a headquarters security analyst is investigating deliberate failures from another region. The first 200 lockouts cannot serve as the entire incident inventory. Several help-desk staff need to recover known employees without viewing every claimed identity or lifting a suspicious pair belonging to a different incident. Concurrent lifts should show "already resolved," preserve the original actor, and avoid encouraging repeated requests.

Recommended controls and UX. Enforce a bounded nonblank reason and incident identifier server-side; expose account resolution confidence and require stronger review where identity cannot be resolved. Add stable cursor pagination, totals or an explicit estimated count, country/workspace filters, absolute UTC plus operator-local timestamps, and a visible truncation indicator. Extend existing audit/recovery paths with an expiring scoped recovery grant, shift owner, escalation timer, and reviewed emergency procedure. Make success and already-lifted responses distinguishable and accessible. Recovery-email delivery should expose provider failure and a tested alternate channel; do not infer successful delivery from a generic acknowledgement.

Dependencies / owner. Security Engineering owns authorization and recovery policy; Identity owns staff/account association; regional service desks own incident response; SRE owns database contention and email-provider rehearsal.

Measurable acceptance criteria. Blank reasons return 400 without updating a row. A regional operator cannot list or lift another region's resolved staff pair through direct API calls. A synthetic 10,000-row queue

remains fully traversable, with p95 list response below one second at 50 simultaneous readers. Fifty concurrent lifts produce one recorded transition. A staffed recovery exercise resolves 95% of verified employee cases within 15 minutes, including keyboard-only operation in all five supported UI languages.

51. Create User - /admin/super/users/create

Mission / what exists. This tab creates a complete account for an administrator-assisted customer or employee onboarding journey. CreateUser delegates to a real shared form and navigates to the resulting user detail. The form supports numerous account types, including operational employees, logistics, business personas, admin, and super-admin; country, region, and postal inputs use existing country helpers. The API validates identity fields and geography, normalizes email and state, maps account type to role/default mode, hashes the submitted password, chooses default currency, and creates helper profiles where needed. User creation and structured auditing share a transaction. Those are substantial foundations that should be extended for workforce onboarding rather than replaced with another user database.

Evidence. client/src/admin/pages/super/CreateUser.jsx:33 mounts CreateUserForm. client/src/components/CreateUserForm.jsx:18 defines account types, :85 constructs the request, and :356 exposes verification toggles. server/routes/admin.js:315 makes creation super-admin-only. server/controllers/superAdminUserController.js:234 validates inputs, :272 checks normalized-email uniqueness, :296 starts transactional creation, :303 writes verification flags, :315 marks onboarding complete, and :341 records the user action. server/migrations/116_admin_user_action_log.sql:10 provides the structured log; server/__tests__/adminAuthSecurity.test.js and server/__tests__/adminPermissionScope.test.js:23 address adjacent admin boundaries, not proof of the full onboarding lifecycle.

Findings. T51-F01 - P1 [code observation]: creation treats completion and verification as administrative assertions. The insert sets onboarding/contact/profile/terms fields true even though the form is not a user acceptance ceremony (superAdminUserController.js:315). Email, identity, and background-check flags are accepted from the body. That can be intentional assisted setup, but the resulting account needs provenance distinguishing an administrative assertion from provider verification and a versioned user acceptance. It should not silently inherit the assurance level of an independently checked employee.

T51-F02 - P1 [design requirement]: bulk workforce onboarding and ordinary provisioning require narrower privileges. Creating one employee currently requires the same global role able to create another super-admin. The route correctly blocks regular admins, yet no bounded HR provisioner workflow is exposed here. At 3,500 staff, forcing every joiner through headquarters increases privileged-account use and delays shift staffing. Existing role mappings and corporate assignments should remain the source of access; a provisioner must not assign arbitrary admin or verification flags.

T51-F03 - P2 [code observation]: there is no request-level recovery contract for assisted creation. The form sends one POST without an idempotency key (CreateUserForm.jsx:101); the controller performs a pre-insert uniqueness lookup and returns raw error detail for failures (superAdminUserController.js:370). A network timeout after commit can leave an operator uncertain whether to retry. Email uniqueness limits duplication but is not an operator-friendly replay/result contract. Also, business_name is collected and destructured but not persisted in the displayed creation insert; the UI should clarify what creating a business persona actually creates.

Expansion scenario. HR must onboard 400 seasonal logistics employees in four countries before Monday, while a support supervisor creates a corrected customer account. Operators need staged imports, row-specific errors, invitations, assignments, and reconciliation against existing emails. They must not distribute administrator-chosen permanent passwords or mark applicants background-checked merely to finish a form. A partially interrupted import needs a resumable result list, not hundreds of manual duplicate lookups.

Recommended controls and UX. Add an invitation-first option to the existing account creation flow, with expiring activation, user-set credentials, and explicit enrollment requirements. Require evidence and separate permissions for assurance overrides and privileged roles; capture who asserted each flag, why, and its source/reference. Use the existing legal acceptance store for later genuine acceptance instead of treating a boolean as equivalent evidence. Add a scoped provisioner grant restricted to allowed employee types, country, department, and business assignment. Implement a dry-run import followed by a bounded durable job using per-row idempotency and downloadable results; show created, invited, pending evidence, and failed states separately. Bind labels to inputs and focus validation summaries so localized field errors are

usable by assistive technology.

Dependencies / owner. Identity Engineering owns creation, invitations, and assurance provenance; HR Operations owns imported workforce records; Security approves privileged-role workflow; Legal Operations reviews acceptance wording and evidence requirements by jurisdiction.

Measurable acceptance criteria. Replaying a creation key returns the original user ID. A scoped provisioner cannot create an admin, set assurance flags, or cross the permitted geography through direct HTTP calls. A 3,500-row synthetic import supports dry run, cancellation, restart, and per-row reconciliation without duplicate users. Every successful assisted account visibly distinguishes administrative completion from user acceptance. Keyboard and screen-reader review reaches every required field and its error in English, Spanish, French, Haitian Creole, and Arabic.

52. Search Logs - /admin/super/search-logs

Mission / what exists. Search Logs supervises privileged discovery of platform records, helping security and operational leads investigate who searched for sensitive users, messages, jobs, or businesses. The tab already offers a 50-row page, query and administrator filters, summary counters, top queries, top searchers, debounced input, abortable loads, retry, and date formatting. Its API escapes wildcard characters in the search-log query, returns totals, and caps results at 200. This is an administrative search audit, not an end-user search analytics dashboard; recommendations should preserve that mission and avoid conflating popularity with legitimate employee access.

Evidence. client/src/admin/pages/super/SearchLogs.jsx:40 establishes filters/paging; :52 retrieves logs; :65 separately retrieves statistics; :78 implements a shared debounce timer. server/routes/admin.js:284 restricts the list to super-admin. server/controllers/adminSearchController.js:338 builds the list query, :349 escapes ILIKE input, :375 counts results, and :395 computes statistics. server/migrations/049_admin_search_and_permissions.sql:7 stores search evidence; :17 adds administrator/time indexes and an English full-text expression index. server/__tests__/adminPaginationContracts.test.js:8 inspects general pagination source contracts; it is not a measured search-log query plan or a specific negative-limit behavioral test.

Findings. T52-F01 - P2 [code observation]: statistics repeatedly aggregate the audit population. The summary scans the entire log for counts/distinct administrators/average results, and top lists group a 30-day window (adminSearchController.js:397). List pagination also performs an exact count. At global workforce search rates, historical events can grow much faster than the six-million-user registry. A full-text index on to_tsvector('english', query) does not directly index the displayed substring ILIKE predicate; a planner/load check is required before declaring any measured bottleneck.

T52-F02 - P2 [confirmed defect]: the filter debounce can lose a field update. Query and admin-ID inputs share one timer. A change to the second field within 400 milliseconds cancels the first pending update; the callback commits only its own key (SearchLogs.jsx:79). The visible inputs can therefore disagree with effective request filters. This matters during an investigation where an operator believes a query is constrained to one employee and collects the wrong evidence.

T52-F03 - P1 [design requirement]: privacy review needs purpose-bound delegation and disclosure controls. The UI/API require unrestricted super-admin access while returning administrator email, raw query text, and IP-related evidence. Query strings can themselves contain personal data. Existing permission machinery is available, including historical view_search_logs terminology, but this route uses a super-admin boundary. Add a specific reviewer capability and scope-aware read path rather than weakening the existing global grant rule or giving every regional auditor a global role.

T52-F04 - P2 [confirmed defect]: negative list limits are not lower-bounded. Math.min(parseInt(limit) || 50, 200) at adminSearchController.js:356 allows a negative value through to SQL. This is a request-validation defect; it does not establish SQL injection or an existing production outage. Clamp or reject invalid values consistently with other admin endpoints.

Expansion scenario. A security analyst investigates repeated lookups of a prominent restaurant owner by staff in two continents over six months. Several supervisors need restricted evidence, while a central analyst needs cross-region correlation. The operator changes both filters quickly and expects an exact evidentiary subset. Whole-table statistics must not compete with service traffic, and a statistical leaderboard must not be interpreted as proof of misconduct.

Recommended controls and UX. Commit both debounced input values as one snapshot, reset continuation consistently, and display effective filters. Add bounded date ranges, stable (created_at,id) cursors, summarized time buckets, and separately refreshed/cached statistics with an "as of" timestamp. Evaluate appropriate substring/full-text indexes with actual representative plans; do not simply add a redundant index. Add purpose, case ID, masking by default, audited sensitive-value reveal, region-aware reviewer grants, retention ownership, and asynchronous evidence export linked to immutable filter metadata. Present top-search statistics as activity context, not accusations. Preserve localized labels and add announced loading/results states.

Dependencies / owner. Security Analytics owns investigation semantics and access; Data Platform owns partitioning/rollups/query plans; Privacy Operations owns disclosure and retention decisions; frontend maintainers own filter consistency.

Measurable acceptance criteria. Rapid edits to both fields produce the combined filter after debounce. Limits -1, zero, malformed, and oversized are handled deterministically without SQL errors. A 100-million-event synthetic log meets p95 list latency under one second for bounded indexed queries and statistics freshness within five minutes. Reviewer scope tests deny cross-region reads and sensitive reveal without permission. Export manifests reconcile event counts and carry the effective filter, actor, time zone, and case identifier.

53. Settings - /admin/super/settings

Mission / what exists. Settings governs global fees, job parameters, system controls, and feature flags. This is a mature implemented tab: grouped settings, search, dirty indicators, reset, individual and save-all actions, direct rollback, flags, proposal/preview/approval/scheduling, and change history exist. The API validates value types and numeric bounds, locks individual settings, increments versions, and commits audited changes transactionally. Rollback uses an audited previous state as a new forward change. The separate change-management path already enforces proposer/reviewer separation. The enterprise requirement is to make these mechanisms consistent and mandatory where risk warrants it, not propose a second settings service.

Evidence. client/src/admin/pages/super/PlatformSettings.jsx:319 defines the change-request API; :995 checks approve_settings; :1101 saves directly; :1172 implements sequential save-all. server/routes/admin.js:253 exposes direct mutation/rollback with manage_settings; server/routes/settingsChanges.js:21 separates proposal from approval permissions. server/controllers/superAdminController.js:2205 locks, validates, versions, and audits direct changes; :2248 restores an audited value; :2311 strictly validates feature flags. server/migrations/591_admin_control_plane_hardening.sql:8 adds bounds/version metadata. server/__tests__/adminControlPlaneHardening.test.js:55 covers transactional audit and :111 rollback; server/__tests__/settingsChangeRequests.test.js:213 covers separation of duties.

Findings. T53-F01 - P1 [code observation]: approval remains optional for direct global controls. A user with manage_settings can directly PUT a setting or flag and roll it back, even though the proposal workflow needs a separate approver. The direct controller does not select a risk class or enforce an approved request. For global fees, identity requirements, maintenance mode, or release-delay policy, a well-intentioned operator can bypass the very review process the tab provides. This is a current authorization-path difference; whether it violates policy depends on the organization's adopted policy.

T53-F02 - P1 [confirmed defect]: a setting version is incremented but not used to reject stale edits. The client submits {value} at PlatformSettings.jsx:1105; the server reads the current row under lock then overwrites it without an expected-version comparison (superAdminController.js:2213). Row locks serialize writes, but two editors can still overwrite each other's decisions. The same concern applies to rollback targets that an operator reviewed before another change occurred. A counter alone is not optimistic concurrency control.

T53-F03 - P2 [code observation]: save-all is a series of independent writes. The loop at PlatformSettings.jsx:1183 can commit some values before another request fails. The UI counts successes, which is preferable to pretending atomicity, but coupled fee and escrow parameters need a reviewed change set with explicit all-or-partial semantics. Restoring a previous setting changes future configuration; it cannot undo already executed settlements, refunds, or contractual commitments.

Expansion scenario. Europe proposes a fee adjustment for a Monday launch while the Americas incident commander changes maintenance behavior and an escrow delay. A stale headquarters browser then clicks save-all. Individual writes may succeed in an unexpected mixture, and later rollback could restore a setting unrelated to the operator's intended incident. Worldwide teams need bounded authority, clear ownership, a scheduled effective instant, and evidence that every consuming worker has observed the approved configuration version.

Recommended controls and UX. Extend existing change requests with risk-class enforcement at every mutation endpoint, including flags and rollback. Permit direct low-risk changes where explicitly allowed, and preserve a tightly audited emergency path with expiry and retrospective review. Send expected versions; return 409 with before/current/proposed diff and preserve the operator's draft. Package coupled settings in one reviewed change set with atomic validation, previewed affected consumers, reason, owner, effective time, and explicit rollback target. Add propagation acknowledgements and stale-cache alarms. Explain fee units, currencies, and future-only rollback effects. A financial recovery plan must reconcile downstream transactions through existing ledger/reconciliation workflows, with finance and jurisdiction review where relevant.

Dependencies / owner. Platform Control Plane owns policy/schema; Finance Operations owns fee and settlement impact; Security owns privileged emergency access; SRE owns worker propagation and incident handoff.

Measurable acceptance criteria. A stale expected version returns 409 and writes neither value nor audit transition. Sensitive direct writes without the required approved change return 403/409 through direct API calls. A multi-key invalid change commits zero values. Approved schedules execute once and every consumer reports the new version within 60 seconds. Rollback previews identify the exact audited target and clearly exclude reversal of already posted financial activity. Simultaneous regional change exercises retain both drafts and a complete actor/reviewer history.

54. Audit Log - /admin/super/audit-log

Mission / what exists. Audit Log provides the general administrative activity record, supporting incident reconstruction, supervision, and evidence collection. It already has action/type filters, 50-row paging, refresh, a last-loaded indicator, error display, and localized copy. The API supports an administrator filter beyond the visible basic controls and requires the global view_audit_log permission for delegated admins. The underlying log is more than an ordinary editable table: migrations implement hashes, predecessor links, immutable updates, and a read-only integrity verifier. A scheduled verification job records a critical incident for broken links. These mechanisms are existing strengths; enterprise recommendations must build on them and distinguish a chain-integrity check from complete event coverage or independent evidence custody.

Evidence. client/src/admin/pages/super/AuditLog.jsx:19 fetches page/filter results and :90 renders filters. server/routes/admin.js:260 grants list access; server/controllers/superAdminController.js:2407 handles missing schema, caps page/limit, and orders by timestamp. server/migrations/406_admin_audit_hash_chain.sql:49 serializes chain inserts through an advisory transaction lock; :79 prevents updates. server/migrations/407_admin_audit_hash_chain_verification.sql:15 scans ordered chain evidence. server/jobs/adminAuditChainVerification.js:32 runs verification and :44 creates an incident. server/__tests__/auditServiceFailClosed.test.js and server/__tests__/ws5-audit-evidence.test.js inspect adjacent audit guarantees; the inspected job explicitly calls external anchoring future work at :15.

Findings. T54-F01 - P2 [code observation]: deep browsing is offset-based and lacks stable tie ordering. The server caps page number at one million but computes offset as page times limit, then orders only by created_at (superAdminController.js:2412). A page-size cap does not bound database work at large offsets. Concurrent activity and equal timestamps can move evidence across pages. The route does not expose a date window, target key, correlation ID, or integrity checkpoint despite storing richer records.

T54-F02 - P1 [design requirement]: sensitive audit access is globally delegated or unavailable. Current permission middleware rejects regional grants, correctly preventing a regional grant from authorizing a global read (server/middleware/adminAuth.js:151). The route therefore cannot support regional auditors with least-privilege evidence access. A UI filter is not an authorization boundary. A region/tenant-aware projection must handle global setting events separately and avoid leaking personal before/after states across business or workforce boundaries.

T54-F03 - P2 [verification gap]: chain throughput and evidence custody have not been demonstrated at target scale. The advisory lock is an intentional consistency mechanism, while verification scans the chain. No load measurement or restored-data rehearsal was performed here. The daily job's own scope note states that a complete recomputed tail requires external anchoring to detect. Successful links also do not prove every administrative writer emitted an event; transactional settings writes coexist with nonblocking audit paths elsewhere, for example user-creation legacy logging at superAdminUserController.js:354.

Expansion scenario. A financial incident spans support, settings, and a country launch across three time zones. The investigator needs the precise sequence of related requests and configuration changes, while local reviewers need only their business scope. Six million user records generate many service events, and 3,500 staff can create bursts of admin actions. Repeated deep paging and a full-chain scan must not stall the same database serving customers. Missing events must be visible as a coverage failure, not confused with valid chain links.

Recommended controls and UX. Add stable (created_at,id) cursors, bounded date filters, target-key and correlation search, saved case views, and explicit source/integrity status. Preserve existing chain verification and incident reporting; measure lock waits and verification duration before deciding on segmentation or incremental checkpoints. Publish reviewed chain heads to independently controlled append-only storage with auditable retrieval. Add a writer coverage catalog defining which actions fail closed, which have durable retry, and who owns unresolved gaps. Produce asynchronous case exports with manifest, count, hashes, filter scope, and last verified checkpoint. Present audit events as evidence, never as a one-click financial undo mechanism. Provide masked details by default and accessible expansion controls.

Dependencies / owner. Security Engineering owns audit guarantees and reviewer access; Data Platform owns query/index/retention design; SRE owns verification health and anchoring custody; Finance owns interpretation of financial activity.

Measurable acceptance criteria. A concurrent-write pagination test visits every event in a fixed snapshot once, including identical timestamps. Regional reviewers cannot request global or foreign-tenant evidence by changing filters. At a synthetic 100-million-event volume, common bounded reads meet p95 under one second and chain writes meet a documented latency budget. A deliberate chain break creates one deduplicated incident; a recomputed tail disagrees with the independent anchor. Every privileged mutation category has a documented writer guarantee and evidence owner.

55. User Action Log - /admin/super/audit-log/user-actions

Mission / what exists. This tab exposes structured per-field changes to user accounts, complementing the general administrative log. It already supports actor, target, action, region, and date filters, paginated reads, CSV download, and before/after detail. The service bounds list requests to 500 rows and offset to one million; CSV uses formula neutralization and non-cacheable response headers. Creation writes structured evidence transactionally, and sensitive field redaction is part of the logging service. This existing system is the correct place to improve account-change traceability, rather than invent a competing history store or assert that account auditing does not exist.

Evidence. client/src/admin/pages/super/AuditLogUserActions.jsx:30 builds filter/offset requests; :76 downloads the CSV. server/routes/admin.js:616 grants both list/export to view_audit_log, with the strict limiter on export. server/controllers/auditController.js:19 maps filters, :48 escapes spreadsheet values, and :57 exports. server/services/auditService.js:180 assembles the query and :203 clamps pagination. server/migrations/116_admin_user_action_log.sql:26 provides action/time/region indexes; server/migrations/124_admin_user_action_log_nullable_ids.sql:24 permits actor/target links to become null during deletion. server/__tests__/auditExportSafety.test.js:22 checks inert CSV values and cache headers; it does not establish complete multi-page exports or deleted-identity visibility.

Findings. T55-F01 - P1 [confirmed defect]: export silently represents only the first 500 matching records. The controller fixes limit:500, offset:0 and writes those rows (auditController.js:66). It discards the total returned by the service. The UI offers "export CSV" without a partial-result manifest. With multiple field changes per account, one onboarding batch can exceed that cap. Bounding a synchronous response is sensible, but a partial evidence file must be labeled as such or completed by a durable job.

T55-F02 - P1 [confirmed defect]: retained events can disappear from the viewer when identity links are null. Migration 124 intentionally permits null actor/target links so evidence survives deletion. The data query still uses inner joins to both user rows (server/services/auditService.js:217), whereas the count query counts log rows alone (:225). Null-linked events remain stored but cannot appear in results, and totals can exceed visible rows. This is a deterministic query/schema mismatch; it is not an assertion that historical data has already been erased.

T55-F03 - P1 [design requirement]: read and export permissions need separate scope and disclosure decisions. The existing route permits export with the same global viewing grant, even though structured old/new values can include contact or assurance information. Regional filtering is supplied by the caller; middleware intentionally only accepts unscoped grants. Safe regional access needs a server-derived record scope and handling for events whose region is null, not reliance on region_id in a query string. Preserving identity snapshots must be balanced with approved minimization/retention policies.

Expansion scenario. A six-month investigation reviews 700 employee account updates and 20,000 customer corrections. Some targets have closed accounts, and a former administrator no longer has an active user record. The analyst expects the displayed count, CSV manifest, and retained history to reconcile. A regional HR reviewer needs local employee changes, while Security needs cross-region provenance. The organization must retain explanatory context without automatically restoring access to deleted accounts or disclosing every personal field to HR.

Recommended controls and UX. Use left joins and a clear "deleted/unavailable actor" representation; store policy-approved immutable identity references so present-day email changes do not rewrite the meaning of historical evidence. Add request/correlation grouping, stable cursor pagination, explicit effective region/business scope, and authorized reveal of sensitive values. Extend the existing CSV implementation with an asynchronous export using the same validated filter and scope as the list, durable progress, replay protection, manifest counts, expiry, and audited download. Either require a dedicated export permission or document deliberate equivalence with sensitive viewing. Show partial results and unavailable notification evidence explicitly rather than treating booleans as delivery proof.

Dependencies / owner. Identity owns account-change contracts; Security owns audit retention and disclosure; Privacy Operations approves identity snapshots; Data Platform owns complete exports and consistent joins.

Measurable acceptance criteria. Synthetic events with null actor, null target, and deleted identities remain visible and reconcile with totals. A 25,000-event filtered export returns all authorized rows exactly once or declares its incomplete state and resumable job ID. CSV formula/caching protections remain intact. Direct API tests prevent regional reviewers from widening scope or downloading unapproved sensitive fields. Replaying an export request returns one job, and cancellation/restart produces a reconcilable manifest without duplicate evidence.

56. Legal (all subpages) - /admin/legal/{acceptances,audit-log,compliance-metrics,ccpa-requests}

Mission / what exists. Legal has four genuinely routed administrative subpages: Acceptances, Legal Audit Log, Compliance Metrics, and CCPA Requests. They have localized copy, a shared subnavigation, filters, charts or metrics, paginated tables, detail panels, revocation controls, CSV actions, and request-status/notification controls. Separately, repository components exist for Documents, New/Edit Document, and Approval: intended paths are /admin/legal/documents, /admin/legal/documents/new, /admin/legal/documents/:id/edit, /admin/legal/documents/:id/approve, and /admin/legal/approvals. They are present in source but are not registered by the current admin manifest/app route declarations. Their backing document API also exists. This distinction avoids incorrectly calling all Legal functionality missing while still identifying incomplete publication workflow wiring.

Evidence. client/src/admin/components/LegalSubNav.jsx:5 lists the four live tab paths; client/src/admin/AdminApp.jsx:292 mounts them. client/src/admin/pages/AdminLegalAcceptancesPage.jsx:183 reads acceptance results and :222 exports; client/src/admin/pages/AdminLegalAuditLogPage.jsx:115 reads audit results and :147 exports; AdminCCPARequestsPage.jsx:296 reads requests; AdminComplianceMetricsPage.jsx:235 renders document-status counts. client/src/pages/admin/AdminLegalDocumentsPage.jsx:28 loads documents; AdminLegalDocumentEditorPage.jsx:59 loads a document and :126 requests approval; AdminLegalApprovalPage.jsx:31 creates fixture approvals. server/routes/legalDocuments.js:13 defines process-local drafts, :128 begins document administration, :253 begins acceptance administration, and :325 routes privacy requests. server/controllers/legalDocumentsController.js:198, :307, :383, and :488 implement acceptance, metrics, audit, and revocation respectively. Migration 588_legal_audit_log_reconcile.sql:38 repairs writer/reader columns; :65 drops incompatible historical constraints. server/__tests__/legalAuditLogContract.test.js:94 covers that reconciliation. Those repaired column/taxonomy issues are not repeated here as current defects.

Findings. T56-F01 - P1 [confirmed defect]: current acceptance, audit, and request count queries still bind the wrong parameter arrays. Fresh source inspection found params.slice(0, paramCount - 2) at legalDocumentsController.js:265, :449, and ccpaController.js:222. After N filters, paramCount=N+1 and pagination adds two more values. The count query requires N values, while the slice yields N−1 for filtered requests; without filters, it supplies the limit to a query with no placeholders. A read-only in-memory harness extracted the actual three functions and replaced only database access with placeholder-arity checking. All three failed for both unfiltered and one-filter cases: required/supplied counts 0/1 and 1/0. No real database or production request was used. This is newly verified current code evidence, not a copied historical SQL allegation. CI success and readiness do not cover it away.

T56-F02 - P1 [confirmed defect]: list and UI response contracts disagree even after binding is corrected. Controllers return pagination.total for acceptance and audit (legalDocumentsController.js:295, :477) and for requests (ccpaController.js:262), while the pages read res.data.total (AdminLegalAcceptancesPage.jsx:196, client/src/admin/pages/AdminLegalAuditLogPage.jsx:131, AdminCCPARequestsPage.jsx:310). Acceptance rows also return camelCase user/document fields (legalDocumentsController.js:282) while visible components reference fields such as document_version (AdminLegalAcceptancesPage.jsx:465). Normalize the complete schema through a tested adapter; do not fix only a count and assume all pagination/detail fields align.

T56-F03 - P1 [confirmed defect]: export filters and completeness do not match the reviewed screen. Acceptance and audit clients send active filters, but the corresponding handlers run fixed unfiltered queries with LIMIT 5000 (legalDocuments.js:264, :304). A user can receive evidence outside the intended case and lose everything beyond the cap. Existing export_data authorization is valuable, but it does not make an unfiltered file the same evidence set. The shared CSV helper at legalDocuments.js:85 quotes values without the formula-prefix neutralization already implemented by User Action Log. Extend that established export safety rather than treating all CSV implementations as equivalent.

T56-F04 - P1 [confirmed defect]: document publication is not durable or connected to public content. Document mutations change a Map at legalDocuments.js:148, :196, :208, and :222. Restart or another API process sees different drafts. Approval only sets a status; it does not retain a scheduled instant,

enforce an independent reviewer, or publish a database-backed immutable version. Public terms load bundled locale content at :26, so marking a draft "live" does not update that public source. The approval component's fixture list is a separate code observation reinforcing that this authoring workflow is unfinished, not evidence that all Legal pages are fixtures. Existing document/version migrations should be evaluated for durable integration before creating parallel stores.

T56-F05 - P2 [confirmed defect]: metrics can misstate assurance and publication coverage. The denominator counts active nondeleted users, while acceptance numerators count distinct users without matching active/deleted eligibility or deleted_at on acceptances (legalDocumentsController.js:309). Revoked records are soft-deleted at :512; numerator/list queries still omit that exclusion. Document-status metrics are literal zeros at :359. The dashboard therefore needs an "unavailable/unconnected" state rather than suggesting a measured zero, and rates need a consistent eligible population and applicable version/jurisdiction. This is an engineering metric defect, not a legal conclusion about whether a specific agreement remains enforceable.

T56-F06 - P1 [code observation]: privacy-request completion needs stronger workflow evidence. CCPA request updates can set completed and fulfillment metadata in an independent UPDATE (ccpaController.js:306, :349) followed by auditing/notification. There is no expected-version comparison in that path, and optional dataProvided is not a verified fulfillment manifest. Existing deadlines and notification actions should be preserved, but expanded into configurable jurisdiction-reviewed routing, responsible owner, identity assurance, retention/legal-hold review, fulfillment evidence, and delivery retry. No statutory deadline or erasure obligation is asserted by this audit.

Expansion scenario. A multilingual agreement update targets one service in two countries while several hundred access/deletion requests and a revoked acceptance are under review. Legal reviewers need authoritative documents, language-specific approvals, exact acceptance versions, clear eligibility denominators, and exports limited to a case. Privacy staff must hand work across time zones without two people marking different fulfillment outcomes. A restart during approval cannot discard a negotiated text or cause a later publish operation to apply another draft. A restored production-like database must preserve earlier versions and all evidence links.

Recommended controls and UX. First repair current count binding and define a single tested client/API schema for every routed subpage. Apply screen filters and authorization scope to exports, show explicit row limits, and use durable asynchronous jobs for complete results. Reuse inert CSV encoding and no-store headers. Wire document components only when their actual durable workflow is ready; migrate drafts into the existing document/version model, preserve immutable content/language hashes, record proposer/reviewer/version, and publish exactly that approved snapshot through a durable scheduled worker. Require server-side version conflicts, independent approval where organizational policy demands it, visible diffs, rollback-as-new-version, and public-content confirmation. Update acceptance recording to verify the version actually presented, rather than merely accepting a caller-supplied positive number (legalDocumentsController.js:26). Keep provenance for administrative revocations. Extend request handling with assignment, configurable reviewed clocks, escalation, evidence manifests, retryable delivery, and explicit cross-tenant restrictions. Financial-policy text, provider-specific terms, and jurisdiction applicability require human legal/provider review.

Dependencies / owner. Legal Operations owns approved wording, locale/jurisdiction applicability, publication policy, and evidence standards; Privacy Operations owns request fulfillment; Identity owns authenticated acceptance; Platform/Data Engineering owns durable versions, adapters, and jobs; Finance/provider owners review financial-policy dependencies.

Measurable acceptance criteria. Each of the four routed pages returns correct totals, fields, and page continuation in real isolated PostgreSQL tests with zero, one, and multiple filters. Acceptance/audit exports match filtered authorized counts for 50,000 synthetic rows and neutralize formula values. Restarting or load balancing preserves drafts and reviewed content. Concurrent edits return 409; self-approval is rejected where mandated; scheduled publication makes the exact approved locale/version publicly retrievable. Revoked/ineligible records do not inflate an applicable-version metric. Privacy requests cannot reach completed without the required reviewed fulfillment evidence and retain one outcome under concurrent updates. Run the full authenticated matrix without skip plus a production-like restored-data rehearsal; successful fresh-schema migrations alone are insufficient acceptance.

57. Visitors - /admin/super/visitors

Mission / what exists. Visitors is a real first-party traffic analytics surface, with date range, country, path, device, bot inclusion, and submitted search filters. It shows paginated visit history plus totals, country distribution, popular pages, and daily trends. Country names use locale-aware display names. Recording hashes IPs, strips URL query strings, respects Do Not Track, classifies devices/bots, and tolerates analytics insert failure without blocking the visitor. An existing pruning function supplies retention. The administrative grant is enforced on both the route manifest and analytics API. These privacy and availability measures should be retained while improving analytical accuracy and operating cost.

Evidence. client/src/admin/pages/VisitorHistoryPage.jsx:24 establishes controls, :39 localizes country names, and :70 retrieves both list and summary for every parameter change. server/routes/siteVisits.js:14 requires view_visitor_analytics. server/controllers/siteVisitController.js:31 parses date ranges, :72 performs list/count, and :115 computes four summary queries. server/services/siteVisitService.js:77 handles DNT and hashed-IP recording; :114 deletes old rows. server/migrations/613_site_visits.sql:25 provides time/country/path/user indexes. server/__tests__/siteVisits.test.js:68 checks sanitized paths, :92 DNT, :106 pruning, and :141 filtered pagination. These mocked/helper tests do not establish target-scale analytics latency or consent adequacy in every jurisdiction.

Findings. T57-F01 - P2 [code observation]: pagination repeats aggregate work unnecessarily. VisitorHistoryPage.jsx:75 fetches summary whenever page changes, though it omits page/limit from the summary request. The backend repeats count, distinct-IP, country, top-path, and daily groupings across the same time range. At millions of users and multiple views per visit, this can dominate a relatively cheap table page. No measured outage or saturated query is claimed; query plans and load need evidence.

T57-F02 - P2 [code observation]: input bounds permit expensive historical queries and unstable continuation. parseRange substitutes defaults for invalid dates but does not limit a valid wide range or reject reversed dates. The list uses timestamp-only ordering and offset paging (siteVisitController.js:103), while wildcard path/search predicates can broaden the requested match. Long ranges and insertion during browsing can produce costly, moving result sets despite a bounded page size. A browser's 30-second timeout is not a database budget or cancellation strategy.

T57-F03 - P1 [design requirement]: aggregate analytics and identifiable visit investigation require separate access. The list joins registered-user name/email (siteVisitController.js:97). The current grant is global; a selected country filter does not limit the operator's actual authorization. Country analysts usually need aggregates, while trust/support staff need a purpose-bound individual lookup. Hashed-IP distinct counts are estimates of network identifiers, not six million distinct people: shared networks, address rotation, DNT, and bot heuristics affect interpretation.

T57-F04 - P2 [verification gap]: ingestion loss and retention execution need observable evidence. Recording catches and suppresses insert failures (siteVisitService.js:107), an appropriate availability choice that can hide lost analytics unless a metric records them. The pruning helper exists, but this audit did not execute or verify its runtime schedule/retention outcome. Its one-statement DELETE also needs a volume/lock-budget rehearsal; the existence of pruning is not proof of cheap retention at worldwide traffic levels.

Expansion scenario. Marketing compares a restaurant campaign in Haiti with a logistics launch in Canada while support examines one customer's failed journey. A single office address can correspond to hundreds of people, and a traffic spike can create tens of millions of pageviews. Country analysts must not browse unrelated user identities, and next-page clicks should not rerun every aggregate. Regional day boundaries and translation differences must not create apparently contradictory daily charts.

Recommended controls and UX. Separate list and summary dependencies, cache/roll up summaries by validated filter snapshot, expose freshness/sample/loss metadata, and label unique-network estimates honestly. Reject or queue expensive date windows, use stable cursors and documented time zones, and limit wildcard search cost. Provide aggregate-only grants with country/business scope; require audited case-based permission for identifiable histories. Instrument received/recorded/dropped counts, pruning lag, query duration, and bot classification coverage. Rehearse bounded batch retention or time partitions with the existing retention policy. Show partial summary failure independently from a usable list, and provide

accessible chart alternatives and localized range descriptions.

Dependencies / owner. Data/Analytics Engineering owns definitions and rollups; Privacy/Security own identifiable access and retention policy; SRE owns ingestion and pruning health; regional Marketing owns interpretations and campaign context.

Measurable acceptance criteria. Twenty page changes issue one summary fetch for unchanged filters. A 100-million-view synthetic dataset meets p95 page latency below one second and summary freshness within five minutes. Reversed/oversized ranges return a clear validation or job response. Aggregate-only regional users cannot obtain names/emails through direct API requests. Injected recording failures increment loss metrics without blocking page navigation. Retention rehearsals stay within agreed lock/latency budgets, and charts have keyboard/screen-reader summaries in all supported locales.

58. Profile Visibility - /admin/super/profile-visibility

Mission / what exists. Profile Visibility is an extensive moderation control surface, not a missing page. It searches several profile types, respects owner listing/activity in effective visibility, supports pagination, public preview, active-work inspection, reasoned hide/restore, history, undo, saved views, export, bulk dry run, idempotency keys, two-person approvals, scheduled changes with time zones, and owner-notification status/retry. The server gates single-profile hides with active work and requires approval for bulk requests above ten items. Existing scheduled-action and notification workers should remain the execution paths. Enterprise work concerns consistency among these paths, atomic approval execution, complete evidence, and safe distributed operation.

Evidence. client/src/admin/pages/ProfileVisibilityPage.jsx:131 defines the workspace, :224 searches, :317 applies a change, and :348 begins active-work review. server/routes/admin.js:103 protects search/export/scheduling/bulk/approval/history operations with manage_profile_visibility. server/controllers/profileVisibilityController.js:364 gates single hides; :545 implements bulk; :683 undo; :770 scheduling; :922 export. server/services/profileVisibilityApprovals.js:253 reviews requests. Migrations 617_profile_visibility_scheduled_actions.sql and 618_profile_visibility_approvals.sql:10 persist schedules and approvals. server/__tests__/profileVisibilityWorkstreamB.test.js:68, :121, and :158 cover self-review, payload fingerprints, and consumption with mocked queries, not simultaneous PostgreSQL execution.

Findings. T58-F01 - P1 [confirmed defect]: approval review is not one database transaction. profileVisibilityApprovals.js:257 uses db.query('SELECT ... FOR UPDATE'), followed by a separate unconditional UPDATE at :279, without acquiring a client/BEGIN across those calls. The lock ends with the standalone statement. Two reviewers can both observe pending and overwrite one another; approval-triggered execution can also happen twice. Existing self-review and fingerprint checks are real, but they do not remove this race.

T58-F02 - P1 [confirmed defect]: small bulk hides bypass the single-profile active-work gate. Single hides call getActiveWork and require approval when work exists (profileVisibilityController.js:375). Bulk execution requires approval only when item count exceeds ten (:598), then calls applyVisibilityChange directly (:620). That helper performs the UPDATE without an active-work gate (:310). Dry run displays active work, but callers can skip preview and submit one to ten items. The same target can therefore require approval through one endpoint and succeed without it through another.

T58-F03 - P1 [confirmed defect]: retry storage follows effects, and audit before-state is synthesized. Bulk checks for an existing key, executes rows, and only afterward inserts the replay response (:588, :613, :658). Concurrent same-key requests can both perform changes before converging on a response. applyVisibilityChange records before.is_hidden_by_admin as the inverse of the desired state (:337), not the loaded state. A no-op repeated hide can produce misleading history, which undo subsequently relies on. These are current source defects, not proof of a past incident.

T58-F04 - P2 [code observation]: scale exports and delegation remain bounded global operations. Export sends only the first 5,000 rows (:948) without a continuation manifest, while scheduled list returns 500 (:861). The API's global grant protects scope, but there is no region/tenant-derived authorization on these routes. A broader workforce must not receive a global moderation grant merely to manage local profiles.

Expansion scenario. A regional incident requires hiding 800 compromised profiles, including restaurants with open orders, while another team restores wrongly flagged businesses. The operator needs previewed impact, independent approval, per-item status, and notification recovery. Duplicate request delivery or a second reviewer must not create duplicate execution or fabricated before-states. Undo must not reopen a business whose later independent restriction remains valid. Existing bulk, approvals, and schedules are valuable but need one consistent state machine.

Recommended controls and UX. Claim approval and idempotency keys before effects inside a transaction or durable job; enforce status/version transitions, payload binding, requester/reviewer scope, and expiry. Apply active-work rules to every hide path, including small batches, scheduled execution, and undo that re-hides. Capture actual locked before-state and retain the originating change ID. Use conditional undo against the exact version. Extend existing workers for larger async batches, cancellation, per-item

retries, and notification reconciliation. Add scoped moderation grants, meaningful totals, complete asynchronous exports, owner escalation, and accessible previews/diffs with localized time-zone explanations.

Dependencies / owner. Trust & Safety owns risk and exception policy; Platform owns atomic workflow; Business Operations owns active-work handoff; SRE owns jobs/notification health; Security owns delegated scope.

Measurable acceptance criteria. Fifty concurrent reviewers produce one final decision and one execution. Same-key retries cause no repeated transitions or notifications. One-item bulk hides of active-work profiles require the same approval as single hides. Repeated no-op changes do not invent an opposite before-state. An 800-profile job resumes after worker loss, and a 25,000-row export reconciles fully. Cross-region HTTP requests fail, while keyboard users can review impact, approve, schedule, and inspect failures in all supported languages.

59. One Truth - /admin/super/one-truth

Mission / what exists. One Truth consolidates platform statistics, scoped user exploration, potential duplicate pairs, and a detailed user view. Country and configured region filters exist, with explicit warnings for geography fields that cannot fully represent a boundary. User and suspect panels have visible pagination; the server supplies scope metadata. The page is read-only and super-admin-only, which avoids turning a duplicate heuristic into automatic destructive identity management. Existing cross-domain summaries and enrichment are useful for investigation. Enterprise expansion requires a scalable read model, reliable scope behavior, currency-aware metrics, and a formal adjudication handoff rather than an additional "truth" database competing with business records.

Evidence. client/src/admin/pages/super/OneTruthPage.jsx:463 builds scope, :479 loads statistics, :493 loads suspect pages, and :515 loads users. server/routes/oneTruth.js:6 requires super-admin throughout. server/controllers/oneTruthController.js:98 resolves region definitions, :254 emits partial-geography warnings, :606 lists users, :674 generates suspects, and :729 computes statistics. :693 selects all scoped candidate users; :702 enriches and generates pairs; :711 slices after computation. :916 aggregates marketplace cents; :966 aggregates payments. Region definitions come from existing admin_regions migrations 115_super_admin_regions.sql / 119_super_admin_regions.sql, and grant-scope regression tests at server/__tests__/adminPermissionScope.test.js:23 protect adjacent delegated endpoints, not One Truth's query load or domain-metric semantics.

Findings. T59-F01 - P1 [confirmed defect]: suspect pagination does not bound processing. Every suspect request loads all users in scope, enriches them, constructs duplicate pairs, then slices the result in application memory (oneTruthController.js:693). Statistics separately selects the full scoped user population to count pairs (:983). Even requesting twenty visible rows can therefore process millions of records. This is an unbounded source path, not a measured memory crash; algorithmic pair complexity depends on the actual grouping logic and is not asserted as universally quadratic.

T59-F02 - P1 [confirmed defect]: an invalid requested region falls back to unscoped results. getRegionScope returns a warning and null definition when a region is not found (:106). That is not an authorization bypass for this super-admin-only route, but it is a dangerous analytical scope widening. An operator expecting a regional incident can receive global records. Partial geography mappings also warrant a deliberate coverage label, rather than a subtle warning alongside apparently definitive totals.

T59-F03 - P1 [code observation]: financial volumes are aggregated without currency grouping. Marketplace total_cents and payment amount are summed for the selected scope (:918, :969) with no currency group or conversion provenance. A global or multicountry scope can therefore produce a number whose unit is not established by this query. This is a metric-design limitation; it does not prove ledger corruption or that the current data necessarily contains mixed currencies. Currency-safe display and test fixtures are required before management relies on the total.

T59-F04 - P2 [verification gap]: freshness, adjudication, and representative query plans lack acceptance evidence. The inspected route contract exposes filters/stats/users/suspects/detail, not case ownership, dismissal disposition, or adjudicated identity links. No six-million-user or restored-data rehearsal was executed. The CI matrix skip also leaves interactive evidence, localized warnings, and stale-response handling unproven despite existing UI implementation.

Expansion scenario. A fraud team reviews suspected duplicates after expansion into markets with shared surnames, recycled phone numbers, family devices, and multiple legitimate businesses per person. A regional definition is retired while an analyst has it selected. Headquarters also requests a global paid-volume figure. The workspace must neither silently widen the investigator's scope nor imply that an unmatched heuristic proves fraud, and its read cost must remain bounded during customer traffic.

Recommended controls and UX. Generate bounded candidate sets using indexed identifiers and incremental background processing, store versioned candidate evidence, and paginate persisted cases. Reuse existing user/business records and incident/case tooling for assignment, triage, false-positive disposition, and reversible adjudication; never merge or move funds solely from a score. Reject missing regions or require explicit global confirmation. Display boundary coverage and per-section snapshot/freshness. Group money by currency and precision, or show a separately labeled reporting

conversion with rate/source/as-of metadata. Add purpose-bound analyst delegation only after server-enforced scope exists; keep sensitive detail separate from aggregate access. Present accessible localized evidence explanations and review checklists.

Dependencies / owner. Identity/Trust owns duplicate adjudication; Data Platform owns incremental candidates and metrics; Finance validates currency semantics; Security owns investigator disclosure and scoped access.

Measurable acceptance criteria. A twenty-case page reads a bounded candidate batch and meets p95 under one second against six million synthetic users. Invalid region IDs return 400/404 and never global results. Mixed-currency fixtures produce separate totals and no unlabeled summed money value. Every candidate records evidence version, reviewer, disposition, and case owner. False-positive decisions preserve legitimate accounts and financial isolation. A production-like restored dataset and unskipped authenticated browser matrix verify warning visibility, pagination, and localized detail.

60. Franchise HQ - /admin/super/franchises

Mission / what exists. Franchise HQ implements an application workspace with status/country/search filters, selected-application detail, notes, owner assignment, checklist progress, agreement terms, training/launch status, activity history, and store activation. The server has bounded paginated application listing, transactions for several writes, activity records, and a row lock preventing duplicate business linkage during activation. Agreement fields include currency-related initial fee, royalty/marketing basis points, term, exclusivity, document reference, and signer metadata. This is a functioning franchise workflow foundation; enterprise enhancements should strengthen its gates, ownership, and evidence rather than claim that agreements or launch tools are absent.

Evidence. client/src/admin/pages/super/FranchiseHQPage.jsx:197 derives metrics from loaded applications; :205 loads the list; :256 initializes agreement fields; :302 saves agreements. client/src/admin/AdminApp.jsx:431 keeps the page super-admin-only. server/routes/franchise.js:28 grants application, notes, checklist, agreement, and activation APIs to admins with global manage_operations. server/controllers/franchiseController.js:363 defaults listing to twenty rows and :459 returns pagination metadata; :770 upserts agreements; :876 activates a store. server/migrations/247_franchise_programs.sql:122, :143, and :176 store checklist, agreement, and activity records. No dedicated franchise lifecycle/concurrency test surfaced in the inspected test inventory; that is a verification gap, not proof that all related behavior is untested.

Findings. T60-F01 - P1 [confirmed defect]: the page ignores existing pagination and labels page-local counts as workspace metrics. FranchiseHQPage.jsx:213 requests without page/limit and reads only data; its metrics count that returned array (:198). The server returns twenty rows by default plus total/pages. No page continuation is consumed in this loader. A 21st application can therefore be inaccessible through the default visible list, and approval/launched totals are incomplete even though backend paging exists.

T60-F02 - P1 [code observation]: UI and API authority differ materially. Client entry is super-admin-only, but all franchise management endpoints accept the broad manage_operations grant (franchise.js:28). Hiding the page does not restrict direct agreement or activation requests. Current global-only grant semantics protect regional isolation by refusal; they do not create dedicated franchise/legal/launch separation. Decide whether broad operations authority is intentional, then align discoverability and server capabilities instead of granting super-admin to every regional franchise team.

T60-F03 - P1 [confirmed defect]: activation does not enforce represented readiness prerequisites. The controller locks the application and checks existing linkage and owner (franchiseController.js:887), then creates a business. It does not check approved application status, agreement completion, training, or checklist readiness before insertion. Those statuses exist elsewhere in the workspace. A manual direct call can activate a store without the administrative gates its record suggests. This is an engineering workflow defect; no assertion is made about contractual or jurisdiction-specific eligibility.

T60-F04 - P2 [confirmed defect]: agreement initial-fee editing loses fractional units. FranchiseHQPage.jsx:258 converts stored cents to a fixed-zero-decimal string; saving uses parseMoney into cents (franchiseController.js:774). A 123.45 amount can become 123 on an otherwise routine save. More generally, agreement upsert lacks an expected-version contract and stores signer metadata without demonstrated provider-signature verification. Metadata should be labeled as recorded information until evidence is reviewed.

Expansion scenario. Teams launch 60 franchises across currencies and time zones. Legal reviewers amend a fee while operations marks training complete and another manager activates the same store. The default twenty-row view hides the remaining pipeline. Fractional fees must survive round trips, and a signed-document reference must point to the exact reviewed terms. Territorial exclusivity and provider acceptance require explicit human evaluation, not inference from a checkbox.

Recommended controls and UX. Consume backend pagination, show authoritative aggregate counts separately, and add queue ownership, SLA aging, localized status labels, and accessible detail forms. Introduce dedicated scoped franchise read/manage/agreement/launch capabilities using existing grants and

record geography/business ownership. Enforce an adopted readiness state machine server-side, with reviewed exceptions, versioned agreement evidence, expected-version edits, and idempotent activation result replay. Preserve currency precision and validate rates/amount bounds. Integrate existing checklist/activity records with provider signature evidence and launch approvals; add reconciliation for fee collection and refunds through existing payment/ledger systems, without creating a parallel financial ledger.

Dependencies / owner. Franchise Operations owns pipeline/SLA and checklist; Legal owns agreements and jurisdiction review; Finance/provider teams own collection/royalty semantics; Platform owns state transitions and scope.

Measurable acceptance criteria. A 10,000-application fixture is fully navigable with accurate filtered totals. Direct activation without required reviewed prerequisites fails before business creation. Concurrent activations return one business ID. A 123.45 fee survives load/save unchanged, with correct currency precision. Stale agreement edits return 409. Regional grants cannot alter another territory or agreement; every approved exception and signature reference identifies the reviewed version and accountable owner.

61. Careers - /admin/super/careers

Mission / what exists. Careers is implemented through the shared HiringPipelineBoard in platform-employer mode. The underlying system supports public openings, international country/language/compensation fields, authenticated employer management, applications, staged pipeline, interviews, offers, applicant decisions, and business-team membership on hire. The board can post openings, select a role, move applicants, book interviews, create offers, and close an opening when hiring. Server validation includes catalog-driven stages and transitions; business hiring uses existing business edit permission checks. These are existing applicant-tracking capabilities, not an empty careers page. Enterprise work should connect the platform hiring lifecycle to controlled workforce provisioning and improve security, queue handling, and recovery.

Evidence. client/src/admin/pages/super/CareersAdmin.jsx:6 mounts platform hiring. client/src/components/careers/HiringPipelineBoard.jsx:43 stores openings/applications; :131 moves stage; :143 prompts for interview time; :172 groups active stages. server/routes/careers.js:17 exposes employer routes using ordinary authentication. server/controllers/careerController.js:35 defines platform admins, :109 authorizes openings, :317 lists employer openings, :490 lists applications, :535 changes stage, and :697 processes offer decisions. server/migrations/531_career_hiring.sql:15 defines openings, :71 applications, and :138 offers. server/__tests__/careerHiring.test.js:172 covers platform posting, :201 rejects helper posting, and :208 checks business-team conversion with mocked database calls.

Findings. T61-F01 - P1 [confirmed defect]: platform hiring API access bypasses the admin control-plane gate. The page requires super-admin, but employer routes use authenticate only and platform authorization accepts any admin or super_admin role (careerController.js:35, :111). No specific hiring grant or requireAdmin two-factor enforcement appears in this route chain. Ordinary authentication rejects provisional MFA tokens (server/middleware/auth.js:65), checks active users (:134), and checks applicable identity assurance (:162), but does not perform the configured-admin-2FA enrollment check in server/middleware/adminAuth.js:29. It is not a public-access defect: authentication still exists and business ownership checks remain. It is a direct-API least-privilege/assurance mismatch that matters for applicant personal data and salary offers.

T61-F02 - P1 [code observation]: employer lists and nested application histories are unbounded. Platform opening queries have no LIMIT (careerController.js:338), and each selected opening returns every matching application plus JSON aggregates of events/interviews/offers (:502). The board groups the entire returned array. A public cap of 100 openings does not solve employer-side access to a popular opening with tens of thousands of applications. Query/index coverage and response-size targets need measurement.

T61-F03 - P1 [confirmed defect]: stage and accepted-offer effects are not transactional workflow transitions. Stage validation reads an application, then unconditionally updates by ID (:537, :547) before independently recording an event and possibly adding membership/closing the opening. Concurrent edits can pass the same old-state validation. Offer decision conditionally claims pending, which is good, but later application/event/member writes occur separately (:720). A failure after acceptance leaves an accepted offer whose downstream hire cannot be retried through the pending-only path without explicit repair tooling.

T61-F04 - P2 [design requirement]: global recruiting needs ownership and reliable scheduling/provisioning handoff. A prompt for interview time is not an enterprise calendar workflow, and platform hires do not use the business-team conversion branch. The reviewed code does not demonstrate a controlled HR provisioning handoff, interview assignment, regional recruiter scope, or provider invitation delivery. These are expansion requirements, not a claim that applicant offers or interview records do not exist.

Expansion scenario. An international logistics opening attracts 40,000 applicants. Recruiters in three countries handle local queues, interviewing candidates across daylight-saving boundaries. Two managers move the same applicant toward rejection and offer, while an offer acceptance succeeds but membership/provisioning fails. The system needs an authoritative outcome, a recoverable handoff, and region-specific disclosure without turning every recruiter into a super-admin or exposing all compensation data.

Recommended controls and UX. Route platform hiring through existing admin assurance plus explicit scoped hiring/read/offer permissions; retain business ownership checks for business employers. Page openings/applicants, lazy-load histories, offer server-side stage/search/age filters, and use bounded asynchronous bulk actions with per-applicant results. Make stage/offer decisions conditional on version/status with transactional event recording and durable outbox tasks for notifications/provisioning. Add hiring manager/recruiter ownership, aging/SLA, interview time-zone confirmation, calendar-provider status, accommodations workflow, and role-approved employee invitation handoff. Review compensation, work authorization, applicant retention, and local hiring requirements with HR/legal; do not infer legal suitability from catalog fields.

Dependencies / owner. HR Talent Acquisition owns recruiting policy and candidate communications; Identity owns workforce handoff; Security owns scope/assurance; Platform owns transaction/outbox repair; calendar/email owners own provider delivery.

Measurable acceptance criteria. An authenticated admin without hiring capability or required assurance receives 403 on direct platform endpoints. Cross-region recruiters cannot read foreign applicants/offers. A 40,000-applicant opening returns bounded pages below one-second p95 and histories only on demand. Concurrent decisions yield one valid stage/event sequence. Failure after offer acceptance resumes exactly once without duplicate membership or notifications. Time-zone/DST scheduling and keyboard-only applicant review pass in all supported UI languages, with provider failure visibly actionable.

62. Country Packs - /admin/super/country-packs

Mission / what exists. Country Packs is the implemented country registry and availability workspace. Operators can see active, coming-soon, and disabled packs, currencies, locales, persona counts, payment methods, expandable configuration, and status actions with confirmation. The public API supplies country selection and country configuration; admin reads and writes require super-admin. PostgreSQL stores the registry and JSON configuration, with status constraints and updated-at maintenance. The service caches registry/configuration for sixty seconds and invalidates its own cache after updates. This is a useful existing architecture for market configuration. Enterprise readiness concerns launch assurance, validation, propagation, and emergency withdrawal, not creating an additional registry or asserting that country support is absent.

Evidence. client/src/admin/pages/super/CountryPacks.jsx:29 loads packs, :45 confirms and changes status, and :145 displays currency/locales/payment methods. server/routes/countryConfig.js:13 exposes public reads and :15 protects administration. server/controllers/countryPackController.js:68 patches via the service and logs the actor. server/services/countryPackService.js:14 defines the local cache, :87 reads a pack, :122 serves fallback on errors, and :160 accepts updates. server/migrations/304_country_packs.sql:10 defines schema and :26 maintains updated timestamps; 366_country_pack_launch_payment_methods.sql records later payment-method alignment. The inspected test inventory did not establish a dedicated country activation/provider/replica-consistency suite. Fresh migration success is credited globally, but no production-like restore or provider acceptance was demonstrated.

Findings. T62-F01 - P1 [code observation]: activation is a status update without an enforced readiness package. The UI confirms a status change and sends only {status} (CountryPacks.jsx:51). The service validates the status enum but does not require approved jurisdiction review, provider capability, localized required documents, support ownership, or reconciliation readiness before making it active. This is an expansion governance gap, not a claim that every current active country is unapproved or that its providers are down.

T62-F02 - P1 [confirmed defect]: arbitrary object configuration can override canonical public metadata. Updates only require config to be an object (countryPackService.js:188), while public response construction spreads ...row.config after canonical code, currency, locale, and updated time (:111). A supplied config containing these keys can replace the registry identity shown to clients. No hostile write is asserted; admin-only scope remains. Schema validation and reserved-key protection must also catch unknown payment methods, unsupported locales, and incompatible currency precision.

T62-F03 - P1 [code observation]: failure and replication semantics can diverge from the intended registry state. Any database-read error for US can return a built-in active fallback (:122), potentially hiding a deliberate disable when the database becomes unavailable. Cache invalidation is process-local (:211), leaving other API processes with prior status for up to the TTL. This does not establish that financial mutations trust country configuration alone; backend payment enforcement must be tested independently. It establishes that picker/config state is not an instantaneous globally consistent emergency control.

T62-F04 - P2 [design requirement]: edits need versioned review and an operating owner. The UPDATE has no expected version or durable before/after administrative audit in this path (:204); controller logging records code/status/actor only. Headquarters ownership, country launch delegate, on-call escalation, and service withdrawal procedures are not represented in this tab. A country filter elsewhere cannot substitute for scoped market-management authority.

Expansion scenario. Two launch managers prepare a new market while a payment provider withdraws one local method and an incident commander disables registrations. A second API replica still serves active state, and a transient registry error triggers fallback. Existing customers still need access to receipts, support, and lawful/approved recovery processes, while new transactions may need to stop. A single active/disabled switch cannot silently encode all those distinct customer outcomes.

Recommended controls and UX. Extend the existing registry with typed configuration, immutable approved versions, reserved-key rejection, expected-version writes, before/after auditing, rollout timestamps, and separate browse/onboard/transact/support capability states. Add a launch checklist linking

provider test evidence, finance reconciliation, human jurisdiction review, translated documents, staffed SLA, and accountable owner. Publish cache invalidations or enforce version checks at sensitive consumer boundaries. Make fallback explicitly degraded and prevent it from authorizing launches or overriding emergency restrictions. Show effective version and replica propagation, dependency failures, rollback target, and localized accessible status actions. Provider approval and legal suitability remain human prerequisites distinct from engineering configuration.

Dependencies / owner. International Operations owns country launch/withdrawal; Platform owns registry validation/distribution; Finance/provider teams own payment readiness and recovery; Legal reviews jurisdiction applicability; SRE owns rollout health.

Measurable acceptance criteria. Reserved keys and unknown capabilities return 400. A stale edit returns 409 with no mutation. Activation without required reviewed evidence fails server-side. A synthetic three-replica test observes an emergency restriction within ten seconds, including database failure, while preserving support/receipt access. UI capability status matches direct API enforcement for each country/payment method. Every active market has a named owner, escalation SLA, versioned evidence, and rollback exercise; no mixed-currency value is inferred from a locale setting.

63. Corporate Setup - /admin/super/corporate/departments

Mission / what exists. Corporate Setup manages departments, titles, default permissions, and staffing context, linking to existing assignments and org-chart functionality. The page includes department/title editors, permission selection, role/workspace playbooks, country-specific staffing views, and assignment counts. The corporate service already resolves department defaults plus assignment overrides, supports country-filtered assignments, and prevents some delegated assignment/override escalation. Department/title/default writes are restricted to super-admin inside the controller, while routes also check manage_admin_accounts. Existing country/region fields and assignment lifecycle are real. Enterprise requirements should strengthen these shared workforce capabilities rather than duplicate the corporate hierarchy in HR or regional tools.

Evidence. client/src/admin/pages/super/CorporateDepartments.jsx:38 edits departments; :181 edits titles; :369 saves allowed defaults; :518 loads catalog/assignments; :539 derives staffing counts. server/routes/corporateRoles.js:21 protects catalog/read paths and :25 department mutations. server/controllers/corporateRolesController.js:115 restricts department edits; :160 replaces defaults; :332 lists assignments; :388 creates them; :519 updates under lock; :700 guards override escalation. server/services/corporateRolesService.js:62 resolves permissions, :105 filters country, and :155 handles overrides. server/migrations/246_corporate_hierarchy.sql:474 persists geographic/reporting/lifecycle assignments and :497 adds active-assignment indexes. server/__tests__/adminPermissionScope.test.js:23 proves the separate admin grant boundary, not comprehensive corporate scope or workforce lifecycle behavior.

Findings. T63-F01 - P1 [confirmed defect]: staffing counts are derived from a truncated assignment list. The page loads active assignments without requesting an aggregate (CorporateDepartments.jsx:524) and counts that array by title/department/country (:546). The controller caps results at 500 without paging continuation (corporateRolesController.js:355). With 3,500 employees and potentially multiple assignments each, displayed staffing can materially undercount and omit countries. The underlying data is not necessarily lost; the workspace's representation is incomplete.

T63-F02 - P1 [code observation]: effective corporate permissions are not fully bound to target geography/lifecycle. The resolver accepts country but no target region, unions active assignments, and does not check department/title activity or start/end effective times in the inspected assignment predicate (corporateRolesService.js:107). requireCorporatePermission.js:53 derives country from caller query/header. These observations are not proof of a bypass in every downstream route; each consumer needs target-record authorization. The stored region field alone does not enforce regional least privilege, and turning a department inactive cannot be assumed to revoke access immediately.

T63-F03 - P1 [confirmed defect]: unknown default permission codes are silently ignored while success echoes the submitted set. setDepartmentDefaults maps known codes, deletes existing defaults, skips unknown codes (corporateRolesController.js:193), then returns the incoming permissions (:204). An operator can believe a permission was applied when it was not, or unintentionally replace a complete set with a malformed partial one. Full-set replacement also lacks expected-version comparison and dedicated before/after audit in this path, affecting every employee sharing those defaults.

T63-F04 - P2 [verification gap]: workforce changes need impact simulation and distributed revocation evidence. Permission and catalog caches are process-local, with invalidation at mutation time and TTL-based reads. No 3,500-worker access recertification, multi-replica revocation, restored-data hierarchy, or HR lifecycle rehearsal was performed. Existing seniority protections should be retained, but they do not by themselves document conflict-of-duty rules or country-aware manager delegation.

Expansion scenario. A global compliance department reorganizes 600 staff, retiring two titles and changing a default permission. Regional leads must review local assignments while Security removes access for departed employees. A 500-row sample makes some countries appear understaffed, and an obsolete permission code is silently skipped. A future-dated assignment should not open production access today. The process needs before/after impact and a reversible organizational plan without reviving terminated accounts.

Recommended controls and UX. Compute authoritative server-side counts distinct from assignment-row counts; paginate assignment drill-down and handle multiple roles explicitly. Resolve permissions against the target's country/region/tenant, effective interval, active title/department, and approved conflict-of-duty rules. Validate every permission code before replacing any defaults. Version changes, lock the relevant department, capture audited before/after, preview affected users and resulting permissions, and route high-impact updates through existing approval concepts. Add HR joiner/mover/leaver batch jobs, dry run, named owner, recertification due dates, replica revocation acknowledgement, and accessible permission explanations. Keep organizational labels localizable without changing stable authorization codes.

Dependencies / owner. HR Operations owns workforce truth and reporting lines; Security/IAM owns effective privileges and recertification; Corporate Operations owns catalog/playbooks; Platform owns resolver, versioning, counts, and jobs.

Measurable acceptance criteria. All 3,500 synthetic employees and multiple assignments reconcile by department/title/country. Unknown permission codes reject the entire update. Stale full-set edits return 409. Inactive departments/titles, future/ended assignments, and foreign target regions produce the adopted denied outcome through actual consumer APIs. Revocation reaches all replicas within ten seconds. A reorganization dry run lists every affected person and permission delta, and reviewed execution produces a complete auditable result manifest.

64. Mode Policies - /admin/super/mode-policies

Mission / what exists. Mode Policies already edits whether a mode requires platform approval, whether application requests auto-approve, which action codes are gated, and the application form schema. The UI displays updated time, dirty/save state, JSON parsing errors, accessible pressed-state toggles, and confirmation before turning approval off. Both admin policy list/update routes require super-admin. PostgreSQL persists policies with updated-at triggers; the service caches them for sixty seconds. This system deliberately separates choosing an account mode from performing gated activities. Consultant paid-booking checks are implemented in two controllers. Readiness work should align configurable promises with actual enforcement and manage changes safely, not duplicate the mode switch or approval service.

Evidence. client/src/admin/pages/super/ModePolicies.jsx:22 defines accessible toggles, :42 stages editor values, :60 parses/saves JSON, and :86 confirms gate removal. server/routes/modePlatformApprovals.js:27 protects policy APIs. server/controllers/modePlatformApprovalController.js:119 maps updates. server/services/modePlatformApprovalService.js:66 persists arbitrary provided fields, :128 evaluates a gate, and :153 validates submitted applications. server/controllers/consultantNetworkController.js:331 checks paid referrals; consultantSchedulingController.js:261 checks the consultant booking target. server/migrations/272_mode_platform_approvals.sql:19 defines policy state and :51 seeds known modes. server/__tests__/adminPaginationContracts.test.js:13 references the adjacent approval list; it does not establish policy downgrade/schema/enforcement acceptance tests.

Findings. T64-F01 - P1 [code observation]: editable action codes are not an enforcement inventory. The service accepts arbitrary action names, and the policy editor offers text input. Repository call-site inspection found active gate calls for consultant accept_paying_booking, while routes such as tool listing creation (server/routes/toolRentals.js:413) and job assignment (server/routes/jobs.js:48) do not install the generic gate in their route chains. No claim is made that all those activities should currently be gated or lack other restrictions. The concrete gap is that configuring a new helper/tool action cannot itself insert an authorization check into its mutation path.

T64-F02 - P1 [confirmed defect]: policy changes lack semantic validation and stale-edit rejection. updatePolicy accepts schema JSON/action arrays directly and unconditionally updates by mode (modePlatformApprovalService.js:80, :91). A syntactically valid schema such as {fields:42} is accepted; submission validation treats a non-array fields as empty (:155). Unknown field types/action codes and incompatible auto-approval combinations are not validated here. Two editors can overwrite each other, while enabling auto-approve bypasses independent review through request submission without the gate-off confirmation specifically shown by the UI.

T64-F03 - P1 [code observation]: assurance decisions can survive missing policy or stale policy caches. canPerformAction allows when no policy row exists (:130) and approves based on an existing user/mode status without binding it to a policy version (:135). Cache invalidation affects only the writing process. The generic middleware also catches service errors and calls next() (server/middleware/requirePlatformApproval.js:56), but no route installation of that middleware was found in the inspected route inventory; this is dormant-code risk, not evidence that active consultant callers fail open on database exceptions. Error policy must be assessed per actual consumer.

T64-F04 - P2 [design requirement]: country applicability, review ownership, and policy evidence need explicit modeling. Current policies are global per mode and hold updated-by/time but no immutable reviewed revision, applicability scope, change reason, reviewer, or adoption date. This may suffice for today's narrow consultant gate. Worldwide approval policy requires a documented connection to country packs, provider readiness, staff review capacity, and human jurisdiction review.

Expansion scenario. A country launch needs additional consultant evidence, while another market pauses one paid activity because a provider capability is unavailable. Headquarters edits schema/action text and assumes all user actions now enforce it. Existing approved users retain access, and another replica still holds the older policy. Operations must know which endpoints enforce the published revision and which applicants need re-review, without globally preventing harmless mode browsing.

Recommended controls and UX. Build an action registry from actual checked mutation paths and let the editor select only supported actions; show endpoint coverage and explicit unsupported configuration.

Validate schema structure/types/options/bounds and locale labels before publication. Add expected-version updates, audited immutable revisions, change reasons, reviewed downgrades/auto-approval, staged effective time, impact preview, and policy-version binding for approvals. Define safe availability/assurance behavior per action; protected financial/provider actions should not silently become permitted due to missing configuration. Propagate invalidations and expose effective versions. Reuse country packs for applicability and existing approval jobs for re-review migration, with precise explanations and accessible localized forms.

Dependencies / owner. Trust/Compliance owns admission policy and review SLA; Platform owns action registry and atomic revisions; Finance/provider teams approve financial-capability dependencies; Legal reviews jurisdiction requirements; SRE owns propagation evidence.

Measurable acceptance criteria. Unsupported actions/schema types return 400 without mutation. Stale edits return 409. Every published gated action has a direct API denial test for unapproved users and explicit reviewed admin exceptions. Downgrades and auto-approval changes require the adopted independent approval. Three replicas converge on a revision within ten seconds. A new evidence revision produces a measurable re-review queue and never silently treats old evidence as new-provider approval.

65. Mode Approvals - /admin/super/mode-approvals

Mission / what exists. Mode Approvals is the implemented super-admin review queue for submitted mode applications. It shows applicant name/email, mode, submission time, application fields, approve confirmation, and rejection reason entry. The API provides bounded pending-list parameters, approve/reject/revoke actions, applicant status endpoints, schema validation, and automatic approval when the policy permits it. PostgreSQL has one row per user/mode plus status/request-time indexes. Approval affects specific gated activities rather than granting the entire mode switch. The enterprise requirement is a governable case workflow with durable decisions, delegation, concurrency control, and applicant/provider evidence, extending these existing mechanisms.

Evidence. client/src/admin/pages/super/ModeApprovals.jsx:19 renders a case, :26 approves, :36 requires a UI rejection reason, :119 renders submitted fields, and :148 loads pending applications using only a mode filter. server/routes/modePlatformApprovals.js:31 requires super-admin and exposes revocation. server/controllers/modePlatformApprovalController.js:139 clamps list limit/offset; :154 maps decisions. server/services/modePlatformApprovalService.js:195 upserts applications, :243 decides, and :278 lists requested records. server/migrations/272_mode_platform_approvals.sql:76 persists state, :93 enforces user/mode uniqueness, and :96 indexes status/time. server/__tests__/adminPaginationContracts.test.js:19 checks offset-clamping source text, not a complete queue or simultaneous reviewer test.

Findings. T65-F01 - P1 [confirmed defect]: the queue cannot browse beyond the first API page. The UI sends no limit/offset and reads only approvals (ModeApprovals.jsx:149). The controller defaults to 100 results (modePlatformApprovalController.js:144), and the response carries no total/continuation. The database indexes and server offset parameter exist, but neither makes later cases discoverable from this page. Processing the oldest cases one at a time may eventually expose newer ones; that is not a complete operational queue with accurate backlog visibility.

T65-F02 - P1 [confirmed defect]: decisions overwrite current state without transition/version validation. decideApproval updates any row by ID (modePlatformApprovalService.js:250) and does not require requested status for approve/reject or approved status for revoke. Two super-admins can overwrite decisions; a stale approval can restore an already revoked case. The API also permits rejection without a reason (:264), even though the UI requires one. Existing super-admin authorization prevents ordinary-user review but does not provide safe concurrent decision semantics.

T65-F03 - P1 [confirmed defect]: resubmission destroys prior decision provenance. The user request upsert overwrites status, application data, request/review timestamps, reviewer, notes, and rejection reason (:218) without restricting the prior state. Even an approved user can resubmit and become requested under a non-auto policy; a revoked user can enter automatic approval if policy now permits it. The single current row and logger messages do not preserve immutable application/decision versions. This is a current state-machine defect; it does not assert that any actual revoked user regained a paid activity.

T65-F04 - P2 [design requirement]: a worldwide review service needs scoped ownership and renewal. Every reviewer needs global super-admin; the screen has no case assignee, country/workspace jurisdiction, SLA age buckets, queue claim, delegation expiry, renewal deadline, or visible revocation/history controls despite a revoke API. Applicant-data rendering exposes arbitrary submitted fields and converts false/zero to an empty marker through String(val || '-') at ModeApprovals.jsx:126. Typed evidence and localized field labels need deliberate display rather than assuming every JSON value is an ordinary string.

Expansion scenario. A launch produces 12,000 consultant applications across five countries. Regional reviewers split evidence checks by language and jurisdiction, while headquarters handles exceptions. A reviewer approves a stale card after another reviewer revokes it for new evidence. The applicant resubmits corrected documentation, and an email-provider failure means they never learn the outcome. The queue needs a traceable sequence of submission, review, revocation, renewal, and delivery, not an overwritten row or a headquarters bottleneck.

Recommended controls and UX. Preserve the existing current-status lookup but add immutable submission/decision events tied to policy/evidence version. Require conditional transitions, expected version, reviewer assignment/claim, bounded nonblank reasons, and idempotent decision replay. Implement dedicated region/mode reviewer grants enforced from the applicant/case record; reserve global policy and

emergency authority for appropriate roles. Page/search/sort queues with authoritative counts, SLA buckets, assignee/language filters, and async low-risk batch decisions under a reviewed policy. Add renewal/revocation views, evidence links, applicant-visible outcome, durable notification outbox/retry, and conflict recovery preserving drafts. Display typed false/zero values accurately. Human provider/jurisdiction approval remains distinct from the software status.

Dependencies / owner. Trust/Compliance Review owns case policy, staffing, and SLA; Security owns scoped reviewer authority; Platform owns transitions/events/idempotency; Identity owns applicant lifecycle; notification/provider teams own delivery evidence.

Measurable acceptance criteria. All 12,000 synthetic cases are reachable with accurate backlog and oldest-case age. A stale approve cannot overwrite revoke and returns 409. Reasonless rejection fails server-side. Resubmission creates a new version while retaining every prior reviewer/outcome, and revoked cases cannot regain access outside the adopted policy. Regional direct API tests deny foreign-case decisions. Worker/notification failure resumes without duplicate decisions; 95% of ordinary cases meet the adopted 24-hour SLA. Zero/false fields, conflict messages, queue navigation, and decision dialogs pass accessible five-language review in an unskipped browser matrix.

Management capabilities required for the target company

These five reviews describe **missing or fragmented enterprise capabilities**, not five claims that OxSteed has no related code. Existing support staffing, Org Chart, Corporate Titles, Customer/Business 360, delivery dispatch/tracking, incident management, financial controls, legal/privacy, and credit sub-workspaces are credited below. Recommended route names are proposals, not live destinations. Build integrations and coherent workspaces over authoritative services; do not recreate payroll, accounting, payment ledgers, or legal systems without an explicit business reason.

Management page 1. Workforce Identity, Delegation, and Access Lifecycle

Proposed workspace: /admin/workforce-access. **Accountable owners:** Chief People Officer and Security/IAM, with regional employment administrators. **Scope:** the controlled lifecycle of 3,500 employees and contractors, not a replacement for the customer Users tab.

Existing foundation and precise gap

Admin Accounts, Permissions, Internal Access, Org Chart, Corporate Titles, Corporate Setup, support-agent onboarding, shifts, agents, and team hierarchy exist. The route inventory confirms those surfaces in client/src/admin/adminRouteManifest.js:50-51,74-87,132-136,145. Admin authentication checks active state and MFA, and permission middleware intentionally constrains existing delegated actions to global grants (server/middleware/adminAuth.js:43-71,126-177). Those controls are useful. The inspected admin authentication, permission, and account surfaces did not establish a complete enterprise SSO/SCIM employment lifecycle, synchronized HR status, regional object-level administration, or automatic cross-provider deprovisioning. This is a required capability to validate/build, not an assertion that no identity integration exists anywhere in the repository.

An organizational title is not an authorization policy. A person can simultaneously supervise a region, work a temporary support shift, and have narrowly approved financial-review rights. Customer identity, corporate identity, outsourced agent identity, driver identity, and service-account identity must be distinguishable. Staff moving teams or leaving the company must not retain rights merely because a browser has a cached menu or a provider account is still connected. E05 describes the global-grant limitation; solving it is a prerequisite for safe workforce delegation.

Required workflows and data

M01-F01, P1 [design requirement]: joiner/mover/leaver automation. Integrate with the chosen HRIS and identity provider. Keep an employment identifier, employer/legal entity, region, manager, team, employment state, effective dates, device/session inventory, and approved entitlement bundles. A hire should enter a staged identity with no production rights until training and required reviews are complete. A team transfer recalculates access; a departure disables authentication and revokes operational tokens and active sessions without deleting business audit evidence. Provide rehire and mistaken-termination recovery with independent approval.

M01-F02, P1 [design requirement]: least privilege and separation of duties. Define read, investigate, initiate, approve, export, and override capabilities separately, with object/region/entity scope and monetary limits. Finance reviewers should not be able to both create and approve their own payout; support supervisors should not gain credit reserve policy rights. Emergency access requires a reason, incident reference, expiry, and independent after-action review. Self-grants, circular approvals, expired delegations, and manager changes must have explicit rules. Backend authorization must run again when a queued action executes, not only when its draft was created.

M01-F03, P1 [design requirement]: access inventory and attestation. Show source-of-truth employment state versus application rights, last use, upcoming expiry, dormant privileged accounts, active sessions, and provider access. Schedule manager and security reviews with evidence of removal or accepted exceptions. Keep service identities separate, with owners and rotation/expiry policy. Federation outages need an independently secured recovery path, not a permanent shared super-admin password.

Operator design and expansion scenario

Use task-based access packages and scope chips, not a flat list of dozens of cryptic grants. An employee drawer should show current/effective/pending rights, why each exists, who approved it, when it expires, and a before/after preview. The account lifecycle queue should expose blocked onboarding, overdue approvals, failed provider revocation, and temporary duties ending during a shift. Bulk onboarding must preview eligibility and record per-person outcomes. Notifications must contain safe summaries rather than sensitive personnel files.

Consider 200 seasonal staff hired in three countries, ten supervisors changing regions, and an outsourced support vendor losing its contract on the same day. The system must onboard only approved duties, prevent managers from granting global privileges to solve a queue problem, and revoke the vendor across HTTP, sockets, exports, and connected tools. A removed operator cannot approve a previously opened financial dialog. A country director can delegate coverage for a holiday without exposing another country's conversations. Payroll and employment decisions remain in the approved HR system; OxSteed records the operational entitlement result and its evidence.

Acceptance and boundaries

Test all lifecycle actions for active sessions and delayed jobs. Proposed revocation objective: application HTTP and websocket rights cease within 60 seconds; external provider revocation failures remain in an owned exception queue until resolved. Load 3,500 employee identities and realistic entitlement histories. A region A operator must be denied region B list/detail/export/socket/actions. A maker, their temporary delegate, and a second account controlled by the same identity must not bypass declared conflict rules. Complete a quarterly access review in the tool with no spreadsheet-only exceptions. Identity outages, duplicate HR events, rehires, missing managers, and unknown region mappings must fail safely. These are acceptance gates, not current passing results.

Management page 2. Treasury, Entity Accounting, Tax, and Financial Close

Proposed workspace: /admin/finance-close. **Accountable owner:** CFO/Controller. **Supporting owners:** payments engineering, treasury, regional tax/accounting, risk, and external accounting-system administrators.

Existing foundation and precise gap

Money, Payments, Orders, Revenue, Financials, Payouts, reconciliation exceptions, fees/pricing, restaurant wallets, and the OxSteed Credit ledger/settlement/reserve/compliance/reconciliation pages exist. The manifest records the related financial routes at client/src/admin/adminRouteManifest.js:42-43,108-130. The tab reviews examine their actual totals, reporting bases, currencies, history, and recovery controls. This management workspace must connect them to entity books and close procedures rather than introduce another transaction ledger or call every gross movement revenue.

The enterprise gap is a finance-owned control process proving that balances, obligations, provider statements, bank cash, tax documents, and reports agree at a defined cutoff. Existing operational balance screens do not by themselves establish a multi-entity financial close. Worldwide participation also does not mean every credit or payout product is authorized everywhere; market approvals belong in Management page 5.

Required workflows and controls

M02-F01, P1 [design requirement]: entity and currency reporting contract. Every applicable financial movement needs its authoritative legal entity, native currency, product/channel, provider reference, posting/effective time, accounting treatment, and reversible linkage. Keep the original units and precision. A reporting-currency conversion must use an identified FX snapshot and rounding policy; it cannot sum CAD, USD, and JPY cents as one USD figure. Document whether Money, Revenue, Financials, and provider reports reflect recognized income, collected cash, customer liability, GMV, pipeline, or net margin. Reconcile differences instead of forcing all dashboards to show the same number.

M02-F02, P1 [design requirement]: close workflow and reconciliation ownership. Define daily settlement checks and monthly/quarterly close tasks by entity: provider-to-ledger, ledger-to-bank, outstanding authorizations, reserves/customer liabilities, refunds, disputes, failed payouts, commissions, taxes, and intercompany activity where applicable. Track reconciled/unmatched/ambiguous states, evidence, owner, aging, and materiality. A manual adjustment must be a reasoned compensating entry through existing authorized services, never SQL editing a posted row. Lock approved periods against casual restatement; reopen only under a controlled policy that records prior versions and changes.

M02-F03, P1 [design requirement]: treasury and payment authority. Show liquidity forecast, upcoming merchant/helper obligations, protected/customer funds according to the approved legal model, reserves, settlement delays, currency exposure, and provider limits. Separate initiator, approver, and executor; require independently verified bank-account changes and limits. Ambiguous timeouts must reconcile with the provider before retrying. Payout failure, return, dispute loss, refund after payout, and negative balance must have owned recovery procedures. Employee payroll and vendor accounting should integrate with the selected accounting/payroll platforms, not use customer payout controls as a substitute.

M02-F04, P1 [verification requirement]: tax and document lifecycle. Establish invoice/receipt numbering, entity identity, tax evidence, retention, correction/credit-note linkage, and required contractor/merchant reporting as determined by qualified regional advisors. Do not infer tax obligations from a Country Pack or a single tax-rate field. Test exports to the accounting system for duplicate delivery, mapping errors, missing references, changed chart-of-account versions, and reconciled totals. Track document issuance failures without pretending a PDF generated locally means the legal document was successfully delivered.

Operator design and expansion scenario

Organize the workspace by entity and close period, with a progress checklist, unreconciled balance, oldest exception, responsible reviewer, evidence completeness, and "cannot close because..." explanation. Provide a drill-through bridge from each total to authoritative entries and provider artifacts. Large exports should be asynchronous, resumable, access-controlled, and watermarked with scope/time/version. Show per-currency totals by default; the consolidated view must visibly disclose FX and eliminations. Separate payment-operation permissions from finance read-only analytics.

A regional controller closes the month while a provider webhook is delayed and a payout is returned after cutoff. A restaurant's restricted credit balance is a liability, not automatically revenue; a helper's receivable is not cash already settled. The close must record the timing difference and reopen/adjust only under policy. If two teams process the same exception, one authoritative outcome should result. A weekend treasury forecast should account for provider operating windows rather than promising instant transfers because an API call succeeded.

Acceptance and boundaries

Finance must sign off exact report definitions and a reproducible bridge between tabs, ledger, provider, bank, and accounting platform. In isolated fixtures, demonstrate complete lifecycle and duplicate/out-of-order cases for subscription, order, job, delivery extra, credit load/transfer/refund, payout return, and dispute. Reconcile every smallest currency unit or explicitly account for approved rounding entries. A closer cannot approve their own material adjustment or modify a locked period silently. Show native-currency totals and original FX evidence. An entity A reviewer cannot access B's bank details or financial exports. Accounting/tax/provider approvals are documented business gates, not claims that code alone satisfies them.

Management page 3. Global Service Operations, Capacity, and Safety

Proposed workspace: expand /admin/operations with scoped task workspaces and an enterprise service-control view. **Accountable owner:** COO. **Supporting owners:** delivery, restaurant operations, Direct, marketplace, rentals, consulting, support, trust/safety, and regional management.

Existing foundation and precise gap

This is explicitly **not a proposal to create a second delivery dispatch or customer CRM system**. client/src/pages/DeliveryDispatchPage.jsx, DeliveryTrackingOpsPage.jsx, client/src/admin/pages/Customer360.jsx, and Business360.jsx already exist. The route manifest includes incident handling, extensive support operations, logistics sub-workspaces, jobs/orders/quotes, and corporate organization (client/src/admin/adminRouteManifest.js:23-105). Operations Overview already combines domain signals; its findings identify query amplification, scope, freshness, ownership, and actionability gaps.

The target company needs coherent command, resource planning, safe escalation, and accountable handoffs across these domains. A restaurant order can be delayed by kitchen preparation, an unavailable runner, a payment ambiguity, or a customer-address problem; a generic "pending" count does not identify who can resolve it. A delivery may use OxSteed staff or an explicitly released marketplace helper while the customer sees one authorized lifecycle. The control plane must coordinate the existing owners, not blur fulfillment source, move payments outside approvals, or overload future /logistics with customer-delivery semantics.

Required workflows and controls

M03-F01, P1 [design requirement]: accountable exception queues. Each blocked or overdue engagement needs reason, customer-impact severity, scope, owner/team, due time, last action, allowed next actions, and links to evidence. Use explicit work-context references. Operators must claim work atomically, hand it over, and escalate under SLA. Two agents resolving the same issue must receive a state/version conflict, not silently overwrite one another. Escalation should reach the duty owner rather than always requiring the founder.

M03-F02, P1 [design requirement]: capacity and commitment control. Connect approved operating hours, available staff, driver/vehicle equipment, shared resource intervals, geographic coverage, capability matching, business calendars, and current demand. Do not advertise a deliverable time because a vehicle profile exists or because a marketplace helper might accept. Plans and tentative times remain distinct from commitments. Include approved fallback, backlog limits, service pause/resume, and overload rejection rules. Forecast staffing from actual workload and skills, with human review; headcount alone does not establish available qualified capacity.

M03-F03, P1 [design requirement]: safety and incident handling. Create controlled safety escalation for injury, threats, damaged/missing cargo, unsafe delivery access, fraud, food-quality concerns, and provider/merchant incidents. Record evidence, immediate protective actions, dispatch/customer communication, required qualified review, and restricted access. Jurisdiction-specific notification and insurance requirements must be determined by approved owners. Ordinary support staff cannot silently settle a safety incident by refunding money and deleting its history. Location tracking must have justified access, consent/retention policy, stale-location cues, and missing-data states; exact coordinates cannot become an unrestricted admin feed.

M03-F04, P2 [design requirement]: global handover and partner quality. Provide follow-the-sun shift handover, language/skill routing, unassigned cases, service quality trend, partner readiness, business coverage, equipment maintenance, and overdue compliance documents. Extend existing support shifts/team tools rather than duplicate rosters. Maintain operating playbooks for no-shows, late fulfillment, inaccessible addresses, outages, failed proof, returns, compensation, and customer disputes. Pause affected products/regions explicitly while keeping unrelated work operating.

Operator design and expansion scenario

Offer role-specific workspaces with an attention queue, map only where needed, timeline, owned next actions, and scope/timezone chips. A dispatcher sees tasks they can route; a support agent sees verified customer communication and recovery paths; regional leadership sees capacity/quality without unrestricted private messages or wallet controls. Display source timestamp, predicted versus confirmed ETA, uncertainty, and "cannot verify availability" rather than fabricated precision. Related records must retain their own record-level authorization.

During a severe weather event, internal drivers become unavailable in one region and hundreds of restaurant handoffs are delayed. Operators must pause new unsupported commitments, safely release eligible deliveries to the approved pool, notify customers consistently, triage perishables, preserve base-price snapshots, and route extra-charge requests for customer approval. Cases inherited at the next shift must retain ownership and evidence. A marketplace helper cannot see or claim unreleased internal work. A regional outage must not cause a global shutdown simply because all dashboards share a counter.

Acceptance and boundaries

Demonstrate complete journeys for jobs, Direct, rental, consulting, food/credit, and delivery, including abandonment and every permitted recovery. Under 1,500 active staff, shared assignment/claim races produce one owner and actionable conflicts; permissions remain regional and field-specific. Test overload, shift gaps, DST, unverified capacity, missing tracking, rejected proof, safe cancellation/return, and provider ambiguity. A blocked engagement always lands in an owned queue with a deadline. Customer-facing fulfillment remains source-neutral except required disclosures, and no routing action silently changes quote price, payment approval, or authoritative stock. Regional duty staff can operate for 24 hours without direct database edits or founder intervention.

Management page 4. Reliability, Releases, Integration Recovery, and Cost

Proposed workspace: /admin/platform-control, with separate observation, operational, and emergency privileges. **Accountable owner:** CTO/SRE. **Supporting owners:** security, database, integrations, finance operations, and product engineering.

Existing foundation and precise gap

Readiness, release metadata, audit health, operation summaries, incidents, reconciliation exceptions, credit health, durable workflow/outbox mechanisms, migration CI, backups, and recovery scripts exist. server/services/readinessService.js:45-127 establishes initialization/schema/dependency checks. client/src/admin/adminRouteManifest.js:23,114,118,130 records incidents and recovery-related routes. scripts/backup-coolify-production.sh and verify-production-restore.sh provide operational artifacts. The gap is **a verified, independent, governed control surface and operating procedure**, not an absence of every retry or backup capability.

The same Mac/Colima deployment hosts important production dependencies, and browser acceptance skips without configuration. Those facts make E01, E06, E07, and E08 prerequisites. An attractive health page on the failed application host is not an independently available recovery console. A successful dump, a green HTTP probe, and a connected provider record are three different kinds of evidence.

Required workflows and controls

M04-F01, P1 [design requirement]: service and dependency inventory. Enumerate app/worker/database/storage/socket/queue/scanner dependencies and Stripe, maps, messaging, AI, calendar, tracker, and other providers, with owner, environment, allowed products/countries, authentication setup, capacity/quota, expiry, last successful probe, and relevant SLO. Show connected, unconfigured, unsupported, degraded, and revoked states separately. Suppress secret values; privileged credential maintenance should use the approved secret workflow, not a general admin export.

M04-F02, P1 [design requirement]: governed recovery and replay. Expose existing failed events, webhook results, outbox deliveries, and reconciliation runs with safe correlation references, retry history, queue age, owner, and reason. Classify replayable read-model/notification failures separately from irreversible financial, stock, or state transitions. Retry must retain original idempotency identity and reauthorize the action. Ambiguous provider outcomes require reconciliation before resending. Large replays need preview, scope/volume limits, approval, throttling, progress, cancellation, and evidence of each result. Poison events must not starve unrelated tenants.

M04-F03, P1 [design requirement]: tested change/release control. Record immutable artifact, commit, schema compatibility, configuration version, approvals, environment, canary outcomes, and rollback criteria. Native deployment must not outrun required CI. Rehearse migrations on redacted restored data with target-scale cardinalities, resource budgets, and representative index/lock effects. Old and new application versions must coexist safely during rollout. Configuration changes need staged validation, before/after effect estimates, and rollback rules; restoring an old app cannot erase committed financial transactions.

M04-F04, P1 [verification requirement]: independent disaster recovery and resource budgets. Record backup scope, encryption/access separation, retained recovery points, copy location, restore test evidence, recovery objectives, and operator readiness. Restore database, media, mappings, and configuration on a separate environment; reconcile external provider state before resuming jobs. Track database connection/lock/query load, disk/WAL growth, object retention, websocket pressure, worker lag, vendor quota/spend, and AI budget. A daily spend anomaly must have a kill/degrade option that preserves customer commitments and evidence.

Operator design and expansion scenario

Use a service map and severity-sorted attention queue, not a wall of green icons. Link every symptom to affected business journeys and data timestamps. On-call operators need a precise runbook, safe command preview, emergency contact, and change record. High-risk restart/freeze/replay operations must identify scope, expected customer effect, prerequisites, independent approval, and post-checks. Read-only staff can observe without obtaining production secrets or write permissions. Export incident evidence with redaction and immutable release identifiers.

Suppose a host fails while Stripe has captured money but the app has not recorded its webhook, and the customer has uploaded proof to local storage. A restore that brings back only SQL can leave missing proof; replaying every payment request can charge twice. The recovery plan must reconstruct safe durable state, reconcile provider outcomes, recover media, verify authorization, and selectively resume workers. If an administrator loses the application itself, the independent incident channel and infrastructure access must still work. Mass customer reconnect must be capacity-tested rather than treated as normal steady traffic.

Acceptance and boundaries

Perform a two-day minimum steady/peak soak plus controlled host/node/provider/queue/disk/credential-expiry tests before target-scale launch; extend duration according to traffic and periodic job cycles. Recover from a fresh independent host within the approved RTO/RPO, with audited money/stock/message/proof checks. Record one safe replay of each consumer class, one refused unsafe replay, and one provider ambiguity resolved without duplication. A deliberately failing release cannot promote. Mandatory browser tests cannot silently skip. Finance/security approve action thresholds; SRE records verified capacity and runbooks. Continuing synthetic checks and restore exercises remain necessary after launch.

Management page 5. Market Governance, Privacy, Contracts, and Policy Assurance

Proposed workspace: /admin/governance, combining controlled views over existing domain approvals. **Accountable owners:** Legal/Compliance, Privacy, Security, and regional business leadership. **Supporting owners:** finance, trust/safety, HR, logistics, food operations, and product.

Existing foundation and precise gap

Legal acceptances, legal audit/metrics/privacy-request routes, country packs, regions, internal approvals, business modes, content-removal/moderation, credit compliance/partners, and logistics compliance review exist. See client/src/admin/adminRouteManifest.js:70-73,98-99,111-123,134-148 and server/services/complianceCaseService.js. Those are actual starting points, not missing modules. The enterprise gap is a cohesive evidence and policy lifecycle governing which legal entity can offer which product in which market, who approved it, and how privacy/contract obligations are fulfilled across all authoritative stores and providers.

The mere presence of a country configuration, merchant agreement, payment currency, or untranslated page cannot be treated as a worldwide launch approval. Stored-value products, cards, restaurant wallets, delivery coverage, contractor arrangements, marketing consent, and data processing can each have different restrictions. This report does not determine jurisdiction-specific legal obligations; qualified owners must document them and engineering must enforce their decisions.

Required workflows and controls

M05-F01, P1 [design requirement]: market/product readiness register. Record entity, jurisdiction/service area, supported products, provider approvals, required agreements, approved policy version, insurance/operational coverage, settlement capability, languages, escalation contacts, responsible approvers, expiry, and evidence. Model disabled, preparation, approved, limited pilot, suspended, and retired states. Approval is scoped; enabling restaurant ordering must not automatically enable unrestricted credit, freight, external cards, or cross-border delivery. Domain APIs enforce the approved capability set, not just hide a navigation link.

M05-F02, P1 [design requirement]: privacy request and retention orchestration. Extend existing privacy workflows with identity verification, explicit data inventory, scoped retrieval, time-bound export, deletion/anonymization, retention exceptions, legal holds, provider propagation, and completion evidence. Financial/audit obligations may require retaining minimal evidence while removing ordinary profile data; approved policy decides how. Include media, chats, logs, search histories, trackers, marketing tools, caches, analytical stores, and backups according to realistic deletion/recovery rules. A deletion request is not complete merely because the Users table changed.

M05-F03, P1 [design requirement]: agreement and policy lifecycle. Track document type, contracting parties, version, effective/expiry dates, country/language, signature/acceptance evidence, covered product, renewals, approved variations, and fulfillment obligations. Policy changes need impact analysis and a communication/reacceptance plan where required. Historical engagements retain their applicable version. Provider, franchise, business, consultant, employee, and customer agreements are different controls. Use an approved contract/document platform where practical, with explicit links rather than a parallel file store hidden in an admin note.

M05-F04, P1 [verification requirement]: control assurance and sensitive investigations. Link compliance cases, fraud/safety/security incidents, payment disputes, content appeals, and regulatory inquiries through authorized references. Assign owners and deadlines, retain evidence under access restrictions, and protect against a user reviewing their own case. Require review of sensitive exports and bulk decisions. Keep autonomous AI away from final legal, employment, financial, or access decisions unless separately approved and bounded. Audit not only changed records but sensitive reads, attempted prohibited actions, evidence exports, and emergency access.

Operator design and expansion scenario

Provide market/product readiness matrices, expiry queues, privacy case timelines, contract obligations, outstanding policy approvals, control-test evidence, and exceptions with accountable owners. Use plain-language blockers: "delivery paused: required coverage expired" or "credit unavailable: provider approval pending," not a global boolean that implies every feature is ready. Sensitive information should be masked until a verified task needs it. Display applicable country/entity/language and the historic policy basis on decisions. Bulk renewal or policy migration needs previews and resumable per-record outcomes.

A country launch is approved for discovery and domestic delivery but not the credit rail. A merchant's coverage expires, a user requests erasure while a dispute is open, and a regional regulator asks for selected records. Operators must pause only affected capabilities, retain required minimal dispute evidence under a documented hold, safely export authorized records, and avoid leaking private conversations to unrelated reviewers. When the coverage issue is resolved, resumption needs a new approved record, not merely editing the visible badge. The founder should not be the only person who knows which promises are permitted.

Acceptance and boundaries

Demonstrate server rejection for an unapproved market/product/provider combination, including direct API and delayed-job attempts. Run a verified privacy case across representative DB/media/log/provider stores and prove completion or explicitly justified retention exceptions. Test policy changes, expiry, suspension, appeal, duplicate request, identity mismatch, provider failure, and recovery from backup without resurrecting prohibited active data. A contract/history view must preserve the originally applicable version. Governance review must be signed by qualified business/legal/security owners with expiration and evidence; code tests alone cannot certify legal compliance.

Additional functionality backlog and build-versus-integrate decisions

Prioritize capabilities by business commitments and risk, not how many tabs can be added. Each item below is a **requirement to assess/extend**, not a verified claim that every supporting function is absent.

- **Enterprise customer/partner management:** extend Customer 360/Business 360 with account ownership, commitments, contract scope, case history, health, renewal, escalation, and field-level access. Integrate a CRM if sales account management exceeds the existing operational record model.
- **Fraud/risk investigation:** linked identity/device/payment/order/credit/delivery signals, evidence provenance, rule version, reviewed outcomes, appeals, and false-positive monitoring. Rules need accountable owners and safe rollback; do not use opaque risk scores as sole authority for irreversible action.
- **Merchant/helper/provider readiness:** onboarding checklist, restricted versus ready capabilities, payout prerequisites, expiring verification/coverage, remediation, and independently verified provider state. Extend Stripe setup and partner controls; a connected account is not automatically payout-ready.
- **Inventory/assets/maintenance:** driver equipment, Direct-owned tools, peer rentals, spare stock, custodians, inspections, maintenance, damage/loss, and retirement. Preserve ownership distinctions and authoritative inventory movements.
- **Commercial procurement/vendor lifecycle:** sourcing, supplier assessment, approvals, purchase commitments, receiving, invoices, disputes, and renewals. Integrate an ERP/procurement platform rather than stretching simple customer /delivery into unsupported freight/procurement.
- **Subscription/entitlement operations:** invoice failure, retry/grace rules, plan changes, proration, refund, cancellation, provider outage, duplicate webhook, and access revocation. Effective entitlements must derive from verified domain/provider state, not displayed catalog prices alone.
- **Business continuity/customer communication:** approved outage templates, service-area notices, consent-aware channels, message version, delivery outcome, localization, and incident follow-ups. Extend existing templates/messaging and prevent duplicate campaigns after worker retries.
- **Data/metrics catalog:** approved definitions, source lineage, freshness, access, retention, quality checks, entity/currency semantics, and consistent export totals. Keep analytical warehouses out of synchronous financial authorization.
- **Experiment and configuration governance:** audience/scope validation, safe defaults, staged rollout, monitoring, immutable versions, expiry, owner, and controlled rollback. A feature flag count is not evidence that exposure is safe.
- **AI oversight and cost:** provider/model version, evaluated use case, data minimization, human review, abuse limits, spend quotas, fallback, prompt-injection tests, and incident controls. Free-model availability is not an enterprise capacity guarantee.
- **Workforce scheduling and training:** qualifications, region/language, shift capacity, handover, required training, performance review, and QA. Integrate payroll/HR rather than calculate employee salaries with marketplace payout logic.
- **Document/evidence operations:** malware validation, storage policy, object ownership, expiry, redacted exports, signature/receipt provenance, independent restoration, and legal holds. A successful upload or scanned file is not proof of delivery validity by itself.
- **Mobile/offline operational reliability:** durable pending commands, idempotency, conflict resolution, stale GPS, battery/network loss, lost devices, permission withdrawal, and accessible status. Staff must distinguish queued changes from committed milestones.
- **Integration/partner API governance:** scoped credentials, rate limits, quotas, owner, version/deprecation policy, consent, replay/revocation, observability, and sandbox certification. A working connector key alone is not complete governance.

- **Franchise/enterprise partner assurance:** territory contracts, role delegation, brand standards, settlement obligations, controlled reporting, inspection exceptions, suspension/appeal, and renewals. Extend Franchise HQ with enforceable domain boundaries rather than trusting its map alone.

Verification contract: how the report becomes an operable company

Every-tab acceptance template

For **each of the 65 numbered tab reviews**, maintain an executable test record containing its manifest key, list/detail routes, data sources, scopes, sensitive fields, every visible action, allowed state transitions, and denial conditions. Cover authorized read; unauthorized HTTP/direct navigation/export/socket; valid action; invalid input; stale version; simultaneous operator conflict; duplicate request; provider timeout; unavailable dependency; recover/retry; audit evidence; and post-action authoritative refresh. Dependent detail pages and hidden workspaces are included, not just sidebar URLs. A missing test environment is a failed release prerequisite, not a pass.

Exercise roles: super-admin, global specialist admin, country/regional admin, entity/restaurant-scoped staff, support agent, dispatcher, internal driver, marketplace helper, business owner/manager/member, customer, guest, suspended/revoked/expired-grant user, and service account. Test all five UI languages with actual application language selection, Arabic RTL, light/dark themes, mobile and desktop, keyboard and screen readers, reduced motion and animation fallback. Browser locale alone may not change a saved application preference. Use Chromium, Firefox, and WebKit for critical workflows, and document any consciously unsupported environment.

Minimum isolated end-to-end scenarios: customer request to quote to approval to scheduled work to completion to receipt/payment to support; food discovery to customization to stock reservation to order/payment to kitchen/runner handoff to receipt/refund; restaurant credit request/load/QR/transfer/history and failure recovery; broader credit authorization/hold/settlement/refund/dispute/reconciliation with separate balances; delivery internal assignment or marketplace release/claim to pickup/transit/proof/delivery/extra approval/receipt/return; rental pickup/return/damage; consulting intake/session/invoice/recovery; subscription change/cancel/payment failure/entitlement; account recovery/suspension/deletion/appeal; access joiner/mover/leaver; regional launch pause/resume; incident/outage/restore/reconciliation. The relevant authoritative service decides the outcome in each case.

Data, concurrency, and recovery testing

Generate six million synthetic users and realistic relationships/history without copying raw production personal data. Model hot tenants, popular restaurants/helpers, old disputes, large wallet histories, multi-currency/entity records, high audit volumes, rare unsupported statuses, incomplete legacy data, and unusual timezones. Confirm fresh migrations already passing, then add redacted restored-data rehearsal, migration interruption/restart, mixed app versions, forward/rollback compatibility, long-lock budgets, and independent restore. Verify index/query plans and cost at representative scales rather than extrapolating from four production users.

Run concurrent operator/customer races for every scarce or financial resource: delivery ownership, helper claims, driver/vehicle windows, booking slots, stock, quote acceptance, credit holds/limits, payouts, approval checker identity, and case assignment. Crash between provider success and local commit; deliver webhooks twice and out of order; expire a worker lease; reconnect clients to another node; revoke an operator while an export/action is queued. Assert no duplicate financial/stock effect, no stale-state overwrite, no cross-scope leakage, no fabricated proof, and a recoverable owned exception.

Benchmark the declared capacity model with workload distribution, hardware/topology, latency percentiles, error rates, queue growth, DB waits, provider quotas, memory, CPU, socket reconnect, and storage growth published. Include at least 48 hours of soak and periodic jobs; longer validation may be needed to exercise weekly/monthly cycles. A short peak test does not certify the future company. Resilience tests should lose an independent compute node and provider, not destructively kill the sole production database with customers active.

Release and operating gates

Gate A - Integrity/security: all P0 and applicable P1 ownership, payment, proof, authorization, privacy, and idempotency issues closed with tests. **Gate B - Scope/operability:** every current tab and supporting workflow has approved specialist ownership, server scope, failure handling, and a trained recovery path. **Gate C - Capacity/continuity:** measured workload targets, independent failure domains, restored-data migration proof, and independent disaster restore meet agreed objectives. **Gate D - Market/business:** finance/legal/provider/operations approvals are complete for the actual products and countries being launched. **Gate E - Release evidence:** mandatory CI/browser scenarios run without silent skips; exact tested artifact/configuration/schema is promoted; canary and rollback are exercised.

Roll out in bounded markets and traffic tiers. Validate at current demand first, then agreed expansion thresholds (for example 10%, 25%, 50%, 100% of the design peak), with stop conditions for integrity failures, sustained lag, scope leaks, or recovery-objective violations. Pilot limits are business exposure limits, not permission to bypass correctness. Publish queue/service freshness and periodic access reviews, restore exercises, dependency-expiry checks, metric review, and capacity reassessment. These ongoing controls are what prevent the same class of issue from returning unnoticed.

Ordered delivery program and closure evidence

Stage 1: establish truth and safe gates. Confirm source/deployed identity, mandatory staging/browser execution, domain route/action coverage, defect reproductions, shared permission boundaries, and complete evidence inventory. Owners: engineering lead, QA, Security, SRE. Outputs: verified backlog with finding IDs, scoped test accounts, release manifest, and reproducible failures. Do not expand permissions just to make tests pass.

Stage 2: correct irreversible and sensitive workflows. Close money, credit, assignment, proof, state/version, privacy, and access lifecycle issues before visual polish. Preserve authoritative ledgers and recovery histories. Owners: domain engineering with finance/risk/security reviewers. Outputs: negative/concurrency/provider-ambiguity evidence, approval rules, and runbooks. A business owner signs that failure cases can be resolved without SQL or founder intervention.

Stage 3: build resilient shared infrastructure and read models. Separate workloads, add tested shared realtime/presence, worker ownership/fencing, DB budgets/query isolation, independent backup/restore, promotion controls, scoped aggregates, and operational SLOs. Owners: SRE/database/platform/data teams. Outputs: capacity results, independent recovery evidence, restore-point records, and safe release drills. Avoid a premature architecture rewrite of every domain.

Stage 4: complete enterprise management and operator design. Integrate HR/identity, accounting/treasury, global service control, reliability, and governance workspaces described above. Apply consistent accessible grids, task-based navigation, status clarity, translation QA, and owned exceptions to all tabs. Owners: business domain heads and admin UX. Outputs: usability evidence, role-specific training, handover procedures, finance/legal approvals, and closed per-tab acceptance records.

Stage 5: certify the defined deployment envelope. Independently review security and finance controls; run complete journeys, restores, regional scope tests, long-running load/resilience checks, and limited-market pilots. Owners: accountable executives, QA, Security, SRE, Finance, Legal/Compliance. Outputs: an explicit approved capacity/market/product envelope, remaining accepted P2/P3 risks, expiration/review dates, and signed launch decision. Do not describe unresolved P1 blockers as completed enhancements.

Each finding's closure record must include source revision, exact behavior changed, test/trace evidence, migration/configuration dependency, rollout state, rollback/recovery path, business owner approval, and any residual exposure. A merge or passing unit test alone does not close an operational risk. An accepted deferral needs a bounded customer/employee/market exposure, compensating controls, accountable approver, and review date. No report can honestly promise the business will never encounter a new scenario; this program provides the controls to identify and handle new scenarios without unsafe improvisation.

Final certification decision

Healthy observed deployment: yes. Current CI success: yes. Fresh migration runner repeatability: yes. All admin business actions executed and verified: no. Six-million-user capacity demonstrated: no. Worldwide workforce/financial/market readiness certified: no.

The business can expand only within a verified operating envelope. The path to the requested full-company capability is the per-tab remediation, shared enterprise controls, five management workspaces, and measurable gates above. Until those gates are satisfied, neither this report nor the present green deployment should be represented to employees, customers, partners, or investors as proof that every operational, financial, legal, or failure scenario is covered.